

Hola Claude desde vscode01.jj@gmail.com

Tengo que crear un README para el equipo.

El directorio es:

https://github.com/Bootcamp-IA-P6/P8_Data_Analysis.git

Antes de hacer un push a main se hacen pruebas en las ramas feature y develop.

Las instrucciones son clonar:

https://github.com/Delo-sangeles/Data_Analysis.git

Crear README.md

Intentar ejecutar en mi máquina el código.

Contiene lo que se subirá a P8_Data_Analysis.git

¿Empiezo con

git clone https://github.com/Delo-sangeles/Data_Analysis.git

?.

Gracias Claude desde vscode01.jj@gmail.com

20260501045623*

Hola JJ,

Sí, git clone https://github.com/Delo-sangeles/Data_Analysis.git es exactamente el primer paso correcto. Pero antes de ejecutarlo, una recomendación rápida: clónalo en un directorio de trabajo aparte (no dentro de tu repo personal P8-Data-Analyst ni del clon del equipo), para que sea solo una zona de inspección/validación.

Algo así:

```
bash
```

```
mkdir -p ~/work/p8-staging && cd ~/work/p8-staging
```

```
git clone https://github.com/Delo-sangeles/Data_Analysis.git
```

```
cd Data_Analysis
```

```
ls -la
```

Una vez clonado, antes de saltar al README, te sugiero esta secuencia (tu patrón habitual de "ejecuta → pega salida → seguimos"):

1. **Inspeccionar estructura** – `ls -la, find . -maxdepth 3 -type f -not -path './.git/*'`, ver si hay `pyproject.toml`, `requirements.txt`, `notebooks`, `datasets`, `scripts`, etc. Pega la salida y la analizamos juntos.
2. **Intentar reproducir el entorno** con `uv` (tu stack):
 - Si hay `pyproject.toml` → `uv sync`
 - Si solo hay `requirements.txt` → `uv venv && uv pip install -r requirements.txt`
 - Si no hay nada → habrá que deducir dependencias mirando los imports.
3. **Probar a ejecutar** lo principal (notebook con `uv run jupyter lab` o el script de entrada) y registrar qué funciona y qué falla.
4. **Solo entonces** redactamos el README, porque va a describir cosas reales: cómo levantar el entorno, qué hace cada notebook/script, qué datasets se esperan, salidas, etc. Hacerlo antes de ejecutar suele acabar en un README idealizado que no coincide con la realidad.

Una pregunta antes de seguir: ¿el README que tienes que crear va dentro del clon de Delo-sangeles/Data_Analysis (para luego copiarlo/PR-earlo al repo del equipo en una rama feature), o va directamente sobre tu clon local de Bootcamp-IA-P6/P8_Data_Analysis en una rama feature? El flujo es ligeramente distinto:

- **Caso A** (README en el repo origen): trabajas en el clon de Delo-sangeles, generas el README ahí, y luego ese contenido se integra al repo del equipo.
- **Caso B** (README directo en repo equipo): clonas Delo-sangeles solo como referencia/inspección, pero el README lo creas en tu working copy de P8_Data_Analysis en `git checkout -b feature/readme` → `push` → `PR a develop`.

Cuéntame cuál de los dos y arrancamos. Mientras, puedes ir lanzando el `git clone` y pasarme la salida de `ls -la` y de `find . -maxdepth 3 -type f -not -path './.git/*'`.

Hola Claude desde vscode01.jj@gmail.com

```
git clone https://github.com/Delo-sangeles/Data_Analysis.git
cd Data_Analysis
```

```
01>_ Shell      Coder@F5-LAPMAD-124 MINGW64 /
02>_ Shell      $ cd ~/Proyectos/
03>_ Shell
04>_ Shell      Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos
05>_ Shell      $ git clone https://github.com/Delo-sangeles/Data_Analysis.git
06>_ Shell      Cloning into 'Data_Analysis'...
07>_ Shell      remote: Enumerating objects: 60, done.
08>_ Shell      remote: Counting objects: 100% (60/60), done.
09>_ Shell      remote: Compressing objects: 100% (41/41), done.
10>_ Shell      remote: Total 60 (delta 11), reused 58 (delta 9), pack-reused 0 (from 0)
11>_ Shell      Receiving objects: 100% (60/60), 78.47 KiB | 1.03 MiB/s, done.
12>_ Shell      Resolving deltas: 100% (11/11), done.
13>_ Shell
14>_ Shell      Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos
15>_ Shell      $ mv Data_Analysis Data_Analysis.Delo-sangeles
16>_ Shell
17>_ Shell      Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos
18>_ Shell      $ cd Data_Analysis.Delo-sangeles/
19>_ Shell
20>_ Shell      Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (main)
21>_ Shell      $ ls
22>_ Shell      README.md
```

He cambiado el nombre para más claridad a Data_Analysis.Delo-sangeles.
No voy a hacer push a ese repo.
Debo mostrar lo que he hecho.
¿Vemos que ramas hay?.

Gracias Claude desde vscode01.jj@gmail.com

20260501051656*

Perfecto, buen movimiento renombrarlo a Data_Analysis.Delo-sangeles – deja claro de dónde viene y evita confusión con el repo del equipo.

El hecho de que main solo tenga README.md es muy sospechoso: probablemente el código real vive en otras ramas (develop, feature/...) que aún no se han mergeado. Vamos a verlo:

bash

1. Ver todas las ramas (locales y remotas)

```
git branch -a
```

2. Ver el grafo completo de commits en todas las ramas

```
git log --all --oneline --graph --decorate --max-count=30
```

3. Ver qué archivos existen en cada rama remota (sin cambiar de rama)

```
for branch in $(git branch -r | grep -v HEAD | sed 's/origin/\\/'); do
```

```
  echo "=== Rama: $branch ==="
```

```
  git ls-tree -r --name-only "origin/$branch" | head -50
```

```
  echo ""
```

```
done
```

Qué esperar de cada uno:

- **git branch -a** te listará algo como main, remotes/origin/main, remotes/origin/develop, remotes/origin/feature/xxx... Si solo aparece main, significa que el repo realmente solo tiene esa rama y el código está en otro sitio (o aún no se ha subido).
- **git log --all --graph** te muestra el historial completo y verás si hay ramas con commits paralelos, merges, etc.
- **El bucle for** lista los archivos de cada rama remota sin necesidad de hacer checkout – útil para detectar dónde está el código real.

Pasa la salida y vemos:

- Si hay una rama develop con el código → la siguiente acción es git checkout develop (o git switch develop) y reproducir desde ahí.
- Si solo hay main con un README casi vacío → tendremos que preguntar al equipo / a Delo-sangeles dónde está el código, porque sin código no hay nada que reproducir ni que documentar en tu README.

Lánzalo y me pegas las tres salidas.

Hola Claude desde vscode01.jj@gmail.com

```
01>_ Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (main)
02>_ Shell $ git branch -a
03>_ Shell * main
04>_ Shell remotes/origin/HEAD -> origin/main
05>_ Shell remotes/origin/develop
06>_ Shell remotes/origin/main
07>_ Shell
08>_ Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (main)
09>_ Shell $ for branch in $(git branch -r | grep -v HEAD | sed 's/origin/\\/'); do
10>_ Shell echo "=== Rama: $branch === date $(date +%Y%m%d%H%M%S)"
11>_ Shell git ls-tree -r --name-only "origin/$branch" | head -50
12>_ Shell echo ""
13>_ Shell done
14>_ Shell === Rama: develop === date 20260501052402
15>_ Shell .env.example
16>_ Shell .eslintrc.cjs
17>_ Shell .gitignore
18>_ Shell BACKEND_EXAMPLES.md
19>_ Shell Dockerfile
20>_ Shell README.md
21>_ Shell SETUP.md
22>_ Shell docker-compose.yml
23>_ Shell index.html
24>_ Shell package-lock.json
25>_ Shell package.json
26>_ Shell postcss.config.js
27>_ Shell src/App.tsx
28>_ Shell src/components/Charts.tsx
29>_ Shell src/components/MapChart.tsx
30>_ Shell src/components/Navbar.tsx
31>_ Shell src/components/footer.tsx
32>_ Shell src/hooks/useApi.ts
33>_ Shell src/main.tsx
34>_ Shell src/pages/Dashboard.tsx
35>_ Shell src/services/apiClient.ts
36>_ Shell src/styles/App.css
37>_ Shell src/styles/index.css
38>_ Shell src/utils/formatters.ts
39>_ Shell tailwind.config.js
40>_ Shell tsconfig.json
41>_ Shell tsconfig.node.json
42>_ Shell vite.config.ts
43>_ Shell
44>_ Shell === Rama: main === date 20260501052402
45>_ Shell README.md
46>_ Shell
47>_ Shell
48>_ Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (main)
49>_ Shell $ git log --all --oneline --graph --decorate --max-count=30
50>_ Shell * 6e23c73 (origin/develop) Frontend Ready
51>_ Shell * 4a76d75 Ready aplicacion
52>_ Shell * 9709aca Page web of Analysis
53>_ Shell * 9290d5f (HEAD -> main, origin/main, origin/HEAD) first commit
54>_ Shell * 0e86814 first commit
55>_ Shell
56>_ Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (main)
57>_ Shell $
```

Gracias Claude desde vscode01.jj@gmail.com

20260501052931*

¡Hallazgo clave! Aquí lo importante:

El código real está en **develop**, no en **main**. En **main** solo hay un README placeholder ("first commit"). En **develop** están los 3 commits con el contenido real (Page web of Analysis → Ready aplicación → Frontend Ready). El flujo del repo de Delo-sangeles es: trabaja en **develop** y aún no ha mergeado nada a **main**.

Y un dato importante de contexto: mirando los archivos (package.json, vite.config.ts, tailwind.config.js, src/App.tsx, src/components/Charts.tsx, Dockerfile)... esto **no es código Python de análisis**. Es un **frontend en React + TypeScript + Vite + Tailwind**, con Docker. O sea, es la **UI del dashboard** del proyecto, no las notebooks de EDA. Eso encaja con tu rol: tú haces el análisis (Python/Jupyter/Plotly) y este repo es la parte web que probablemente consumirá tus resultados o se integrará con el resto del equipo.

Esto cambia ligeramente las expectativas:

- Para ejecutarlo necesitas **Node.js + npm** (no uv).
- Ojo a **BACKEND_EXAMPLES.md** y **SETUP.md** y **.env.example** – pueden indicar que el frontend espera un backend o una API que aún no existe.

Próximos pasos sugeridos:

bash

```
# 1. Cambiar a develop (ahí está todo)
git switch develop
```

```
# 2. Leer la documentación que ya existe en el repo
```

```
cat README.md
echo "===== SETUP.md ====="
cat SETUP.md
echo "===== BACKEND_EXAMPLES.md ====="
cat BACKEND_EXAMPLES.md
echo "===== .env.example ====="
cat .env.example
```

```
# 3. Ver qué declara package.json (scripts, dependencias)
```

```
cat package.json
```

```
# 4. Comprobar si tienes Node.js disponible
```

```
node --version
```

```
npm --version
```

Pega las cuatro salidas y desde ahí decidimos:

- Si hay un **npm run dev** claro y todas las deps son frontend puro → hacemos **npm install + npm run dev** y vemos si arranca.
- Si el frontend exige un backend que no existe en este repo → lo documentamos como bloqueante en tu README (tu trabajo es reportar realidad, no inventarla).
- Si hay variables de entorno necesarias en **.env.example** → las anotamos.

Una pregunta organizativa antes de seguir: ¿el README que tienes que escribir es para **este repo (frontend de Delo-sangeles)**, o el README **global del repo del equipo P8_Data_Analysis** que englobe tanto tu EDA como este frontend? Marca claramente qué se espera y ajustamos el alcance.

Hola Claude desde vscode01.jj@gmail.com

Perdona Claude, no te he introducido la estructura de trabajo.

Naiza (Delo-sangeles) es la Scrum Master.

Juanma es el Product Owner.

Andy es Data Analyst.

Yo, JJ, soy el chico de los recados.

Luego está Pal, que va por libre.

Naiza me ha pedido un README y me ha dado la url de su repo lab.

De ahí a P8_Data_Analysis (donde va el README que espero que hagamos).

Juanma ha hecho un Power Bi espectacular, así como un EDA (todos hemos hecho EDA, aunque yo con bugs y carycooper777).

Tendría que hacer Power Bi, pero no tengo licencia ni potencia de máquina. De ahí Looker Studio (que lo tengo en el aire). Todos han hecho Power Bi, menos yo, que estoy a dos velas.

Esperaba encontrar en P8_Data_Analysis/develop el UI del Dashboard de Naiza, pero luego caí que en el chat puso su GitHub con instrucciones.

Me dice que instale NODE.JS (que no sé de qué va NODE.JS ni npm).

Estoy más perdido que una aguja en un pajar.

Antes del README.md, ¿puedes introducirme la metodología AGILE y descripciones de los términos empleados kanban, roles, issues, UI del Dashboard, Dashboard, API, ...?.

¿Corresponde hacer

git fetch

desde

~/Proyectos/Data_Analysis.Delo-sangeles (develop)

?.

¿Vamos?.

```
001> Shell      Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (main)
002> Shell      $ git switch develop
003> Shell      branch 'develop' set up to track 'origin/develop'.
004> Shell      Switched to a new branch 'develop'
005> Shell
006> Shell      Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (develop)
007> Shell      $ cat README.md
008> Shell      echo "===== SETUP.md ====="
009> Shell      cat SETUP.md
010> Shell      echo "===== BACKEND_EXAMPLES.md ====="
011> Shell      cat BACKEND_EXAMPLES.md
012> Shell      echo "===== .env.example ====="
013> Shell      cat .env.example
014> Shell      # Frontend Data Analysis
015> Shell
016> Shell      Interfaz gráfica moderna para visualización de datos, con soporte para Power BI y análisis
de datos personalizados.
017> Shell
018> Shell      ## 🚀 Características
019> Shell
020> Shell      - ✨ Interfaz moderna con React + TypeScript
021> Shell      - 📊 Gráficos interactivos con Recharts
022> Shell      - 🎨 Estilos con Tailwind CSS
023> Shell      - 🔗 Cliente API integrado con Axios
024> Shell      - 📱 Diseño responsivo
025> Shell      - 🔄 Hooks personalizados para manejo de estado
026> Shell      - 🗂 Soporte para variables de entorno
027> Shell
028> Shell      ## 📁 Estructura de Carpetas
029> Shell
030> Shell      ```
031> Shell      src/
032> Shell      ├── components/      # Componentes reutilizables
033> Shell      │   ├── Charts.tsx      # Gráficos (LineChart, BarChart, PieChart)
034> Shell      │   ├── MetricCard.tsx  # Tarjetas de métricas
035> Shell      │   ├── Navbar.tsx      # Navegación
036> Shell      │   └── LoadingStates.tsx # Estados de carga y error
037> Shell      ├── pages/            # Páginas principales
038> Shell      │   ├── Dashboard.tsx   # Dashboard principal
039> Shell      │   └── Analytics.tsx    # Análisis detallado
040> Shell      ├── services/         # Servicios de API
041> Shell      │   └── apiClient.ts    # Cliente HTTP
042> Shell      ├── hooks/           # Hooks personalizados
043> Shell      │   └── useApi.ts       # Hook para llamadas API
044> Shell      ├── types/           # Tipos TypeScript
045> Shell      │   └── index.ts        # Definiciones de tipos
046> Shell      ├── utils/          # Funciones utilitarias
047> Shell      │   └── formatters.ts   # Formateadores de datos
048> Shell      ├── styles/         # Estilos CSS
049> Shell      │   ├── index.css      # Estilos globales
050> Shell      │   └── App.css        # Estilos de la aplicación
051> Shell      ├── main.tsx         # Punto de entrada
052> Shell      └── App.tsx           # Componente principal
053> Shell
054> Shell      public/            # Archivos estáticos
055> Shell
056> Shell      .env.example        # Variables de entorno de ejemplo
```

Descripción:

```

057>_ Shell    ````
058>_ Shell
059>_ Shell    ## 🚀 Instalación
060>_ Shell
061>_ Shell    ### Requisitos
062>_ Shell    - Node.js 18+
063>_ Shell    - npm o yarn
064>_ Shell
065>_ Shell    ### Comandos
066>_ Shell
067>_ Shell    ```bash
068>_ Shell    # Instalar dependencias
069>_ Shell    npm install
070>_ Shell
071>_ Shell    # Iniciar servidor de desarrollo
072>_ Shell    npm run dev
073>_ Shell
074>_ Shell    # Compilar para producción
075>_ Shell    npm run build
076>_ Shell
077>_ Shell    # Preview de build
078>_ Shell    npm run preview
079>_ Shell
080>_ Shell    # Linting
081>_ Shell    npm run lint
082>_ Shell    ```
083>_ Shell
084>_ Shell    ## 🛠 Configuración
085>_ Shell
086>_ Shell    ### Variables de Entorno
087>_ Shell
088>_ Shell    Copia `.env.example` a `.env` y configura:
089>_ Shell
090>_ Shell    ```env
091>_ Shell    VITE_API_URL=http://localhost:8000
092>_ Shell    VITE_API_TIMEOUT=30000
093>_ Shell
094>_ Shell    # Para Power BI
095>_ Shell    # VITE_POWERBI_TENANT_ID=tu_tenant_id
096>_ Shell    # VITE_POWERBI_CLIENT_ID=tu_client_id
097>_ Shell    ```
098>_ Shell
099>_ Shell    ## 🔄 Integración con Backend
100>_ Shell
101>_ Shell    El frontend espera los siguientes endpoints:
102>_ Shell
103>_ Shell    ### Dashboard
104>_ Shell    ```
105>_ Shell    GET /api/dashboard
106>_ Shell    ```
107>_ Shell    Respuesta esperada:
108>_ Shell    ```json
109>_ Shell    {
110>_ Shell      "success": true,
111>_ Shell      "data": {
112>_ Shell        "totalUsers": 15420,
113>_ Shell        "activeUsers": 8234,
114>_ Shell        "revenue": 45890,
115>_ Shell        "growth": 12.5
116>_ Shell      }
117>_ Shell    }
118>_ Shell    ```
119>_ Shell
120>_ Shell    ### Análisis
121>_ Shell    ```
122>_ Shell    GET /api/analytics
123>_ Shell    ```
124>_ Shell
125>_ Shell    ### Filtros de datos
126>_ Shell    ```
127>_ Shell    POST /api/analytics/filter
128>_ Shell    Body: { "startDate": "2024-01-01", ... }
129>_ Shell    ```
130>_ Shell
131>_ Shell    ### Power BI Token (si aplica)
132>_ Shell    ```
133>_ Shell    GET /api/powerbi/token
134>_ Shell    ```
135>_ Shell
136>_ Shell    ## 📊 Alternativas de Visualización
137>_ Shell
138>_ Shell    El proyecto actualmente utiliza **Recharts**, pero puedes cambiar a:
139>_ Shell
140>_ Shell    ### Chart.js

```



```

141>_ Shell    ````bash
142>_ Shell    npm install chart.js react-chartjs-2
143>_ Shell    ````
144>_ Shell
145>_ Shell    ### Apache ECharts
146>_ Shell    ````bash
147>_ Shell    npm install echarts echarts-for-react
148>_ Shell    ````
149>_ Shell
150>_ Shell    ### Visx (Airbnb)
151>_ Shell    ````bash
152>_ Shell    npm install @visx/visx
153>_ Shell    ````
154>_ Shell
155>_ Shell    ## 🐳 Docker
156>_ Shell
157>_ Shell    Para builds de producción:
158>_ Shell
159>_ Shell    ````bash
160>_ Shell    npm run build
161>_ Shell    ````
162>_ Shell
163>_ Shell    Los archivos compilados estarán en la carpeta `dist/`.
164>_ Shell
165>_ Shell    Ejemplo de Dockerfile:
166>_ Shell    ````dockerfile
167>_ Shell    FROM node:18-alpine as build
168>_ Shell    WORKDIR /app
169>_ Shell    COPY package*.json ./
170>_ Shell    RUN npm install
171>_ Shell    COPY . .
172>_ Shell    RUN npm run build
173>_ Shell
174>_ Shell    FROM node:18-alpine
175>_ Shell    WORKDIR /app
176>_ Shell    RUN npm install -g serve
177>_ Shell    COPY --from=build /app/dist ./dist
178>_ Shell    EXPOSE 3000
179>_ Shell    CMD ["serve", "-s", "dist", "-l", "3000"]
180>_ Shell    ````
181>_ Shell
182>_ Shell    ## 📦 Dependencias Principales
183>_ Shell
184>_ Shell    - **React 18**: Framework UI
185>_ Shell    - **React Router DOM 6**: Enrutamiento
186>_ Shell    - **TypeScript**: Type safety
187>_ Shell    - **Vite**: Build tool
188>_ Shell    - **Recharts**: Visualización de gráficos
189>_ Shell    - **Axios**: Cliente HTTP
190>_ Shell    - **Tailwind CSS**: Utility-first CSS
191>_ Shell    - **Lucide React**: Iconografía
192>_ Shell
193>_ Shell    ## 🔗 Conexión con Docker Compose
194>_ Shell
195>_ Shell    Ejemplo de configuración para unir frontend y backend:
196>_ Shell
197>_ Shell    ````yaml
198>_ Shell    version: '3.8'
199>_ Shell
200>_ Shell    services:
201>_ Shell      backend:
202>_ Shell        build: ../Backend_API
203>_ Shell        ports:
204>_ Shell          - "8000:8000"
205>_ Shell        environment:
206>_ Shell          - NODE_ENV=production
207>_ Shell
208>_ Shell      frontend:
209>_ Shell        build: .
210>_ Shell        ports:
211>_ Shell          - "3000:3000"
212>_ Shell        environment:
213>_ Shell          - VITE_API_URL=http://backend:8000
214>_ Shell        depends_on:
215>_ Shell          - backend
216>_ Shell    ````
217>_ Shell
218>_ Shell    ## 🛠️ Desarrollo
219>_ Shell
220>_ Shell    Para agregar un nuevo gráfico:
221>_ Shell
222>_ Shell    1. Crea el componente en `src/components/Charts.tsx`
223>_ Shell    2. Define los tipos en `src/types/index.ts`
224>_ Shell    3. Utiliza el hook `useApi` en tu página

```

```
225>_ Shell 4. Integra con `apiClient.ts` vía tu servicio
226>_ Shell
227>_ Shell ## 🚩 Troubleshooting
228>_ Shell
229>_ Shell ### CORS Error
230>_ Shell Asegúrate que tu backend tenga CORS habilitado para `http://localhost:5173`
231>_ Shell
232>_ Shell ### API no responde
233>_ Shell Verifica `VITE_API_URL` en las variables de entorno
234>_ Shell
235>_ Shell ### TypeScript errors
236>_ Shell Ejecuta:
237>_ Shell ```bash
238>_ Shell npm run type-check
239>_ Shell ```
240>_ Shell
241>_ Shell ## 📄 Licencia
242>_ Shell
243>_ Shell MIT
244>_ Shell
245>_ Shell ## 👤 Autor
246>_ Shell
247>_ Shell Generado como estructura base para análisis de datos
248>_ Shell
249>_ Shell ---
250>_ Shell
251>_ Shell **¿Necesitas ayuda?** Revisa la documentación de:
252>_ Shell - [Vite](https://vitejs.dev)
253>_ Shell - [React](https://react.dev)
254>_ Shell - [Recharts](https://recharts.org)
255>_ Shell - [Tailwind CSS](https://tailwindcss.com)
256>_ Shell ===== SETUP.md =====
257>_ Shell # Quick Start Guide - Guía de Inicio Rápido
258>_ Shell
259>_ Shell ## 🚀 Opción 1: Desarrollo Local (SIN Docker)
260>_ Shell
261>_ Shell ### Paso 1: Instalar dependencias
262>_ Shell ```bash
263>_ Shell cd Frontend_Data_Analysis2026
264>_ Shell npm install
265>_ Shell ```
266>_ Shell
267>_ Shell ### Paso 2: Crear archivo .env
268>_ Shell ```bash
269>_ Shell cp .env.example .env
270>_ Shell ```
271>_ Shell
272>_ Shell ### Paso 3: Completar .env
273>_ Shell ```env
274>_ Shell VITE_API_URL=http://localhost:8000
275>_ Shell VITE_API_TIMEOUT=30000
276>_ Shell ```
277>_ Shell
278>_ Shell ### Paso 4: Iniciar servidor de desarrollo
279>_ Shell ```bash
280>_ Shell npm run dev
281>_ Shell ```
282>_ Shell
283>_ Shell La aplicación abrirá automáticamente en `http://localhost:5173`
284>_ Shell
285>_ Shell ---
286>_ Shell
287>_ Shell ## 🐳 Opción 2: Desarrollo con Docker (RECOMENDADO)
288>_ Shell
289>_ Shell ### Paso 1: Asegurate de tener Docker y Docker Compose
290>_ Shell ```bash
291>_ Shell docker --version
292>_ Shell docker-compose --version
293>_ Shell ```
294>_ Shell
295>_ Shell ### Paso 2: Construir y levantar los contenedores
296>_ Shell ```bash
297>_ Shell # Desde la raíz del proyecto (donde están Backend_API y Frontend_Data_Analysis2026)
298>_ Shell docker-compose up --build
299>_ Shell ```
300>_ Shell
301>_ Shell - Frontend estará en: `http://localhost:3000`
302>_ Shell - Backend estará en: `http://localhost:8000`
303>_ Shell
304>_ Shell ### Paso 3: Parar los contenedores
305>_ Shell ```bash
306>_ Shell docker-compose down
307>_ Shell ```
308>_ Shell
```

Página 11 de 72

```

394> Shell      "expiresIn": 600
395> Shell      }
396> Shell      }
397> Shell      ...
398> Shell
399> Shell      ---
400> Shell
401> Shell      ## 🔄 Conectar poder BI (2 métodos)
402> Shell
403> Shell      ### Método 1: Power BI Embedded
404> Shell      Si tienes Power BI Premium, puedes incrustar reportes:
405> Shell
406> Shell      ```typescript
407> Shell      // En tu backend
408> Shell      import { PowerBIClient } from "@microsoft/powerbi-client";
409> Shell
410> Shell      app.get('/api/powerbi/token', async (req, res) => {
411> Shell          const token = await generateEmbedToken({
412> Shell              reportId: process.env.POWERBI_REPORT_ID,
413> Shell              datasetId: process.env.POWERBI_DATASET_ID
414> Shell          });
415> Shell
416> Shell          res.json({
417> Shell              success: true,
418> Shell              data: { token, expiresIn: 600 }
419> Shell          });
420> Shell      });
421> Shell      ...
422> Shell
423> Shell      Luego en el frontend:
424> Shell      ```typescript
425> Shell      import { PowerBIEmbed } from 'powerbi-client-angular';
426> Shell
427> Shell      // Usar el componente para incrustar el reporte
428> Shell      ...
429> Shell
430> Shell      ### Método 2: Power BI API (Recomendado)
431> Shell      Tu backend consulta Power BI y devuelve los datos:
432> Shell
433> Shell      ```typescript
434> Shell      // En tu backend
435> Shell      import { RestClient } from 'microsoft-rest-client-js';
436> Shell
437> Shell      app.get('/api/analytics', async (req, res) => {
438> Shell          const data = await getPowerBIData({
439> Shell              workspaceId: process.env.POWERBI_WORKSPACE_ID,
440> Shell              reportId: process.env.POWERBI_REPORT_ID
441> Shell          });
442> Shell
443> Shell          res.json({
444> Shell              success: true,
445> Shell              data: transformData(data)
446> Shell          });
447> Shell      });
448> Shell      ...
449> Shell
450> Shell      ---
451> Shell
452> Shell      ## 🐛 Troubleshooting
453> Shell
454> Shell      ### Error: CORS Origin not allowed
455> Shell      **Solución:** Agrega en tu backend:
456> Shell      ```javascript
457> Shell      app.use(cors({
458> Shell          origin: ['http://localhost:5173', 'http://localhost:3000'],
459> Shell          credentials: true
460> Shell      }));
461> Shell      ...
462> Shell
463> Shell      ### Error: Cannot GET /api/...
464> Shell      **Solución:** Verifica que tu backend esté corriendo en el puerto configurado
465> Shell
466> Shell      ### Error: Failed to fetch data
467> Shell      **Solución:**
468> Shell      ```bash
469> Shell      # Revisa la terminal del backend
470> Shell      docker-compose logs backend
471> Shell
472> Shell      # 0 si está en desarrollo local
473> Shell      npm run dev # En terminal del backend
474> Shell      ...
475> Shell
476> Shell      ### Actualizar variables de entorno en producción
477> Shell      1. Edita `.env` localmente
478> Shell      2. Rebuild: `docker-compose up --build`

```

```

479> Shell
480> Shell ---
481> Shell
482> Shell ## 📦 Comandos útiles
483> Shell
484> Shell ```bash
485> Shell # Desarrollo local
486> Shell npm run dev # Start dev server
487> Shell npm run build # Build para producción
488> Shell npm run preview # Preview del build
489> Shell
490> Shell # Docker
491> Shell docker-compose up # Levantar servicios
492> Shell docker-compose down # Bajar servicios
493> Shell docker-compose logs # Ver logs
494> Shell docker-compose logs -f backend # Logs en vivo del backend
495> Shell
496> Shell # Build individual
497> Shell docker build -t frontend:latest .
498> Shell
499> Shell # Ejecutar frontend solo
500> Shell docker run -p 3000:3000 frontend:latest
501> Shell ```
502> Shell
503> Shell ---
504> Shell
505> Shell ## 📦 Configuraciones según tu Stack
506> Shell
507> Shell ### Si tu backend es Node.js/Express
508> Shell ```bash
509> Shell # Instala CORS
510> Shell npm install cors express
511> Shell
512> Shell # En tu app.js
513> Shell const cors = require('cors');
514> Shell app.use(cors());
515> Shell ```
516> Shell
517> Shell ### Si tu backend es Python/FastAPI
518> Shell ```python
519> Shell from fastapi.middleware.cors import CORSMiddleware
520> Shell
521> Shell app.add_middleware(
522> Shell CORSMiddleware,
523> Shell allow_origins=["http://localhost:5173", "http://localhost:3000"],
524> Shell allow_credentials=True,
525> Shell allow_methods=["*"],
526> Shell allow_headers=["*"],
527> Shell )
528> Shell ```
529> Shell
530> Shell ### Si tu backend es .NET/C#
531> Shell ```csharp
532> Shell services.AddCors(options => {
533> Shell options.AddDefaultPolicy(builder => {
534> Shell builder.WithOrigins("http://localhost:5173", "http://localhost:3000")
535> Shell .AllowAnyMethod()
536> Shell .AllowAnyHeader();
537> Shell });
538> Shell });
539> Shell
540> Shell app.UseCors();
541> Shell ```
542> Shell
543> Shell ---
544> Shell
545> Shell ## ✅ Checklist de Deploy
546> Shell
547> Shell - [ ] Backend endpoints implementados y probados
548> Shell - [ ] CORS configurado
549> Shell - [ ] Variables de entorno correctas en .env
550> Shell - [ ] `npm install` ejecutado
551> Shell - [ ] `npm run build` sin errores
552> Shell - [ ] `docker build` exitoso
553> Shell - [ ] `docker-compose up` sin errores
554> Shell - [ ] Frontend se carga en http://localhost:3000
555> Shell - [ ] API devuelve datos correctamente
556> Shell - [ ] Gráficos se renderizan
557> Shell
558> Shell ---
559> Shell
560> Shell ¡Listo! Tu interfaz de análisis de datos está lista. 🎉
561> Shell ===== BACKEND_EXAMPLES.md =====
562> Shell # Ejemplos de Integración - Backend

```

```

563>_ Shell
564>_ Shell ## 📁 Ejemplos de Endpoints para tu Backend
565>_ Shell
566>_ Shell ### Node.js/Express
567>_ Shell
568>_ Shell ```javascript
569>_ Shell import express from 'express';
570>_ Shell import cors from 'cors';
571>_ Shell
572>_ Shell const app = express();
573>_ Shell app.use(cors());
574>_ Shell app.use(express.json());
575>_ Shell
576>_ Shell // Dashboard Metrics
577>_ Shell app.get('/api/dashboard', (req, res) => {
578>_ Shell   res.json({
579>_ Shell     success: true,
580>_ Shell     data: {
581>_ Shell       totalUsers: 15420,
582>_ Shell       activeUsers: 8234,
583>_ Shell       revenue: 45890,
584>_ Shell       growth: 12.5
585>_ Shell     }
586>_ Shell   });
587>_ Shell });
588>_ Shell
589>_ Shell // Analytics con gráficos
590>_ Shell app.get('/api/analytics', (req, res) => {
591>_ Shell   res.json({
592>_ Shell     success: true,
593>_ Shell     data: {
594>_ Shell       chartData: [
595>_ Shell         { name: 'Enero', value: 4000 },
596>_ Shell         { name: 'Febrero', value: 3000 },
597>_ Shell         { name: 'Marzo', value: 2000 },
598>_ Shell         { name: 'Abril', value: 2780 },
599>_ Shell         { name: 'Mayo', value: 1890 },
600>_ Shell         { name: 'Junio', value: 2390 }
601>_ Shell       ],
602>_ Shell       metrics: {
603>_ Shell         totalUsers: 15420,
604>_ Shell         activeUsers: 8234,
605>_ Shell         revenue: 45890,
606>_ Shell         growth: 12.5
607>_ Shell       },
608>_ Shell       timestamp: new Date().toISOString()
609>_ Shell     }
610>_ Shell   });
611>_ Shell });
612>_ Shell
613>_ Shell // Filtrado de datos
614>_ Shell app.post('/api/analytics/filter', (req, res) => {
615>_ Shell   const { startDate, endDate, category } = req.body;
616>_ Shell
617>_ Shell   // Aquí va tu lógica de filtrado
618>_ Shell
619>_ Shell   res.json({
620>_ Shell     success: true,
621>_ Shell     data: {
622>_ Shell       chartData: [...],
623>_ Shell       metrics: {...},
624>_ Shell       timestamp: new Date().toISOString()
625>_ Shell     }
626>_ Shell   });
627>_ Shell });
628>_ Shell
629>_ Shell // Chart específico
630>_ Shell app.get('/api/charts/:chartId', (req, res) => {
631>_ Shell   const { chartId } = req.params;
632>_ Shell
633>_ Shell   res.json({
634>_ Shell     success: true,
635>_ Shell     data: [
636>_ Shell       { name: 'Item 1', value: 400 },
637>_ Shell       { name: 'Item 2', value: 300 },
638>_ Shell       // ...
639>_ Shell     ]
640>_ Shell   });
641>_ Shell });
642>_ Shell
643>_ Shell app.listen(8000, () => {
644>_ Shell   console.log('Backend running on port 8000');
645>_ Shell });
646>_ Shell ```
647>_ Shell

```

Descripción:

```

648> Shell      ### Python/FastAPI
649> Shell
650> Shell      ```python
651> Shell      from fastapi import FastAPI
652> Shell      from fastapi.middleware.cors import CORSMiddleware
653> Shell      from datetime import datetime
654> Shell
655> Shell      app = FastAPI()
656> Shell
657> Shell      app.add_middleware(
658> Shell          CORSMiddleware,
659> Shell          allow_origins=["http://localhost:5173", "http://localhost:3000"],
660> Shell          allow_credentials=True,
661> Shell          allow_methods=["*"],
662> Shell          allow_headers=["*"],
663> Shell      )
664> Shell
665> Shell      @app.get("/api/dashboard")
666> Shell      async def get_dashboard():
667> Shell          return {
668> Shell              "success": True,
669> Shell              "data": {
670> Shell                  "totalUsers": 15420,
671> Shell                  "activeUsers": 8234,
672> Shell                  "revenue": 45890,
673> Shell                  "growth": 12.5
674> Shell              }
675> Shell          }
676> Shell
677> Shell      @app.get("/api/analytics")
678> Shell      async def get_analytics():
679> Shell          return {
680> Shell              "success": True,
681> Shell              "data": {
682> Shell                  "chartData": [
683> Shell                      {"name": "Enero", "value": 4000},
684> Shell                      {"name": "Febrero", "value": 3000},
685> Shell                      # ...
686> Shell                  ],
687> Shell                  "metrics": {
688> Shell                      "totalUsers": 15420,
689> Shell                      "activeUsers": 8234,
690> Shell                      "revenue": 45890,
691> Shell                      "growth": 12.5
692> Shell                  },
693> Shell                  "timestamp": datetime.now().isoformat()
694> Shell              }
695> Shell          }
696> Shell
697> Shell      @app.post("/api/analytics/filter")
698> Shell      async def filter_analytics(filters: dict):
699> Shell          # Procesar filtros
700> Shell          return {
701> Shell              "success": True,
702> Shell              "data": {...}
703> Shell          }
704> Shell
705> Shell      if __name__ == "__main__":
706> Shell          import uvicorn
707> Shell          uvicorn.run(app, host="0.0.0.0", port=8000)
708> Shell      ...
709> Shell
710> Shell      ### .NET/C# (ASP.NET Core)
711> Shell
712> Shell      ```csharp
713> Shell      using Microsoft.AspNetCore.Mvc;
714> Shell      using System;
715> Shell      using System.Collections.Generic;
716> Shell
717> Shell      [ApiController]
718> Shell      [Route("api")]
719> Shell      public class DataController : ControllerBase
720> Shell      {
721> Shell          [HttpGet("dashboard")]
722> Shell          public IActionResult GetDashboard()
723> Shell          {
724> Shell              return Ok(new
725> Shell              {
726> Shell                  success = true,
727> Shell                  data = new
728> Shell                  {
729> Shell                      totalUsers = 15420,
730> Shell                      activeUsers = 8234,
731> Shell                      revenue = 45890,
732> Shell                      growth = 12.5

```

```

733>_ Shell      }
734>_ Shell      });
735>_ Shell      }
736>_ Shell
737>_ Shell      [HttpGet("analytics")]
738>_ Shell      public IActionResult GetAnalytics()
739>_ Shell      {
740>_ Shell          var chartData = new List<object>
741>_ Shell          {
742>_ Shell              new { name = "Enero", value = 4000 },
743>_ Shell              new { name = "Febrero", value = 3000 },
744>_ Shell              // ...
745>_ Shell          };
746>_ Shell
747>_ Shell          return Ok(new
748>_ Shell          {
749>_ Shell              success = true,
750>_ Shell              data = new
751>_ Shell              {
752>_ Shell                  chartData = chartData,
753>_ Shell                  metrics = new
754>_ Shell                  {
755>_ Shell                      totalUsers = 15420,
756>_ Shell                      activeUsers = 8234,
757>_ Shell                      revenue = 45890,
758>_ Shell                      growth = 12.5
759>_ Shell                  },
760>_ Shell                  timestamp = DateTime.UtcNow
761>_ Shell              }
762>_ Shell          });
763>_ Shell      }
764>_ Shell
765>_ Shell      [HttpPost("analytics/filter")]
766>_ Shell      public IActionResult FilterAnalytics([FromBody] FilterRequest filter)
767>_ Shell      {
768>_ Shell          // Procesar el filtrado
769>_ Shell
770>_ Shell          return Ok(new
771>_ Shell          {
772>_ Shell              success = true,
773>_ Shell              data = new { /* ... */ }
774>_ Shell          });
775>_ Shell      }
776>_ Shell  }
777>_ Shell
778>_ Shell  public class FilterRequest
779>_ Shell  {
780>_ Shell      public string StartDate { get; set; }
781>_ Shell      public string EndDate { get; set; }
782>_ Shell      public string Category { get; set; }
783>_ Shell  }
784>_ Shell  ...
785>_ Shell
786>_ Shell  ---
787>_ Shell
788>_ Shell  ## 🚩 Integrando con Power BI
789>_ Shell
790>_ Shell  ### Opción A: Power BI Embedded (Node.js)
791>_ Shell
792>_ Shell  ```javascript
793>_ Shell  import { PowerBIEmbedding } from 'powerbi-embedding';
794>_ Shell
795>_ Shell  app.get('/api/powerbi/token', async (req, res) => {
796>_ Shell      try {
797>_ Shell          const token = await powerbi.generateToken({
798>_ Shell              reportId: process.env.POWERBI_REPORT_ID,
799>_ Shell              datasetId: process.env.POWERBI_DATASET_ID,
800>_ Shell              tenantId: process.env.POWERBI_TENANT_ID,
801>_ Shell              clientId: process.env.POWERBI_CLIENT_ID,
802>_ Shell              clientSecret: process.env.POWERBI_CLIENT_SECRET
803>_ Shell          });
804>_ Shell
805>_ Shell          res.json({
806>_ Shell              success: true,
807>_ Shell              data: {
808>_ Shell                  token: token.accessToken,
809>_ Shell                  expiresIn: 600
810>_ Shell              }
811>_ Shell          });
812>_ Shell      } catch (error) {
813>_ Shell          res.status(500).json({
814>_ Shell              success: false,
815>_ Shell              error: error.message
816>_ Shell          });
817>_ Shell      }

```

Descripción:

Fecha de creación:
20260501043700

Versión:
20260505092400

Página 16 de 72


```

818>_ Shell    });
819>_ Shell    ```
820>_ Shell
821>_ Shell    ### Opción B: Power BI API (Obtener datos)
822>_ Shell
823>_ Shell    ```javascript
824>_ Shell    import axios from 'axios';
825>_ Shell
826>_ Shell    async function getPowerBIData() {
827>_ Shell        const token = await getAccessToken();
828>_ Shell
829>_ Shell        const response = await axios.get(
830>_ Shell    `https://api.powerbi.com/v1.0/myorg/groups/${workspaceId}/reports/${reportId}/datasources`,
831>_ Shell        {
832>_ Shell            headers: {
833>_ Shell                Authorization: `Bearer ${token}`
834>_ Shell            }
835>_ Shell        }
836>_ Shell    );
837>_ Shell
838>_ Shell    return response.data;
839>_ Shell    }
840>_ Shell    ```
841>_ Shell
842>_ Shell    ---
843>_ Shell
844>_ Shell    ## 📁 Ejemplo Completo de Estructura
845>_ Shell
846>_ Shell    ```
847>_ Shell    Backend_API/
848>_ Shell    └── src/
849>_ Shell        ├── routes/
850>_ Shell            ├── dashboard.js
851>_ Shell            ├── analytics.js
852>_ Shell            └── powerbi.js
853>_ Shell        ├── controllers/
854>_ Shell            ├── dashboardController.js
855>_ Shell            ├── analyticsController.js
856>_ Shell            └── powerbiController.js
857>_ Shell        ├── models/
858>_ Shell            ├── Dashboard.js
859>_ Shell            └── Analytics.js
860>_ Shell        ├── middleware/
861>_ Shell            ├── auth.js
862>_ Shell            └── errorHandler.js
863>_ Shell        ├── utils/
864>_ Shell            └── powerbi.js
865>_ Shell        └── app.js
866>_ Shell    ├── .env.example
867>_ Shell    ├── package.json
868>_ Shell    └── Dockerfile
869>_ Shell    ```
870>_ Shell
871>_ Shell    ---
872>_ Shell
873>_ Shell    ## 📁 Variables de Entorno Esperadas
874>_ Shell
875>_ Shell    ```env
876>_ Shell    # Servidor
877>_ Shell    PORT=8000
878>_ Shell    NODE_ENV=development
879>_ Shell
880>_ Shell    # Base de datos
881>_ Shell    DB_HOST=localhost
882>_ Shell    DB_PORT=5432
883>_ Shell    DB_NAME=analytics
884>_ Shell    DB_USER=user
885>_ Shell    DB_PASSWORD=password
886>_ Shell
887>_ Shell    # Power BI (opcional)
888>_ Shell    POWERBI_TENANT_ID=
889>_ Shell    POWERBI_CLIENT_ID=
890>_ Shell    POWERBI_CLIENT_SECRET=
891>_ Shell    POWERBI_WORKSPACE_ID=
892>_ Shell    POWERBI_REPORT_ID=
893>_ Shell    POWERBI_DATASET_ID=
894>_ Shell
895>_ Shell    # CORS
896>_ Shell    FRONTEND_URL=http://localhost:5173
897>_ Shell    ```
898>_ Shell
899>_ Shell    ---
900>_ Shell
901>_ Shell    ¡Ahora el frontend podrá conectarse correctamente a tu backend! 🚀

```

```

902>_ Shell ===== .env.example =====
903>_ Shell VITE_API_URL=http://localhost:8000
904>_ Shell VITE_API_TIMEOUT=30000
905>_ Shell
906>_ Shell # Power BI Configuration (if using Power BI API)
907>_ Shell # VITE_POWERBI_TENANT_ID=your_tenant_id
908>_ Shell # VITE_POWERBI_CLIENT_ID=your_client_id
909>_ Shell # VITE_POWERBI_WORKSPACE_ID=your_workspace_id
910>_ Shell
911>_ Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (develop)
912>_ Shell $ cat package.json
913>_ Shell {
914>_ Shell   "name": "frontend-data-analysis",
915>_ Shell   "private": true,
916>_ Shell   "version": "0.0.1",
917>_ Shell   "type": "module",
918>_ Shell   "scripts": {
919>_ Shell     "dev": "vite",
920>_ Shell     "build": "tsc && vite build",
921>_ Shell     "lint": "eslint . --ext ts,tsx --report-unused-disable-directives --max-warnings 0",
922>_ Shell     "preview": "vite preview",
923>_ Shell     "type-check": "tsc --noEmit"
924>_ Shell   },
925>_ Shell   "dependencies": {
926>_ Shell     "@types/leaflet": "^1.9.21",
927>_ Shell     "axios": "^1.5.0",
928>_ Shell     "leaflet": "^1.9.4",
929>_ Shell     "lucide-react": "^0.294.0",
930>_ Shell     "react": "^18.2.0",
931>_ Shell     "react-dom": "^18.2.0",
932>_ Shell     "react-leaflet": "^4.2.1",
933>_ Shell     "react-router-dom": "^6.16.0",
934>_ Shell     "recharts": "^2.15.4",
935>_ Shell     "tailwindcss": "^3.3.0"
936>_ Shell   },
937>_ Shell   "devDependencies": {
938>_ Shell     "@types/react": "^18.2.37",
939>_ Shell     "@types/react-dom": "^18.2.15",
940>_ Shell     "@typescript-eslint/eslint-plugin": "^6.10.0",
941>_ Shell     "@typescript-eslint/parser": "^6.10.0",
942>_ Shell     "@vitejs/plugin-react": "^4.2.0",
943>_ Shell     "autoprefixer": "^10.4.16",
944>_ Shell     "eslint": "^8.53.0",
945>_ Shell     "eslint-plugin-react": "^7.33.2",
946>_ Shell     "eslint-plugin-react-hooks": "^4.6.0",
947>_ Shell     "postcss": "^8.4.31",
948>_ Shell     "typescript": "^5.2.2",
949>_ Shell     "vite": "^7.0.0"
950>_ Shell   }
951>_ Shell }
952>_ Shell
953>_ Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (develop)
954>_ Shell $ node --version
955>_ Shell npm --version
956>_ Shell v24.14.1
957>_ Shell 11.11.0
958>_ Shell
959>_ Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (develop)
960>_ Shell $

```

Echo de menos los colores de Git Bash, Markdown, Python, ... en Notepad ++ para archivos.txt.

Gracias Claude desde vscode01.jj@gmail.com

20260504085332*

Enlace privado:

<https://claude.ai/chat/b5fdf060-e8bd-41e1-85ab-9ffbd0dd624a>

Enlace público:

<https://claude.ai/share/42c4b417-073a-477b-85f6-243e0d170896>

Hola JJ, respira. No estás tan perdido como crees – de hecho lo estás haciendo bien: clonaste, renombraste con criterio, te moviste a develop, leíste la documentación que ya existía, comprobaste node y npm. Ese es exactamente el método. Lo que pasa es que has aterrizado en un mundo (frontend web) con vocabulario propio. Vamos por partes.

Agile, Scrum y Kanban

Agile no es una herramienta, es una filosofía: en vez de planear todo el proyecto al detalle al principio, se trabaja en ciclos cortos, se entrega algo que funcione cada poco tiempo y se ajusta sobre la marcha. Surgió como reacción al modelo "cascada" (especificar todo → diseñar todo → programar todo → entregar a los 18 meses algo que ya nadie quiere).

Scrum es un *framework* concreto dentro de Agile. Define roles, ceremonias (reuniones) y artefactos. Los roles que mencionas encajan así:

- **Product Owner (Juanma)**: representa al "cliente". Decide *qué* se hace y en qué orden de prioridad. Mantiene el *product backlog* (lista de cosas pendientes priorizada).
- **Scrum Master (Naiza)**: facilita el proceso. No es jefa: su trabajo es quitar bloqueos al equipo, asegurar que se siguen las ceremonias y proteger al equipo de interrupciones externas.
- **Development Team (Andy, Pal y tú)**: quienes hacen el trabajo. En Scrum "puro" el equipo es auto-organizado y multidisciplinar.

Las ceremonias típicas son: *sprint planning* (qué haremos las próximas 1-2 semanas), *daily standup* (15 min cada mañana: qué hice ayer, qué haré hoy, qué me bloquea), *sprint review* (demo de lo terminado) y *retrospective* (qué mejoramos como equipo).

Kanban es otro enfoque Agile, más ligero que Scrum. Se basa en un tablero con columnas (típicamente *Backlog* → *To Do* → *In Progress* → *In Review* → *Done*) y tarjetas que avanzan de izquierda a derecha. La regla clave es **limitar el trabajo en curso (WIP)**: no puedes tener 17 cosas a medias. Muchos equipos hacen un híbrido "Scrum + tablero Kanban" – que es probablemente lo vuestro. El tablero del equipo en GitHub Projects (el que practicaste con tu sandbox) es exactamente eso.

Issues: son las "tarjetas" en GitHub. Cada issue describe una tarea, bug, feature o pregunta. Tienen título, descripción, *assignees* (a quién le toca), *Labels* (etiquetas tipo bug, enhancement, documentation), y se pueden vincular al tablero Kanban. El "Issue #2 – Data Viz Expert" del que vienes es una tarjeta de ese tipo.

Glosario técnico

Frontend vs Backend: imagínate un restaurante. El **frontend** es la sala: lo que el cliente ve y toca (la carta, las mesas, los platos servidos). El **backend** es la cocina: donde se preparan los datos, se guardan en frigoríficos (bases de datos), se cocinan según la receta (lógica de negocio). El cliente nunca entra en la cocina; solo recibe el plato. El repo de Naiza que has clonado **es la sala**.

UI (User Interface): la interfaz de usuario. Botones, menús, gráficos, formularios – todo lo visible y manipulable. "UI del Dashboard" = "el aspecto y la interactividad del panel".

Dashboard: panel de control. Una pantalla que reúne métricas y gráficos clave de un vistazo (típicamente: KPIs grandes arriba, gráficos en el medio, tablas o filtros abajo). Power BI hace dashboards. Looker Studio hace dashboards. Y este frontend de React también está hecho para mostrar un dashboard.

API (Application Programming Interface): el "menú" mediante el que el frontend pide cosas al backend. Cuando el frontend necesita los datos del dashboard, no entra en la base de datos: hace una petición tipo GET `/api/dashboard` y el backend le devuelve un JSON con los datos. En el README de Naiza viste exactamente esos endpoints definidos (`/api/dashboard`, `/api/analytics`, etc.).

JSON: formato de texto para representar datos estructurados, tipo `{ "totalUsers": 15420, "revenue": 45890 }`. Es como hablan entre sí frontend y backend.

Node.js: un entorno que permite ejecutar JavaScript fuera del navegador (por ejemplo en tu terminal).

Originalmente JS solo corría en navegadores; Node lo sacó de ahí. Lo necesitas porque las herramientas de desarrollo del frontend moderno (compilador, servidor local, linter, empaquetador) están escritas en JS y se ejecutan con Node. Para ti hoy: Node es lo que permite escribir `npm run dev` y que arranque el servidor de desarrollo. Tienes Node 24 instalado, vas sobrado (el README pedía 18+).

npm (Node Package Manager): el pip de JavaScript. Instala librerías. Cuando hagas `npm install`, leerá `package.json` (equivalente al `pyproject.toml/requirements.txt`) y descargará todas las dependencias en una carpeta llamada `node_modules/` (la cual NO se sube a Git – está en `.gitignore`). `npm run dev` ejecuta el script dev definido en `package.json` (en este caso, vite).

React: librería para construir interfaces en componentes reutilizables (cada botón, cada gráfico es un "componente"). Las extensiones `.tsx` que viste en `src/components/` son componentes React escritos en TypeScript. **TypeScript**: JavaScript con tipos. Es JS al que le añades anotaciones tipo `let edad: number = 30` para evitar errores. Se compila a JS normal antes de ejecutarse.

Vite: el "servidor de desarrollo" y empaquetador. Cuando haces `npm run dev` arranca un servidor local (típicamente en `http://localhost:5173`) que recompila al instante cuando guardas un archivo. Es la versión moderna y rápida de herramientas como Webpack.

Tailwind CSS: forma de dar estilo escribiendo clases pequeñas directamente en el HTML/JSX (`class="bg-blue-500 text-white p-4"`) en lugar de archivos CSS separados.

Recharts / Leaflet: librerías de visualización. Recharts hace gráficos (barras, líneas, tartas – equivalente a Plotly). Leaflet hace mapas interactivos. Encajan con tu proyecto porque tenéis 6 ciudades.

Sobre tu situación con Power BI

No te martirices. Power BI Desktop es solo Windows y consume bastante RAM, y la licencia Pro/Premium para publicar y embeber cuesta dinero. **Looker Studio (gratis, basado en navegador) es perfectamente válido como alternativa profesional** – Google lo usa internamente, lo enseñan en cursos serios, y se conecta nativamente a Sheets, BigQuery, CSVs, etc. Que el resto del equipo haya ido por Power BI no lo hace "el camino correcto", solo el más común. Tu combinación **Plotly (interactividad en notebooks) + Looker Studio (dashboard publicable) + este frontend React si toca integrarse** es totalmente defendible.

¿git fetch ahora?

Sí, es buena práctica y no rompe nada. `git fetch` actualiza las **referencias remotas** locales (lo que tu Git "sabe" sobre el estado del servidor) sin tocar tus archivos ni tu rama actual. Es información, no acción. Como acabas de clonar hace un rato, probablemente no traerá novedades, pero acostúmbrate a hacerlo cada vez que vuelvas a un repo después de un rato:

```
bash
git fetch --all --prune
```

git status

El --prune borra referencias a ramas remotas que ya no existen. git status te dirá si tu develop local está sincronizada con origin/develop o si hay commits nuevos por traer (en cuyo caso harías git pull).

Próximo paso sugerido

Cuando estés listo, prueba a levantar el frontend (es el "intentar ejecutar en mi máquina" de tus instrucciones):

bash

Estás en ~/Proyectos/Data_Analysis.Delo-sangeles (develop)

cp .env.example .env # crear el .env de trabajo (luego lo editas si hace falta)

npm install # instala todas las dependencias en node_modules/

npm run dev # arranca el servidor de desarrollo

npm install puede tardar 1-3 minutos la primera vez y descargará bastantes MB. Cuando npm run dev termine, te dará una URL tipo http://localhost:5173/ que abres en el navegador. **Es muy probable que la UI cargue pero los gráficos estén vacíos o den error**, porque el frontend espera un backend en http://localhost:8000 que no existe. Eso es información valiosa para el README – anótalo.

Pega la salida de npm install y npm run dev y seguimos desde ahí.

(Y sí, lo de los colores en Notepad++ se arregla: *Lenguaje* → *Definir tu Lenguaje* o instalando UDL para Markdown, pero eso lo dejamos para otro día – no es lo crítico ahora.)

<https://gemini.google.com/share/6485e8e0cb99>

Hola Gemini desde vscode01.jj@gmail.com

¿Puedes indicarme la diferencia entre usar:

git fetch --all --prune

y

git fetch

especificando las distintas opciones, es decir:

uso y comportamiento esperado de git fetch a secas a diferencia de usar --all y de --prune.

?

¿Se pueden usar --all y --prune por separado?.

¿Que conlleva usarlos juntos?.

Gracias Gemini desde vscode01.jj@gmail.com

20260504092345*

¡Hola! Gracias por tu excelente pregunta. Es un placer ayudarte a aclarar estos conceptos de Git.

Para entender la diferencia, primero debemos recordar qué hace el comando base y luego cómo los modificadores cambian su comportamiento.

Aquí tienes el desglose detallado de cómo funcionan estas opciones, ya sea solas o combinadas.

1. El comportamiento de git fetch (a secas)

Cuando ejecutas únicamente git fetch:

- **Objetivo:** Git se comunica con tu repositorio remoto predeterminado (generalmente llamado origin) y descarga cualquier confirmación (commit), archivo o referencia (como nuevas ramas) que exista en el servidor pero que tú no tengas localmente.
- **Seguridad:** Es una operación completamente segura porque **no altera tu espacio de trabajo actual** ni fusiona los cambios (a diferencia de git pull). Solo actualiza tus ramas de seguimiento remoto (las que ves como origin/nombre-de-rama).
- **Limitación importante:** Si alguien más eliminó una rama en el servidor remoto (por ejemplo, después de fusionar un Pull Request), un git fetch normal **no eliminará** la referencia de esa rama en tu máquina local. Tu Git seguirá creyendo que esa rama remota existe en el historial.

2. ¿Se pueden usar --all y --prune por separado?

Sí, **absolutamente**. Son opciones completamente independientes y se pueden usar por separado según lo que necesites en ese momento.

El uso de git fetch --all

- **Comportamiento:** Si trabajas en un proyecto donde tienes configurados múltiples repositorios remotos (por ejemplo, origin para tu bifurcación o *fork*, y upstream para el repositorio original del proyecto), este comando le dice a Git que se conecte y descargue las novedades de **todos** los remotes configurados, no solo del predeterminado.
- **Uso:** Es ideal para mantener actualizado tu repositorio local con varias fuentes a la vez, pero al igual que el comando base, no limpia las ramas eliminadas.

El uso de git fetch --prune (o git fetch -p)

- **Comportamiento:** Este comando altera la forma en que Git interactúa con el servidor. Antes de descargar las novedades, Git compara las ramas remotas del servidor con las ramas de seguimiento remoto que tienes en tu máquina. Si detecta que una rama ya no existe en el servidor, **la elimina de tu lista local** (origin/rama-vieja).
- **Uso:** Es la mejor herramienta para hacer "limpieza". Evita que tu lista de ramas locales se llene de referencias "fantasmas" o basura acumulada de ramas que tus compañeros de equipo ya borraron en el servidor.

3. ¿Qué conlleva usarlos juntos (git fetch --all --prune)?

Cuando combinas ambas banderas, estás ejecutando la sincronización y limpieza más completa posible para tu repositorio local:

- **Sincronización total:** Git iterará por **todos** los repositorios remotos que tengas configurados (origin, upstream, etc.).
- **Limpieza total:** En cada uno de esos remotes, Git purgará y eliminará cualquier rama de seguimiento local que ya no exista en el servidor correspondiente.
- **Actualización total:** Finalmente, descargará todo el código y las ramas nuevas de todos los remotes.

En resumen: Ejecutar git fetch --all --prune es como decirle a Git: *"Conéctate a todos Los servidores externos que conozcas, elimina toda la basura local de ramas que ya no existan en esos servidores y luego descárgame exactamente lo último que tengan"*. Es una práctica excelente para mantener la salud y el orden de tu repositorio local.

Enlace privado:

<https://claude.ai/chat/b5fdf060-e8bd-41e1-85ab-9ffbd0dd624a>

Enlace público:

<https://claude.ai/share/42c4b417-073a-477b-85f6-243e0d170896>

Hola Claude desde vscode01.jj@gmail.com

Acción 1:

```
01> Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (develop)
02> Shell $ echo "==">git fetch --all --prune at $(date +%Y%m%d%H%M%S)<==" \
03> Shell git fetch --all --prune \
04> Shell echo "==">git status at $(date +%Y%m%d%H%M%S)<==" \
05> Shell git status
06> Shell ==>git fetch --all --prune at 20260504093408<== git fetch --all --prune echo ==>git status
at 20260504093408<== git status
07> Shell
08> Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (develop)
09> Shell $ echo "==">git fetch --all --prune at $(date +%Y%m%d%H%M%S)<==" ;\
10> Shell git fetch --all --prune ;\
11> Shell echo "==">git status at $(date +%Y%m%d%H%M%S)<==" ;\
12> Shell git status
13> Shell ==>git fetch --all --prune at 20260504093502<==
14> Shell ==>git status at 20260504093503<==
15> Shell On branch develop
16> Shell Your branch is up to date with 'origin/develop'.
17> Shell
18> Shell Untracked files:
19> Shell (use "git add <file>..." to include in what will be committed)
20> Shell .obsidian/
21> Shell
22> Shell nothing added to commit but untracked files present (use "git add" to track)
23> Shell
24> Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (develop)
25> Shell $
26> Shell
```

Acción 2:

```
01> Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (develop)
02> Shell $ echo "==">$(date +%Y%m%d%H%M%S)<==" ;\
03> Shell # Estás en ~/Proyectos/Data_Analysis.Delo-sangeles (develop)
04> Shell echo "==">cp .env.example .env at $(date +%Y%m%d%H%M%S)<==" ;\
05> Shell cp .env.example .env # crear el .env de trabajo (editar si hace falta) ;\
06> Shell echo "==">npm install at $(date +%Y%m%d%H%M%S)<==" ;\
07> Shell npm install # instala todas las dependencias en node_modules/ ;\
08> Shell echo "==">npm run dev at $(date +%Y%m%d%H%M%S)<==" ;\
09> Shell npm run dev # arranca el servidor de desarrollo
10> Shell ==>20260504094732<==
11> Shell ==>cp .env.example .env at 20260504094732<==
12> Shell ==>npm install at 20260504094733<==
13> Shell npm warn deprecated rimraf@3.0.2: Rimraf versions prior to v4 are no longer supported
14> Shell npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not
use it. Check out lru-cache if you want a good and tested way to coalesce async requests by a key value, which
is much more comprehensive and powerful.
15> Shell npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely
publicized security vulnerabilities, which have been fixed in the current version. Please update. Support for
old versions may be purchased (at exorbitant rates) by contacting i@izs.me
16> Shell npm warn deprecated @humanwhocodes/config-array@0.13.0: Use @eslint/config-array instead
17> Shell npm warn deprecated @humanwhocodes/object-schema@2.0.3: Use @eslint/object-schema instead
18> Shell npm warn deprecated eslint@8.57.1: This version is no longer supported. Please see
https://eslint.org/version-support for other options.
19> Shell
20> Shell added 415 packages, and audited 416 packages in 2m
21> Shell
22> Shell 135 packages are looking for funding
23> Shell run `npm fund` for details
24> Shell
25> Shell 7 high severity vulnerabilities
26> Shell
27> Shell To address all issues, run:
28> Shell npm audit fix
29> Shell
30> Shell Run `npm audit` for details.
31> Shell npm notice
32> Shell npm notice New minor version of npm available! 11.11.0 -> 11.13.0
33> Shell npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.13.0
34> Shell npm notice To update run: npm install -g npm@11.13.0
35> Shell npm notice
36> Shell ==>npm run dev at 20260504094949<==
37> Shell
38> Shell > frontend-data-analysis@0.0.1 dev
39> Shell > vite
40> Shell
```

Descripción:

Fecha de creación:
20260501043700

Versión:
20260505092400

Página 22 de 72

```

41>_ Shell
42>_ Shell VITE v7.0.0 ready in 727 ms
43>_ Shell
44>_ Shell → Local: http://localhost:5173/
45>_ Shell → Network: use --host to expose
46>_ Shell → press h + enter to show help
47>_ Shell
48>_ Shell
49>_ Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (develop)
50>_ Shell $
51>_ Shell

```

Porfa Claude, disculpa el lío. He ejecutado en dos pasos a los que he llamado Acción 1 y 2. En vez de pegar toda la salida de la terminal, he editado las dos Acciones con números de línea en los que he vuelto a reiniciar la numeración.

En las líneas 42 a 48 de la Acción 2 se muestra la salida en la terminal de la llamada a vite de la línea 39. Aquí se ha devuelto el control de la terminal tras ctrl + c (No ctrl + d), pero la ventana estaba a la espera de introducir ¿comandos? en vite bajo PowerShell. Como me había perdido hice ^c (^c lo pongo como abreviatura de ctrl + c). Ver captura de pantalla:

c:\Users\Coder\Proyectos\P8-Data-Analyst\docs\promts\

Captura de pantalla 2026-05-04 095926.png

28.320 2026-05-04 10.00 -a--

¿Por qué se ha pasado desde Git Bash a PowerShell?, ahí no sabía cómo continuar (^c).

Gracias Claude desde vscode01.jj@gmail.com

20260504103321*

Hola JJ, no hay lío que disculpar – has descubierto un concepto importante que nadie te había explicado: **un servidor de desarrollo no es un comando que se ejecuta y termina; es un proceso que se queda corriendo**. Vamos por partes.

Lo que pasó (y por qué está bien)

npm install terminó correctamente – los warnings de "deprecated" y "7 high severity vulnerabilities" son ruido normal en proyectos frontend con dependencias antiguas; no son tu problema (son de dependencias indirectas que el dev de Delo-sangeles no ha actualizado). Y Vite arrancó perfectamente: VITE v7.0.0 ready in 727 ms y te dio la URL <http://localhost:5173/>.

Lo que faltó hacer antes del Ctrl+C: **abrir un navegador y visitar <http://localhost:5173/>**. Ese es el truco mental: la terminal "se queda colgada" porque el servidor está vivo escuchando peticiones; mientras tanto, tú abres Chrome/Firefox/Edge en paralelo y ahí ves la web. Cuando hayas terminado, vuelves a la terminal y Ctrl+C para apagar el servidor (eso sí lo hiciste bien – es la forma correcta de pararlo).

Tu instinto de pulsar Ctrl+C por sentirte perdido es razonable y sin daño: simplemente has parado el servidor sin haberlo llegado a usar.

El mensaje press h + enter to show help es Vite ofreciéndote un mini-menú interactivo mientras está corriendo. Si pulsas h y Enter, te lista atajos:

- r + Enter → reinicia el servidor
- u + Enter → muestra la URL otra vez
- o + Enter → abre el navegador automáticamente en la URL
- c + Enter → limpia la pantalla
- q + Enter → salir limpiamente (alternativa elegante a Ctrl+C)

Por qué viste "Windows PowerShell" en el título

Buena pregunta y bien observada. El comportamiento esperado es que Vite herede la terminal desde la que lo lanzaste (Git Bash). El que veas "Windows PowerShell" en la barra de título suele tener una de dos causas:

1. **Lo más probable:** estás en VS Code y VS Code tiene PowerShell configurado como *shell por defecto* del terminal integrado, mientras que tú abriste manualmente una pestaña de Git Bash aparte. Cuando un script npm lanza un subproceso, en Windows usa el script-shell configurado, que por defecto es cmd.exe o PowerShell. La ventana de salida puede mostrar el nombre del shell padre del proceso que la creó.
2. **Menos probable:** que npm config get script-shell devuelva PowerShell explícitamente.

Puedes comprobarlo con:

```
bash
npm config get script-shell
echo $SHELL
```

No es un problema funcional – Vite arrancó igual. Si te molesta el cambio de contexto, se puede forzar a Git Bash con npm config set script-shell "C:\\Program Files\\Git\\bin\\bash.exe", pero no lo recomiendo ahora: estás aprendiendo demasiadas cosas a la vez como para tocar configuración global de npm.

Sobre .obsidian/ apareciendo como untracked

.obsidian/ es la carpeta de configuración de [Obsidian](#) (la app de notas en Markdown). Probablemente abriste el directorio del repo con Obsidian en algún momento – quizá tienes un *vault* configurado para abrir cualquier carpeta – y Obsidian creó su carpeta de config ahí. Como no es tu repo y no vas a hacer push, **ignóralo**. Si quisieras silenciarlo en tu vista local sin tocar el .gitignore del repo (que no es tuyo), puedes añadirlo a tu *exclude* personal:

```
bash
echo ".obsidian/" >> .git/info/exclude
git status
```

Eso lo oculta solo para ti, sin afectar al repo.

Próximo paso: ver de verdad la web

Vamos a relanzar y esta vez sí ver el dashboard:

```
bash
# Estás en ~/Proyectos/Data_Analysis.Delo-sangeles (develop)
npm run dev
```

Cuando aparezca la URL <http://localhost:5173/>:

1. **No toques la terminal.** Déjala viva.
2. Abre tu navegador → pega <http://localhost:5173/> → Enter.
3. Mira qué carga. Probablemente: la estructura visual del dashboard (navbar, layout, gráficos vacíos o con error) porque no hay backend en localhost:8000 que sirva los datos.
4. **Abre la consola del navegador** (F12 → pestaña *Console* y *Network*) y observa los errores. Verás cosas tipo GET <http://localhost:8000/api/dashboard.net>:ERR_CONNECTION_REFUSED – eso confirma que el frontend funciona pero le falta el backend.
5. Haz capturas de pantalla de lo que veas (la UI cargada y los errores de consola). Te van a servir para el README como evidencia de "se ejecuta hasta este punto, falta backend".
6. Cuando termines, vuelve a la terminal y pulsa q + Enter (o Ctrl+C) para apagar.

Cuando tengas las capturas y notas de lo que ves en pantalla, las analizamos juntos y empezamos a redactar el README con datos reales – qué arranca, qué no arranca, qué se necesita para que arranque del todo.

Una nota mental: el hecho de que el frontend de Naiza funcione *visualmente* sin backend ya es información valiosa. Significa que el equipo puede desarrollar la UI en paralelo al backend, y que el siguiente cuello de botella del proyecto es definir y construir esa API que sirva los datos del análisis (la que tú/Andy/Juanma habéis generado en EDA y Power BI).

Hola Claude desde vscode01.jj@gmail.com

```
Acción 1 01>_ Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (develop)
Acción 1 02>_ Shell $ npm config get script-shell
Acción 1 03>_ Shell echo $SHELL
Acción 1 04>_ Shell echo ".obsidian/" >> .git/info/exclude
Acción 1 05>_ Shell git status
Acción 1 06>_ Shell null
Acción 1 07>_ Shell /usr/bin/bash
Acción 1 08>_ Shell On branch develop
Acción 1 09>_ Shell Your branch is up to date with 'origin/develop'.
Acción 1 10>_ Shell
Acción 1 11>_ Shell nothing to commit, working tree clean
Acción 1 12>_ Shell
Acción 1 13>_ Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (develop)
Acción 1 14>_ Shell $
Acción 1 15>_ Shell
```

La ejecución de:

npm run dev

va muy bien.

VITE v7.0.0 ready in 2811 ms

→ Local: http://localhost:5173/

→ Network: use --host to expose

→ press h + enter to show help

Bajo PowerShell.

Abre el navegador por defecto (Edge) y carga la página.

La ejecución anterior se había mostrado en otra pestaña.

He cerrado las pestañas de Edge y copiado la url:

http://localhost:5173/

En Firefox, mejores capturas de pantalla que Edge.

Al abrir desde Git Bash (sin vscode abierto) se pasa a PowerShell.

Ver Acción 1.

Se pierde el control de la terminal mientras dura la ejecución de:

npm run dev

El resultado en Firefox de

http://localhost:5173/

Es:

c:\Users\Coder\Proyectos\P8-Data-Analyst\docs\promts\
Screenshot 2026-05-04 at 11-01-46 Data Analysis - Dashboard.png

2.876.8072026-05-04 11.01 -a--

Muestra Gráficos Estadísticos Interactivos.

No hay errores de consola:

c:\Users\Coder\Proyectos\P8-Data-Analyst\docs\promts\
Captura de pantalla 2026-05-04 110812.png

76.811 2026-05-04 11.08 -a--

Captura de pantalla 2026-05-04 111241.png

79.826 2026-05-04 11.12 -a-

```
Acción 2 01>_ Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (develop)
Acción 2 02>_ Shell $ npm run dev
Acción 2 03>_ Shell
Acción 2 04>_ Shell > frontend-data-analysis@0.0.1 dev
Acción 2 05>_ Shell > vite
Acción 2 06>_ Shell
Acción 2 07>_ Shell
Acción 2 08>_ Shell VITE v7.0.0 ready in 2811 ms
Acción 2 09>_ Shell
Acción 2 10>_ Shell → Local: http://localhost:5173/
Acción 2 11>_ Shell → Network: use --host to expose
Acción 2 12>_ Shell → press h + enter to show help
Acción 2 13>_ Shell q
Acción 2 14>_ Shell
Acción 2 15>_ Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (develop)
Acción 2 16>_ Shell $
Acción 2 17>_ Shell
```

Gracias Claude desde vscode01.jj@gmail.com

20260504111837*

¿Dónde va exactamente el README que tienes que escribir?

1 de 2

- 1 En el repo del equipo P8_Data_Analysis (raíz)
- 2 En tu repo personal P8-Data-Analyst (sandbox)
- 3 En docs/ del equipo, junto a otros documentos
- 4 No lo tengo claro – ayúdame a decidir

Otra cosa

Supongo que irá en la raíz de P8_Data_Analysis.

Está sujeto a la aprobación y la decisión del resto de los miembros del equipo.

Nota: se me pasó indicarte que me mostrases las preguntas en formato Markdown como continuación de las respuestas que vas dando para que yo pueda copiarlas (tus preguntas y mis respuestas), registrarlas y responder. También querría indicarte que me dieras el tiempo que yo necesite para poder responder tras haber documentado y pensado. Disculpas.

¿Puedes ayudarme a crear las indicaciones necesarias para que pueda establecer esto en tu configuración global de forma que siempre actúes de esta manera?.

¿Qué debe describir el README?

2 de 2

- 1 Solo el frontend de Naiza y cómo levantarlo
- 2 Todo el proyecto P8 (EDA, dashboards, frontend, equipo)
- 3 Solo tu parte (Data Viz Expert, Issue #2)
- 4 No estoy seguro

Por ahora debo hacer lo que se me ha dicho ya que tenemos que cumplir unos plazos.

Entiendo que basta con describir el FrontEnd de Naiza de la forma más completa posible.

Más tarde, si se me indica aumentar el alcance del README.md, cumplir con lo que se me diga.

Estas preguntas son muy necesarias Claude. Muchas Gracias.

Por favor añade a la Nota anterior que realices las preguntas que necesites.

¡Excelente noticia, JJ! Esto es mucho mejor de lo que esperaba. Vamos por orden.

Lo que muestran tus capturas

El frontend funciona perfectamente y muestra datos reales – y esto es importante: en el dashboard de Firefox veo las 6 ciudades exactas del proyecto P8 (Sydney, New York, Madrid, London, Milan, Tokyo), 220K alojamientos, scatter de precio vs reseñas, ranking de barrios, mapa de Madrid con marcadores Leaflet, y comparación host vs superhost (73.83% / 26.17%). La consola está limpia: solo [vite] connecting... y [vite] connected. (eso es comunicación interna de Vite con el navegador para hot-reload, no un error). El panel "Red" está vacío porque lo abriste *después* de cargar la página – la pestaña Network solo registra peticiones desde el momento en que se abre. Si quieres ver las peticiones reales, abre F12 *antes* y luego recarga (Ctrl+R).

¿De dónde salen los datos si no hay backend?

Esto es lo verdaderamente interesante. El frontend tiene **datos de ejemplo (mock data) embebidos** que se usan cuando la llamada a la API falla. Es un patrón muy común en desarrollo frontend: permite construir y demostrar la UI sin depender del backend. Si quieres confirmar dónde está, échale un ojo a:

```
bash
cat src/services/apiClient.ts
ls src/components/
grep -r "mock\|fallback\|sample\|sydney\|Madrid" src/ --include="*.ts" --include="*.tsx" | head -30
```

Probablemente verás un try/catch en apiClient.ts o un useEffect en algún componente que devuelve datos hardcoded si axios falla. Esto es **información valiosa para tu README**: el frontend funciona en modo demo standalone.

Sobre el cambio a PowerShell

Has confirmado el dato clave: npm config get script-shell devuelve null y tu \$SHELL es /usr/bin/bash. Eso descarta una mala configuración. Lo que ocurre es un comportamiento conocido de **Git Bash en Windows con procesos Node interactivos**: Git Bash usa MinTTY, que no es una "consola Windows real". Cuando un proceso Node necesita capturar pulsaciones de tecla en crudo (como Vite, que escucha h + enter, q + enter, etc.), el runtime levanta una *consola Windows nativa* (PowerShell o conhost) para poder leer ese input correctamente. No es un fallo, es un workaround del propio Node/npm en Windows.

Para tu vida diaria: ignóralo. No afecta a nada. Si te molesta el cambio visual, hay dos opciones (no las apliques ahora, solo para que las tengas):

```
bash
# Opción A: usar winpty para que se quede en Git Bash
winpty npm run dev
```

Opción B: lanzarlo desde una terminal Windows nativa (PowerShell o cmd) directamente

Sobre "perder el control de la terminal"

Eso es esperado y normal: un servidor de desarrollo es un proceso de larga duración, no un comando que ejecuta y termina. Tres formas de convivir con ello:

- **Pestañas múltiples:** abre otra ventana de Git Bash (o pestaña en Windows Terminal) para seguir trabajando en paralelo. La que tiene Vite la dejas viva.
- **Atajos del propio Vite:** bien usado el q + Enter en tu Acción 2 – esa es la salida limpia.
- **Background:** en Linux funcionaría npm run dev &, pero en Git Bash sobre Windows da problemas con Node, no lo recomiendo.

Estamos listos para el README

Has cumplido los tres pasos de las instrucciones de Naiza:

1.  Clonar https://github.com/Delo-sangeles/Data_Analysis.git
2.  Crear README.md (siguiente)
3.  Intentar ejecutar en tu máquina el código → ¡funciona!

Antes de redactar, necesito que aclares una cosa para no ir al revés:

Mientras lo decides, una sugerencia: pídele a Naiza por el chat del equipo dos cosas concretas – (1) dónde quiere exactamente el README (ruta y rama) y (2) qué espera que cubra (alcance). Así no haces trabajo dos veces. Si prefieres, en mi siguiente respuesta te doy un mensaje corto para preguntárselo de forma profesional.

Has dicho: P: ¿Dónde va exactamente el README que tienes que escribir?

P: ¿Dónde va exactamente el README que tienes que escribir? R: Supongo que irá en la raíz de P8_Data_Analysis. Está sujeto a la aprobación y la decisión del resto de los miembros del equipo. Nota: se me pasó indicarte que me mostrases las preguntas en formato Markdown como continuación de las respuestas que vas dando para que yo pueda copiarlas (tus preguntas y mis respuestas), registrarlas y responder También querría indicarte que me dieras el tiempo que yo necesite para poder responder tras haber documentado y pensado. Disculpas. ¿Puedes ayudarme a crear las indicaciones necesarias para que pueda establecer esto en tu configuración global de forma que siempre actúes de esta manera?.

P: ¿Qué debe describir el README? R: Por ahora debo hacer lo que se me ha dicho ya que tenemos que cumplir unos plazos. Entiendo que basta con describir el FrontEnd de Naiza de la forma más completa posible. Más tarde, si se me indica aumentar el alcance del README.md, cumplir con lo que se me diga. Estas preguntas son muy necesarias Claude. Muchas Gracias. Por favor añade a la Nota anterior que realices las preguntas que necesites.

Mostrar menos

11:50

Claude ha respondido: Buenas decisiones, JJ.

Estructuró preferencias de comunicación y redactó directrices de memoria persistente

Estructuró preferencias de comunicación y redactó directrices de memoria persistente

Buenas decisiones, JJ. Anoto las preferencias en memoria primero y luego seguimos.

Usó 4 herramientas

Usó 4 herramientas

Hecho. Resumen de lo que acabo de guardar para futuras conversaciones:

1. Las preguntas irán siempre en formato Markdown P: / R: al final de la respuesta, no en cuadros interactivos.
2. Tomas el tiempo que necesites – no asumiré nada por silencio ni saltaré preguntas para "avanzar".
3. Haré las preguntas de clarificación que haga falta antes de meterme en trabajo significativo.

Si algún día quieres ajustar o añadir más reglas (por ejemplo: "respóndeme siempre en español", "usa nombres de archivo con tu convención Proyecto.acción.YYYYYMMDDHHMMSSd.ext", etc.), me lo dices y las añado.

Plan de trabajo para el README

Has fijado dos cosas con buen criterio:

- **Ubicación:** raíz de P8_Data_Analysis (sujeto a aprobación del equipo).

Descripción:

- **Alcance:** solo el frontend de Naiza, lo más completo posible. Si más adelante hay que ampliar, se amplía.

Antes de redactar, necesito un poco más de contexto operativo. Hagamos esto: yo te paso ahora un bloque de preguntas (formato P:/R: para que las guardes en tus prompts), tú las contestas cuando puedas, y cuando vuelvas con respuestas redacto un primer borrador del README contra el que iteramos.

Mientras tanto, también te dejo unas tareas opcionales de "investigación local" que puedes ir haciendo en paralelo cuando tengas un momento – todas son comandos de solo lectura, no modifican nada. La salida de esas investigaciones ayudará a que el README describa cosas reales y no genéricas.

Investigación local sugerida (opcional, cuando puedas)

bash

Estás en ~/Proyectos/Data_Analysis.Delo-sangeles (develop)

1. Confirmar de dónde salen los datos del dashboard (mock vs API)

echo "===== src/services/apiClient.ts ====="

cat src/services/apiClient.ts

echo "===== src/hooks/useApi.ts ====="

cat src/hooks/useApi.ts

echo "===== Componentes que cargan datos ====="

grep -rn "axios\|fetch\|mock\|sample\|sydney\|madrid" src/ --include="*.ts" --include="*.tsx" | head -40

2. Listar componentes principales y páginas

echo "===== Estructura src/ ====="

find src -type f \(-name "*.ts" -o -name "*.tsx" \) | sort

3. Ver el componente del mapa (Leaflet) y los gráficos

echo "===== MapChart.tsx ====="

cat src/components/MapChart.tsx

echo "===== Charts.tsx ====="

cat src/components/Charts.tsx

echo "===== Dashboard.tsx ====="

cat src/pages/Dashboard.tsx

4. Comprobar configuración Vite y TypeScript

echo "===== vite.config.ts ====="

cat vite.config.ts

echo "===== tsconfig.json ====="

cat tsconfig.json

5. Comprobar qué scripts existen y qué versiones efectivas se han instalado

echo "===== Scripts de package.json ====="

cat package.json | grep -A 10 '"scripts"'

No hace falta que ejecutes todo de golpe ni que me pegues toda la salida. Pega lo que quieras y cuando quieras. Si alguna de esas salidas es muy larga, me indicas y filtramos.

Preguntas para el README (formato copia/pega)

markdown

Preguntas de Claude para el README – 2026-05-04

P1: ¿En qué rama de P8_Data_Analysis se va a crear el README? ¿feature/readme → develop → main, o directo en otra rama?

R1:

P2: ¿Existe ya algún README en la raíz de P8_Data_Analysis que haya que reemplazar o complementar? Si existe, ¿puedes pegar su contenido actual?

R2:

P3: ¿Quién es la audiencia principal del README? (marca todas las que apliquen)

- [] Profesores/evaluadores del bootcamp Factoria F5
- [] El propio equipo (Naiza, Juanma, Andy, Pal, JJ)
- [] Personas externas que vean el repo en GitHub
- [] Otros estudiantes del bootcamp

R3:

P4: ¿Qué idioma prefieres para el README? (español, inglés, bilingüe)

R4:

P5: ¿El README debe mencionar explícitamente el repo origen del frontend (Delo-sangeles/Data_Analysis) y enlazarlo, o debe presentarse como si el frontend ya formara parte de P8_Data_Analysis?

R5:

P6: ¿El frontend de Naiza se va a integrar al final dentro de P8_Data_Analysis (subiendo el código) o va a quedar como repo externo referenciado?

R6:

P7: ¿Qué secciones consideras imprescindibles? (marca o añade)

- [] Descripción del proyecto
- [] Equipo y roles (Scrum)
- [] Stack tecnológico
- [] Requisitos previos (Node, npm, Git)
- [] Instalación paso a paso
- [] Cómo ejecutar en desarrollo
- [] Cómo construir para producción
- [] Estructura de carpetas
- [] Variables de entorno (.env)
- [] Endpoints de API esperados (futuros)

Descripción:

Fecha de creación:
20260501043700

Versión:
20260505092400

Página 28 de 72

- [] Estado actual (qué funciona, qué no)
- [] Capturas de pantalla
- [] Tablero Kanban / Issues / Roadmap
- [] Licencia
- [] Contribuciones
- [] Otros:

R7:

P8: ¿Quieres incluir capturas de pantalla del dashboard funcionando en el README? ¿Las tienes ya guardadas y dónde?

R8:

P9: ¿El README debe incluir el desglose de errores/warnings de npm install (deprecated, vulnerabilities) y cómo interpretarlos, o eso lo dejamos fuera?

R9:

P10: ¿Hay alguna nota especial del bootcamp/profesores sobre formato del README (extensión mínima/máxima, badges obligatorios, secciones obligatorias)?

R10:

Sin prisa. Cuando tengas las respuestas (parciales o completas) y/o algunas salidas de la investigación local, lo retomamos.

Preguntas de Claude para el README – 2026-05-04

P1: ¿En qué rama de P8_Data_Analysis se va a crear el README? ¿feature/readme → develop → main, o directo en otra rama?

R1:

En mi rama v_01, sujeto a aprobación del equipo.

P2: ¿Existe ya algún README en la raíz de P8_Data_Analysis que haya que reemplazar o complementar? Si existe, ¿puedes pegar su contenido actual?

R2:

Esta opción de valorará después. El README.md actual es un Placeholder,

P3: ¿Quién es la audiencia principal del README? (marca todas las que apliquen)

- [Y] Profesores/evaluadores del bootcamp Factoría F5
- [Y] El propio equipo (Naiza, Juanma, Andy, Pal, JJ)
- [Y] Personas externas que vean el repo en GitHub
- [Y] Otros estudiantes del bootcamp

R3:

Todos.

P4: ¿Qué idioma prefieres para el README? (español, inglés, bilingüe)

R4:

Español.

Valorar la opción de mostrar en Español e Inglés.

P5: ¿El README debe mencionar explícitamente el repo origen del frontend (Delo-sangeles/Data_Analysis) y enlazarlo, o debe presentarse como si el frontend ya formara parte de P8_Data_Analysis?

R5:

Sí, debe mostrar el repo origen Delo-sangeles/Data_Analysis.

P6: ¿El frontend de Naiza se va a integrar al final dentro de P8_Data_Analysis (subiendo el código) o va a quedar como repo externo referenciado?

R6:

El frontend sería repo aparte.

P7: ¿Qué secciones consideras imprescindibles? (marca o añade)

- [Y] Descripción del proyecto
 - [Y] Equipo y roles (Scrum)
 - [Y] Stack tecnológico
 - [N] Requisitos previos (Node, npm, Git)
 - [N] Instalación paso a paso
 - [N] Cómo ejecutar en desarrollo
 - [N] Cómo construir para producción
 - [N] Estructura de carpetas
 - [N] Variables de entorno (.env)
 - [N] Endpoints de API esperados (futuros)
 - [N] Estado actual (qué funciona, qué no)
 - [N] Capturas de pantalla
 - [Y] Tablero Kanban / Issues / Roadmap
 - [Y] Licencia
 - MIT
 - [Y] Contribuciones
 - [Y] Otros:
- Incluir:

Presentación Power BI : <https://drive.google.com/file/d/1i-yYdaCaJXoN9tgTbRrr-hGY1Zb4Lhlo/view?usp=sharing>

Es necesario Power BI Desktop/web, solo funciona en Windows.

R7:

P8: ¿Quieres incluir capturas de pantalla del dashboard funcionando en el README? ¿Las tienes ya guardadas y dónde?

No.

R8:

P9: ¿El README debe incluir el desglose de errores/warnings de npm install (deprecated, vulnerabilities) y cómo interpretarlos, o eso lo dejamos fuera?

No se incluye.

R9:

P10: ¿Hay alguna nota especial del bootcamp/profesores sobre formato del README (extensión mínima/máxima, badges obligatorios, secciones obligatorias)?

R10:

No.

Nota: Roles:

Juanma: Product Owner.

Naiza: Scrum Master.

Andy, JJ y Pal: Data Analyst.

Hola Claude desde vscode01.jj@gmail.com

```

Acción 1 001> Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (develop)
Acción 1 002> Shell $ echo "===>cat src/services/apiClient.ts at $(date +%Y%m%d%H%M%S)<==="
Acción 1 003> Shell cat src/services/apiClient.ts
Acción 1 004> Shell echo "===>ls src/components/ at $(date +%Y%m%d%H%M%S)<==="
Acción 1 005> Shell ls src/components/
Acción 1 006> Shell echo "===>grep -r "mock\\|fallback\\|sample\\|sydney\\|Madrid" src/ --include="*.ts" -
-include="*.tsx" | head -30 at $(date +%Y%m%d%H%M%S)<==="
Acción 1 007> Shell grep -r "mock\\|fallback\\|sample\\|sydney\\|Madrid" src/ --include="*.ts" --
include="*.tsx" | head -30
Acción 1 008> Shell ===>cat src/services/apiClient.ts at 20260504125135<===
Acción 1 009> Shell import axios, { AxiosInstance, AxiosError } from 'axios'
Acción 1 010> Shell import { ApiResponse, ChartData, DashboardMetrics, AnalyticsData } from '../types'
Acción 1 011> Shell
Acción 1 012> Shell const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:8000'
Acción 1 013> Shell
Acción 1 014> Shell class ApiClient {
Acción 1 015> Shell private client: AxiosInstance
Acción 1 016> Shell
Acción 1 017> Shell constructor() {
Acción 1 018> Shell     this.client = axios.create({
Acción 1 019> Shell         baseURL: API_URL,
Acción 1 020> Shell         timeout: parseInt(import.meta.env.VITE_API_TIMEOUT || '30000'),
Acción 1 021> Shell         headers: {
Acción 1 022> Shell             'Content-Type': 'application/json',
Acción 1 023> Shell         },
Acción 1 024> Shell     })
Acción 1 025> Shell
Acción 1 026> Shell     // Interceptor para errores
Acción 1 027> Shell     this.client.interceptors.response.use(
Acción 1 028> Shell         (response) => response,
Acción 1 029> Shell         (error: AxiosError) => {
Acción 1 030> Shell             console.error('API Error:', error.message)
Acción 1 031> Shell             return Promise.reject(error)
Acción 1 032> Shell         }
Acción 1 033> Shell     )
Acción 1 034> Shell }
Acción 1 035> Shell
Acción 1 036> Shell // Obtener datos del dashboard
Acción 1 037> Shell async getDashboardData(): Promise<ApiResponse<DashboardMetrics>> {
Acción 1 038> Shell     try {
Acción 1 039> Shell         const response = await this.client.get('/api/dashboard')
Acción 1 040> Shell         return response.data
Acción 1 041> Shell     } catch (error) {
Acción 1 042> Shell         throw this.handleError(error)
Acción 1 043> Shell     }
Acción 1 044> Shell }
Acción 1 045> Shell
Acción 1 046> Shell // Obtener datos de análisis
Acción 1 047> Shell async getAnalyticsData(): Promise<ApiResponse<AnalyticsData>> {
Acción 1 048> Shell     try {
Acción 1 049> Shell         const response = await this.client.get('/api/analytics')
Acción 1 050> Shell         return response.data
Acción 1 051> Shell     } catch (error) {
Acción 1 052> Shell         throw this.handleError(error)
Acción 1 053> Shell     }
Acción 1 054> Shell }
Acción 1 055> Shell
Acción 1 056> Shell // Obtener gráficos específicos
Acción 1 057> Shell async getChartData(chartId: string): Promise<ApiResponse<ChartData[]>> {
Acción 1 058> Shell     try {
Acción 1 059> Shell         const response = await this.client.get(`/api/charts/${chartId}`)
Acción 1 060> Shell         return response.data
Acción 1 061> Shell     } catch (error) {
Acción 1 062> Shell         throw this.handleError(error)
Acción 1 063> Shell     }
Acción 1 064> Shell }
Acción 1 065> Shell
Acción 1 066> Shell // Obtener datos con filtros
Acción 1 067> Shell async getFilteredData(filters: Record<string, any>):
Promise<ApiResponse<AnalyticsData>> {
Acción 1 068> Shell     try {
Acción 1 069> Shell         const response = await this.client.post('/api/analytics/filter', filters)
Acción 1 070> Shell         return response.data
Acción 1 071> Shell     } catch (error) {
Acción 1 072> Shell         throw this.handleError(error)
Acción 1 073> Shell     }
Acción 1 074> Shell }
Acción 1 075> Shell
Acción 1 076> Shell // Power BI - Obtener token de incrustación (si aplica)
Acción 1 077> Shell async getPowerBIToken(): Promise<ApiResponse<{ token: string }>> {
Acción 1 078> Shell     try {
Acción 1 079> Shell         const response = await this.client.get('/api/powerbi/token')
Acción 1 080> Shell         return response.data

```

Descripción:

```

Acción 1 081>_ Shell } catch (error) {
Acción 1 082>_ Shell throw this.handleError(error)
Acción 1 083>_ Shell }
Acción 1 084>_ Shell }
Acción 1 085>_ Shell
Acción 1 086>_ Shell private handleError(error: any) {
Acción 1 087>_ Shell if (error.response) {
Acción 1 088>_ Shell return new Error(error.response.data.message || 'Error en la solicitud')
Acción 1 089>_ Shell } else if (error.request) {
Acción 1 090>_ Shell return new Error('No se recibió respuesta del servidor')
Acción 1 091>_ Shell } else {
Acción 1 092>_ Shell return new Error('Error al procesar la solicitud')
Acción 1 093>_ Shell }
Acción 1 094>_ Shell }
Acción 1 095>_ Shell }
Acción 1 096>_ Shell
Acción 1 097>_ Shell export default new ApiClient()
Acción 1 098>_ Shell ===>ls src/components/ at 20260504125136<===
Acción 1 099>_ Shell Charts.tsx MapChart.tsx Navbar.tsx footer.tsx
Acción 1 100>_ Shell ===>grep -r mock|fallback|sample|sydney|Madrid src/ --include=*.ts --include=*.tsx
| head -30 at 20260504125137<===
Acción 1 101>_ Shell src/components/footer.tsx: Datos basados en el mercado Airbnb · Sydney
· New York · Madrid · London · Milan · Tokyo
Acción 1 102>_ Shell src/pages/Dashboard.tsx: { name: 'Madrid', value: 129 },
Acción 1 103>_ Shell src/pages/Dashboard.tsx: { neighbourhood: 'Abrantes', city: 'Madrid', price:
55, reviews: 18 },
Acción 1 104>_ Shell src/pages/Dashboard.tsx: { neighbourhood: 'Acacias', city: 'Madrid', price:
75, reviews: 85 },
Acción 1 105>_ Shell src/pages/Dashboard.tsx: { neighbourhood: 'Adelfas', city: 'Madrid', price:
90, reviews: 113 },
Acción 1 106>_ Shell src/pages/Dashboard.tsx: { neighbourhood: 'Aeropuerto', city: 'Madrid', price:
120, reviews: 42 },
Acción 1 107>_ Shell src/pages/Dashboard.tsx: { neighbourhood: 'Aguilas', city: 'Madrid', price:
85, reviews: 38 },
Acción 1 108>_ Shell src/pages/Dashboard.tsx: { neighbourhood: 'Almagro', city: 'Madrid', price:
150, reviews: 25 },
Acción 1 109>_ Shell src/pages/Dashboard.tsx: { neighbourhood: 'Almenara', city: 'Madrid', price:
200, reviews: 75 },
Acción 1 110>_ Shell src/pages/Dashboard.tsx: { neighbourhood: 'Almendrales', city: 'Madrid', price:
95, reviews: 20 },
Acción 1 111>_ Shell src/pages/Dashboard.tsx: { neighbourhood: 'Aluche', city: 'Madrid', price:
70, reviews: 15 },
Acción 1 112>_ Shell src/pages/Dashboard.tsx: { neighbourhood: 'Centro', city: 'Madrid', price:
220, reviews: 61 },
Acción 1 113>_ Shell src/pages/Dashboard.tsx: { neighbourhood: 'Retiro', city: 'Madrid', price:
180, reviews: 49 },
Acción 1 114>_ Shell src/pages/Dashboard.tsx: { neighbourhood: 'Salamanca', city: 'Madrid', price:
250, reviews: 50 },
Acción 1 115>_ Shell src/pages/Dashboard.tsx: { neighbourhood: 'Malasaña', city: 'Madrid', price:
130, reviews: 55 },
Acción 1 116>_ Shell src/pages/Dashboard.tsx: { neighbourhood: 'Lavapiés', city: 'Madrid', price:
100, reviews: 30 },
Acción 1 117>_ Shell src/pages/Dashboard.tsx: Sydney, New York, Madrid, London, Milan y
Tokyo – evaluando precios medios,
Acción 1 118>_ Shell src/pages/Dashboard.tsx: subtitle="Precio vs. reseñas · Madrid & London"
Acción 1 119>_ Shell src/pages/Dashboard.tsx: title="Distribución Geográfica – Madrid"
Acción 1 120>_ Shell src/pages/Dashboard.tsx: title="Mapa de precios por ubicación – Madrid"
Acción 1 121>_ Shell
Acción 1 122>_ Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (develop)
Acción 1 123>_ Shell $
Acción 1 124>_ Shell

```

```

Acción 2 01>_ Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (develop)
Acción 2 02>_ Shell $ echo $SHELL
Acción 2 03>_ Shell npm config get script-shell
Acción 2 04>_ Shell echo $SHELL
Acción 2 05>_ Shell /usr/bin/bash
Acción 2 06>_ Shell null
Acción 2 07>_ Shell /usr/bin/bash
Acción 2 08>_ Shell
Acción 2 09>_ Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/Data_Analysis.Delo-sangeles (develop)
Acción 2 10>_ Shell $
Acción 2 11>_ Shell

```

```

Acción 3 01>_ bash Consulta.sh > Consulta.sh.log 2>&1 & # 1. Confirmar de dónde salen los datos del
dashboard (mock vs API)
Acción 3 02>_ bash Consulta.sh > Consulta.sh.log 2>&1 & echo -e "\n====> src/services/apiClient.ts at
$(date +%Y%m%d%H%M%S) <====="
Acción 3 03>_ bash Consulta.sh > Consulta.sh.log 2>&1 & cat src/services/apiClient.ts
Acción 3 04>_ bash Consulta.sh > Consulta.sh.log 2>&1 & echo -e "\n====> src/hooks/useApi.ts at
$(date +%Y%m%d%H%M%S) <====="
Acción 3 05>_ bash Consulta.sh > Consulta.sh.log 2>&1 & cat src/hooks/useApi.ts
Acción 3 06>_ bash Consulta.sh > Consulta.sh.log 2>&1 & echo -e "\n====> Componentes que cargan datos
at $(date +%Y%m%d%H%M%S) <====="

```

Descripción:

Fecha de creación:
20260501043700Versión:
20260505092400

Página 32 de 72


```

Acción 3 07>_ bash Consulta.sh > Consulta.sh.log 2>&1 & grep -rn
"axios\|fetch\|mock\|sample\|sydney\|madrid" src/ --include="*.ts" --include="*.tsx" | head -40
Acción 3 08>_ bash Consulta.sh > Consulta.sh.log 2>&1 &
Acción 3 09>_ bash Consulta.sh > Consulta.sh.log 2>&1 & # 2. Listar componentes principales y páginas
Acción 3 10>_ bash Consulta.sh > Consulta.sh.log 2>&1 & echo -e "\n====> Estructura src/ at $(date
+%Y%m%d%H%M%S) <====="
Acción 3 11>_ bash Consulta.sh > Consulta.sh.log 2>&1 & find src -type f \( -name "*.ts" -o -name
"*.tsx" \) | sort
Acción 3 12>_ bash Consulta.sh > Consulta.sh.log 2>&1 &
Acción 3 13>_ bash Consulta.sh > Consulta.sh.log 2>&1 & # 3. Ver el componente del mapa (Leaflet) y
los gráficos
Acción 3 14>_ bash Consulta.sh > Consulta.sh.log 2>&1 & echo -e "\n====> MapChart.tsx at $(date
+%Y%m%d%H%M%S) <====="
Acción 3 15>_ bash Consulta.sh > Consulta.sh.log 2>&1 & cat src/components/MapChart.tsx
Acción 3 16>_ bash Consulta.sh > Consulta.sh.log 2>&1 & echo -e "\n====> Charts.tsx at $(date
+%Y%m%d%H%M%S) <====="
Acción 3 17>_ bash Consulta.sh > Consulta.sh.log 2>&1 & cat src/components/Charts.tsx
Acción 3 18>_ bash Consulta.sh > Consulta.sh.log 2>&1 & echo -e "\n====> Dashboard.tsx at $(date
+%Y%m%d%H%M%S) <====="
Acción 3 19>_ bash Consulta.sh > Consulta.sh.log 2>&1 & cat src/pages/Dashboard.tsx
Acción 3 20>_ bash Consulta.sh > Consulta.sh.log 2>&1 &
Acción 3 21>_ bash Consulta.sh > Consulta.sh.log 2>&1 & # 4. Comprobar configuración Vite y TypeScript
Acción 3 22>_ bash Consulta.sh > Consulta.sh.log 2>&1 & echo -e "\n====> vite.config.ts at $(date
+%Y%m%d%H%M%S) <====="
Acción 3 23>_ bash Consulta.sh > Consulta.sh.log 2>&1 & cat vite.config.ts
Acción 3 24>_ bash Consulta.sh > Consulta.sh.log 2>&1 & echo -e "\n====> tsconfig.json at $(date
+%Y%m%d%H%M%S) <====="
Acción 3 25>_ bash Consulta.sh > Consulta.sh.log 2>&1 & cat tsconfig.json
Acción 3 26>_ bash Consulta.sh > Consulta.sh.log 2>&1 &
Acción 3 27>_ bash Consulta.sh > Consulta.sh.log 2>&1 & # 5. Comprobar qué scripts existen y qué
versiones efectivas se han instalado
Acción 3 28>_ bash Consulta.sh > Consulta.sh.log 2>&1 & echo -e "\n====> Scripts de package.json at
$(date +%Y%m%d%H%M%S) <====="
Acción 3 29>_ bash Consulta.sh > Consulta.sh.log 2>&1 & cat package.json | grep -A 10 '"scripts"'

```

Buscar "====> " (Hay 10 coincidencia(s) en 1 archivo(s) de 1 búsqueda(s)) [Normal]

C:\Users\Coder\Proyectos\Data_Analysis.Delo-sangeles\Consulta.sh.log (10 coincidencias)

```

Línea 1: =====> src/services/apiClient.ts at 20260504133141 <=====
Línea 91: =====> src/hooks/useApi.ts at 20260504133141 <=====
Línea 183: =====> Componentes que cargan datos at 20260504133141 <=====
Línea 193: =====> Estructura src/ at 20260504133142 <=====
Línea 204: =====> MapChart.tsx at 20260504133142 <=====
Línea 280: }=====> Charts.tsx at 20260504133142 <=====
Línea 536: }=====> Dashboard.tsx at 20260504133142 <=====
Línea 782: }=====> vite.config.ts at 20260504133142 <=====
Línea 797: =====> tsconfig.json at 20260504133142 <=====
Línea 821: =====> Scripts de package.json at 20260504133143 <=====

```

c:\Users\Coder\Proyectos\Data_Analysis.Delo-sangeles\
Consulta.sh.log

30.351 2026-05-04 13.56 -a--

c:\Users\Coder\Proyectos\Data_Analysis.Delo-sangeles\
Consulta.sh

1.415 2026-05-04 13.55 -a--

c:\Users\Coder\Proyectos\P8-Data-Analyst\logs\
Cuestiones.20260504123745.md

2.835 2026-05-04 12.37 -a--

Gracias Claude desde vscode01.jj@gmail.com

20260504135950*

C:\Users\Coder\Proyectos\P8-Data-Analyst\logs\Cuestiones.20260504123745.md

Preguntas de Claude para el README – 2026-05-04

P1: ¿En qué rama de P8_Data_Analysis se va a crear el README? ¿feature/readme → develop → main, o directo en otra rama?

R1:

En mi rama v_01, sujeto a aprobación del equipo.

P2: ¿Existe ya algún README en la raíz de P8_Data_Analysis que haya que reemplazar o complementar? Si existe, ¿puedes pegar su contenido actual?

R2:

Esta opción de valorará después. El README.md actual es un Placeholder,

P3: ¿Quién es la audiencia principal del README? (marca todas las que apliquen)

- [Y] Profesores/evaluadores del bootcamp Factoría F5
- [Y] El propio equipo (Naiza, Juanma, Andy, Pal, JJ)
- [Y] Personas externas que vean el repo en GitHub
- [Y] Otros estudiantes del bootcamp

R3:

Todos.

P4: ¿Qué idioma prefieres para el README? (español, inglés, bilingüe)

R4:

Español.

Valorar la opción de mostrar en Español e Inglés.

P5: ¿El README debe mencionar explícitamente el repo origen del frontend (Delo-sangeles/Data_Analysis) y enlazarlo, o debe presentarse como si el frontend ya formara parte de P8_Data_Analysis?

R5:

Sí, debe mostrar el repo origen Delo-sangeles/Data_Analysis.

P6: ¿El frontend de Naiza se va a integrar al final dentro de P8_Data_Analysis (subiendo el código) o va a quedar como repo externo referenciado?

R6:

El frontend sería repo aparte.

P7: ¿Qué secciones consideras imprescindibles? (marca o añade)

- [Y] Descripción del proyecto
- [Y] Equipo y roles (Scrum)
- [Y] Stack tecnológico
- [N] Requisitos previos (Node, npm, Git)
- [N] Instalación paso a paso
- [N] Cómo ejecutar en desarrollo
- [N] Cómo construir para producción
- [N] Estructura de carpetas
- [N] Variables de entorno (.env)
- [N] Endpoints de API esperados (futuros)
- [N] Estado actual (qué funciona, qué no)
- [N] Capturas de pantalla
- [Y] Tablero Kanban / Issues / Roadmap
- [Y] Licencia
- MIT
- [Y] Contribuciones
- [Y] Otros:

Incluir:

Presentación Power BI : <https://drive.google.com/file/d/1i-yYdaCaJXoN9tgTbRrr-hGY1Zb4Lhlo/view?usp=sharing>
Es necesario Power BI Desktop/web, solo funciona en Windows.

R7:

P8: ¿Quieres incluir capturas de pantalla del dashboard funcionando en el README? ¿Las tienes ya guardadas y dónde?

No.

R8:

P9: ¿El README debe incluir el desglose de errores/warnings de npm install (deprecated, vulnerabilities) y cómo interpretarlos, o eso lo dejamos fuera?

No se incluye.

R9:

P10: ¿Hay alguna nota especial del bootcamp/profesores sobre formato del README (extensión mínima/máxima, badges obligatorios, secciones obligatorias)?

R10:

No.

Nota: Roles:

Juanma: Product Owner.

Naiza: Scrum Master.

Andy, JJ y Pal: Data Analyst.

20260504123745*

C:\Users\Coder\Proyectos\Data_Analysis.Delo-sangeles\Consulta.sh.log

```

===== src/services/apiClient.ts at 20260504135605 <=====
import axios, { AxiosInstance, AxiosError } from 'axios'
import { ApiResponse, ChartData, DashboardMetrics, AnalyticsData } from '../types'

const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:8000'

class ApiClient {
  private client: AxiosInstance

  constructor() {
    this.client = axios.create({
      baseURL: API_URL,
      timeout: parseInt(import.meta.env.VITE_API_TIMEOUT || '30000'),
      headers: {
        'Content-Type': 'application/json',
      },
    })

    // Interceptor para errores
    this.client.interceptors.response.use(
      (response) => response,
      (error: AxiosError) => {
        console.error('API Error:', error.message)
        return Promise.reject(error)
      }
    )
  }

  // Obtener datos del dashboard
  async getDashboardData(): Promise<ApiResponse<DashboardMetrics>> {
    try {
      const response = await this.client.get('/api/dashboard')
      return response.data
    } catch (error) {
      throw this.handleError(error)
    }
  }

  // Obtener datos de análisis
  async getAnalyticsData(): Promise<ApiResponse<AnalyticsData>> {
    try {
      const response = await this.client.get('/api/analytics')
      return response.data
    } catch (error) {
      throw this.handleError(error)
    }
  }

  // Obtener gráficos específicos
  async getChartData(chartId: string): Promise<ApiResponse<ChartData[]>> {
    try {
      const response = await this.client.get(`/api/charts/${chartId}`)
      return response.data
    } catch (error) {
      throw this.handleError(error)
    }
  }

  // Obtener datos con filtros
  async getFilteredData(filters: Record<string, any>): Promise<ApiResponse<AnalyticsData>> {
    try {
      const response = await this.client.post('/api/analytics/filter', filters)
      return response.data
    } catch (error) {
      throw this.handleError(error)
    }
  }

  // Power BI - Obtener token de incrustación (si aplica)
  async getPowerBIToken(): Promise<ApiResponse<{ token: string }>> {
    try {
      const response = await this.client.get('/api/powerbi/token')
      return response.data
    } catch (error) {
      throw this.handleError(error)
    }
  }

  private handleError(error: any) {
    if (error.response) {
      return new Error(error.response.data.message || 'Error en la solicitud')
    } else if (error.request) {

```

```

        return new Error('No se recibió respuesta del servidor')
      } else {
        return new Error('Error al procesar la solicitud')
      }
    }
  }
}

export default new ApiClient()

===== src/hooks/useApi.ts at 20260504135605 <=====
import { useState, useEffect } from 'react'

interface UseApiOptions {
  retry?: number;
  delay?: number;
}

export function useApi<T>({
  fetchFunction: () => Promise<T>,
  options: UseApiOptions = {}
}) {
  const [data, setData] = useState<T | null>(null)
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState<Error | null>(null)

  useEffect(() => {
    let isMounted = true
    let retries = 0
    const maxRetries = options.retry || 3

    const fetchData = async () => {
      try {
        setLoading(true)
        const result = await fetchFunction()
        if (isMounted) {
          setData(result)
          setError(null)
        }
      } catch (err) {
        if (isMounted) {
          retries++
          if (retries < maxRetries) {
            setTimeout(fetchData, options.delay || 1000)
          } else {
            setError(err instanceof Error ? err : new Error('Unknown error'))
          }
        }
      } finally {
        if (isMounted) {
          setLoading(false)
        }
      }
    }

    fetchData()

    return () => {
      isMounted = false
    }
  }, [])

  return { data, loading, error }
}

export function useLocalStorage<T>(key: string, initialValue: T) {
  const [storedValue, setStoredValue] = useState<T>(() => {
    try {
      const item = window.localStorage.getItem(key)
      return item ? JSON.parse(item) : initialValue
    } catch (error) {
      console.error('Error reading localStorage:', error)
      return initialValue
    }
  })

  const setValue = (value: T | ((val: T) => T)) => {
    try {
      const valueToStore = value instanceof Function ? value(storedValue) : value
      setStoredValue(valueToStore)
      window.localStorage.setItem(key, JSON.stringify(valueToStore))
    } catch (error) {
      console.error('Error writing to localStorage:', error)
    }
  }
}

```

```

    return [storedValue, setValue] as const
  }

export function useDebounce<T>(value: T, delay: number): T {
  const [debouncedValue, setDebouncedValue] = useState<T>(value)

  useEffect(() => {
    const handler = setTimeout(() => {
      setDebouncedValue(value)
    }, delay)

    return () => clearTimeout(handler)
  }, [value, delay])

  return debouncedValue
}

====> Componentes que cargan datos at 20260504135605 <====
src/hooks/useApi.ts:9:  fetchFunction: () => Promise<T>,
src/hooks/useApi.ts:21:    const fetchData = async () => {
src/hooks/useApi.ts:24:      const result = await fetchFunction()
src/hooks/useApi.ts:33:      setTimeout(fetchData, options.delay || 1000)
src/hooks/useApi.ts:45:    fetchData()
src/pages/Dashboard.tsx:73:const madridData = [
src/pages/Dashboard.tsx:225:    data={madridData}
src/services/apiClient.ts:1:import axios, { AxiosInstance, AxiosError } from 'axios'
src/services/apiClient.ts:10:  this.client = axios.create({

====> Estructura src/ at 20260504135606 <====
src/App.tsx
src/components/Charts.tsx
src/components/MapChart.tsx
src/components/Navbar.tsx
src/components/footer.tsx
src/hooks/useApi.ts
src/main.tsx
src/pages/Dashboard.tsx
src/services/apiClient.ts
src/utils/formatters.ts

====> MapChart.tsx at 20260504135606 <====
import { MapContainer, TileLayer, CircleMarker, Popup } from 'react-leaflet'

interface MapPoint {
  lat: number
  lng: number
  neighbourhood: string
  price: number
  reviews: number
}

interface MapProps {
  data: MapPoint[]
  title: string
}

export function PriceMap({ data, title }: MapProps) {
  const avgPrice = Math.round(data.reduce((a, b) => a + b.price, 0) / data.length)
  const maxPrice = Math.max(...data.map(d => d.price))
  const minPrice = Math.min(...data.map(d => d.price))

  const getColor = (price: number) => {
    if (price > 200) return '#dc2626' // rojo oscuro
    if (price > 100) return '#ef4444' // rojo
    return '#fca5a5' // rojo claro
  }

  const getRadius = (price: number) => {
    const min = minPrice
    const max = maxPrice
    return 4 + ((price - min) / (max - min)) * 14
  }

  return (
    <div className="bg-gray-900 rounded-xl p-6 max-w-5xl">
      <h3 className="text-lg font-semibold tracking-wide text-gray-100 border-b border-gray-700 pb-3 mb-4">
        {title}
      </h3>

      {/* Mapa */}
      <div className="rounded-xl overflow-hidden" style={{ height: '480px' }}>
        <MapContainer
          center={[40.4168, -3.7038]}
          zoom={11}

```

```

        style={{ height: '100%', width: '100%' }}
      >
      <TileLayer
        url="https://tile.openstreetmap.org/{z}/{x}/{y}.png"
        attribution='&copy; OpenStreetMap contributors'
      />
      {data.map((point, index) => (
        <CircleMarker
          key={index}
          center={[point.lat, point.lng]}
          radius={getRadius(point.price)}
          fillColor={getColor(point.price)}
          color="white"
          weight={0.5}
          fillOpacity={0.7}
        >
          <Popup>
            <div className="text-sm">
              <p className="font-bold">{point.neighbourhood}</p>
              <p>💰 Precio: <strong>${point.price}</strong></p>
              <p>🌟 Reseñas: <strong>{point.reviews}</strong></p>
            </div>
          </Popup>
        </CircleMarker>
      ))}
    </MapContainer>
  </div>
)
}
====> Charts.tsx at 20260504135606 <====
import {
  BarChart, Bar, XAxis, YAxis,
  CartesianGrid, Tooltip, ResponsiveContainer, Cell,
  ScatterChart, Scatter, ReferenceLine, ZAxis,
  PieChart, Pie, Legend
} from 'recharts'

// ——— Tipos ———

interface CityData {
  name: string
  value: number
}

interface Props {
  data: CityData[]
  title: string
  analysis: string
}

interface ScatterPoint {
  neighbourhood: string
  price: number
  reviews: number
  city: string
}

interface ScatterProps {
  data: ScatterPoint[]
  title: string
  analysis: string
}

interface BarHorizontalPoint {
  neighbourhood: string
  price: number
}

interface HorizontalProps {
  data: BarHorizontalPoint[]
  title: string
  threshold?: number
}

interface DonutData {
  name: string
  value: number
}

interface DonutProps {
  data: DonutData[]
  title: string

```

```

    subtitle?: string
    analysis: string
  }

// — Colores —————

const COLORS = ['#E8622A', '#E8956A', '#8FA882', '#7A9E6E', '#6BB5A0', '#4A9E8A']

// — Tooltips —————

function CustomTooltip({ active, payload, label }: any) {
  if (active && payload && payload.length) {
    return (
      <div className="bg-white/10 backdrop-blur-xl border border-white/20 rounded-xl p-4 shadow-lg shadow-black/40">
        <p className="text-white font-semibold mb-2">{label}</p>
        <p className="text-teal-300">
          Precio medio: <span className="font-bold text-white">€{payload[0].value}</span>
        </p>
      </div>
    )
  }
  return null
}

function ScatterTooltip({ active, payload }: any) {
  if (active && payload && payload.length) {
    const d = payload[0].payload
    return (
      <div className="bg-white/10 backdrop-blur-xl border border-teal-400/30 rounded-xl p-4 shadow-lg shadow-teal-500/20">
        <p className="text-teal-300 font-semibold">{d.neighbourhood}</p>
        <p className="text-gray-300 text-sm">
          Precio: <span className="text-white font-bold">€{d.price}</span>
        </p>
        <p className="text-gray-300 text-sm">
          Reseñas: <span className="text-white font-bold">{d.reviews}</span>
        </p>
        <p className="text-gray-400 text-xs mt-1">{d.city}</p>
      </div>
    )
  }
  return null
}

// — Gráfico 1: Barras verticales por ciudad —————

export function CityBarChart({ data, title, analysis }: Props) {
  return (
    <div className="flex flex-col gap-6 max-w-5xl">

      <div className="bg-white/5 backdrop-blur-2xl border border-white/10 rounded-2xl p-6 shadow-xl shadow-black/30">

        <h3 className="text-lg font-semibold text-white tracking-wide mb-4 border-b border-white/10 pb-3">
          {title}
        </h3>

        <ResponsiveContainer width="100%" height={300}>
          <BarChart data={data}>
            <CartesianGrid stroke="rgba(255,255,255,0.08)" vertical={false} />

            <XAxis dataKey="name" stroke="#9CA3AF" />
            <YAxis stroke="#9CA3AF" />

            <Tooltip
              content={<CustomTooltip />}
              cursor={{ fill: 'rgba(255,255,255,0.05)' }}
            />

            <Bar dataKey="value" radius={[6, 6, 0, 0]}>
              {data.map((_, index) => (
                <Cell key={index} fill="#2DD4BF" />
              ))}
            </Bar>
          </BarChart>
        </ResponsiveContainer>
      </div>

      <div className="bg-white/5 backdrop-blur-2xl border border-white/10 rounded-2xl p-6 shadow-md shadow-black/20">
        <h4 className="text-teal-400 text-sm uppercase tracking-wider mb-3">
          Análisis
        </h4>
        <p className="text-gray-200 text-sm leading-relaxed">{analysis}</p>
      </div>
    </div>
  )
}

```

Descripción:

Fecha de creación:
20260501043700

Versión:
20260505092400

Página 39 de 72

```

    </div>
  </div>
)
}

// — Gráfico 2: Scatter por barrio —————

export function NeighbourhoodScatter({ data, title, analysis }: ScatterProps) {
  return (
    <div className="flex flex-col gap-4 max-w-5xl">
      <div className="bg-gray-900 rounded-xl p-6">
        <h3 className="text-lg font-semibold tracking-wide text-gray-100 border-b border-gray-700 pb-3 mb-4">{title}</h3>
        <ResponsiveContainer width="100%" height={350}>
          <ScatterChart margin={{ top: 10, right: 20, left: 0, bottom: 20 }}>
            <CartesianGrid strokeDasharray="3 3" stroke="#1F3A3A" />
            <XAxis
              type="number" dataKey="price" name="Precio"
              stroke="#4ADECC" tick={{ fontSize: 11, fill: '#9CA3AF' }}
              label={{ value: 'Promedio de price', position: 'insideBottom', offset: -10, fill: '#9CA3AF',
fontSize: 12 }}
              domain={[0, 500]}
            />
            <YAxis
              type="number" dataKey="reviews" name="Reseñas"
              stroke="#4ADECC" tick={{ fontSize: 11, fill: '#9CA3AF' }}
              label={{ value: 'Promedio de number_of_reviews', angle: -90, position: 'insideLeft', fill:
'#9CA3AF', fontSize: 12 }}
              domain={[0, 120]}
            />
            <ZAxis range={[40, 40]} />
            <Tooltip content={<ScatterTooltip />} cursor={{ strokeDasharray: '3 3' }} />
            <ReferenceLine y={20} stroke="#3B82F6" strokeDasharray="4 4" strokeWidth={1} />
            <p data={data} fill="#2DD4BF" fillOpacity={0.8} />
          </ScatterChart>
        </ResponsiveContainer>
      </div>

      {/* Análisis debajo */}
      <div className="bg-gray-900 rounded-xl p-6">
        <h4 className="text-teal-400 text-sm font-medium uppercase tracking-wider mb-3">Análisis</h4>
        <p className="text-gray-200 text-sm leading-relaxed">{analysis}</p>
      </div>
    </div>
  )
}

// — Gráfico 3: Barras horizontales por barrio —————

export function NeighbourhoodHorizontalBar({ data, title, threshold = 300 }: HorizontalProps) {
  const sorted = [...data].sort((a, b) => b.price - a.price)

  return (
    <div className="bg-gray-900 rounded-xl p-6 max-w-5xl">
      <h3 className="text-white font-semibold text-lg mb-4">{title}</h3>
      <ResponsiveContainer width="100%" height={sorted.length * 28}>
        <BarChart data={sorted} layout="vertical" margin={{ top: 0, right: 60, left: 140, bottom: 20 }}>
          <CartesianGrid strokeDasharray="3 3" stroke="#374151" horizontal={false} />
          <XAxis
            type="number" stroke="#9CA3AF" tick={{ fontSize: 11, fill: '#9CA3AF' }}
            label={{ value: 'Promedio de price', position: 'insideBottom', offset: -10, fill: '#9CA3AF',
fontSize: 12 }}
          />
          <YAxis
            type="category" dataKey="neighbourhood"
            stroke="#9CA3AF" tick={{ fontSize: 11, fill: '#9CA3AF' }}
            width={135}
          />
          <Tooltip
            contentStyle={{ backgroundColor: '#1F2937', border: 'none', borderRadius: '8px', color: '#fff' }}
            formatter={(value: any) => [`$${value}`, 'Precio medio']}
          />
          <Bar dataKey="price" radius={[0, 4, 4, 0]}>
            {sorted.map((entry, index) => (
              <Cell key={index} fill={entry.price >= threshold ? '#3B82F6' : '#F97316'} />
            ))}
          </Bar>
        </BarChart>
      </ResponsiveContainer>
    </div>
  )
}

const DONUT_COLORS = ['#2DD4BF', '#0F766E']

```



```

export function DonutChart({ data, title, subtitle, analysis }: DonutProps) {
  const total = data.reduce((a, b) => a + b.value, 0)

  return (
    <div className="flex flex-col gap-4 max-w-5xl">
      <div className="bg-gray-900 rounded-xl p-6">
        <h3 className="text-lg font-semibold tracking-wide text-gray-100 border-b border-gray-700 pb-3 mb-2">{title}</h3>
        {subtitle && <p className="text-teal-400 text-sm mb-4">{subtitle}</p>}
        <ResponsiveContainer width="100%" height={350}>
          <PieChart>
            <Pie
              data={data}
              cx="50%"
              cy="50%"
              innerRadius={100}
              outerRadius={150}
              dataKey="value"
              label={({ name, value }) => `${value.toLocaleString()} (${((value / total) * 100).toFixed(2)}%)`}
              labelLine={true}
            >
              {data.map((_, index) => (
                <Cell key={index} fill={DONUT_COLORS[index % DONUT_COLORS.length]} />
              ))}
            </Pie>
            <Tooltip
              contentStyle={{ backgroundColor: '#1F2937', border: 'none', borderRadius: '8px', color: '#fff'
            }}
              formatter={(value: any) => [value.toLocaleString(), '']}
            />
            <Legend
              formatter={(value) => <span style={{ color: '#9CA3AF' }}>{value}</span>}
            />
          </PieChart>
        </ResponsiveContainer>
      </div>
      <div className="bg-gray-900 rounded-xl p-6">
        <h4 className="text-teal-400 text-sm font-medium uppercase tracking-wider mb-3">🇪🇺 Análisis</h4>
        <p className="text-gray-200 text-sm leading-relaxed">{analysis}</p>
      </div>
    </div>
  )
}

===== Dashboard.tsx at 20260504135606 =====
import { CityBarChart, NeighbourhoodScatter, NeighbourhoodHorizontalBar, DonutChart } from
'../components/Charts'
import { PriceMap } from '../components/MapChart'

const cityData = [
  { name: 'Sydney', value: 208 },
  { name: 'New York', value: 153 },
  { name: 'Madrid', value: 129 },
  { name: 'London', value: 122 },
  { name: 'Milan', value: 115 },
  { name: 'Tokyo', value: 91 },
]

const hostData = [
  { name: 'host', value: 161256 },
  { name: 'superhost', value: 57153 },
]

const scatterData = [
  { neighbourhood: 'Abrantes', city: 'Madrid', price: 55, reviews: 18 },
  { neighbourhood: 'Acacias', city: 'Madrid', price: 75, reviews: 85 },
  { neighbourhood: 'Adelfas', city: 'Madrid', price: 90, reviews: 113 },
  { neighbourhood: 'Aeropuerto', city: 'Madrid', price: 120, reviews: 42 },
  { neighbourhood: 'Aguilas', city: 'Madrid', price: 85, reviews: 38 },
  { neighbourhood: 'Almagro', city: 'Madrid', price: 150, reviews: 25 },
  { neighbourhood: 'Almenara', city: 'Madrid', price: 200, reviews: 75 },
  { neighbourhood: 'Almendrales', city: 'Madrid', price: 95, reviews: 20 },
  { neighbourhood: 'Aluche', city: 'Madrid', price: 70, reviews: 15 },
  { neighbourhood: 'Centro', city: 'Madrid', price: 220, reviews: 61 },
  { neighbourhood: 'Retiro', city: 'Madrid', price: 180, reviews: 49 },
  { neighbourhood: 'Salamanca', city: 'Madrid', price: 250, reviews: 50 },
  { neighbourhood: 'Malasaña', city: 'Madrid', price: 130, reviews: 55 },
  { neighbourhood: 'Lavapiés', city: 'Madrid', price: 100, reviews: 30 },
  { neighbourhood: 'Westminster', city: 'London', price: 320, reviews: 22 },
  { neighbourhood: 'Camden', city: 'London', price: 280, reviews: 18 },
  { neighbourhood: 'Shoreditch', city: 'London', price: 350, reviews: 14 },
  { neighbourhood: 'Brixton', city: 'London', price: 200, reviews: 8 },
  { neighbourhood: 'Hackney', city: 'London', price: 450, reviews: 6 },
]

```

```

const horizontalData = [
  { neighbourhood: 'Fort Wadsworth', price: 721 },
  { neighbourhood: 'Woodrow', price: 631 },
  { neighbourhood: 'San Andrés', price: 537 },
  { neighbourhood: 'Lucero', price: 505 },
  { neighbourhood: 'Hellin', price: 485 },
  { neighbourhood: 'TRE TORRI', price: 468 },
  { neighbourhood: 'Tribeca', price: 442 },
  { neighbourhood: 'Sea Gate', price: 440 },
  { neighbourhood: 'Hachioji Shi', price: 438 },
  { neighbourhood: 'Canillejas', price: 428 },
  { neighbourhood: 'Riverdale', price: 398 },
  { neighbourhood: 'Orcasur', price: 396 },
  { neighbourhood: 'Rosas', price: 393 },
  { neighbourhood: 'Arcos', price: 391 },
  { neighbourhood: "Prince's Bay", price: 369 },
  { neighbourhood: 'Chiyoda Ku', price: 339 },
  { neighbourhood: 'Battery Park City', price: 331 },
  { neighbourhood: 'Randall Manor', price: 319 },
  { neighbourhood: 'Pittwater', price: 308 },
  { neighbourhood: 'Flatiron District', price: 308 },
  { neighbourhood: 'Simancas', price: 307 },
  { neighbourhood: 'NoHo', price: 266 },
  { neighbourhood: 'SoHo', price: 260 },
  { neighbourhood: 'Midtown', price: 255 },
  { neighbourhood: 'BRERA', price: 251 },
  { neighbourhood: 'Kensington and Chelsea', price: 248 },
  { neighbourhood: 'Neponsit', price: 247 },
  { neighbourhood: 'Westminster', price: 247 },
  { neighbourhood: 'West Village', price: 241 },
  { neighbourhood: 'Piovera', price: 239 },
  { neighbourhood: 'Mosman', price: 238 },
]

const madridData = [
  { lat: 40.4168, lng: -3.7038, neighbourhood: 'Centro', price: 120, reviews: 45 },
  { lat: 40.4200, lng: -3.6900, neighbourhood: 'Retiro', price: 150, reviews: 38 },
  { lat: 40.4300, lng: -3.7100, neighbourhood: 'Salamanca', price: 200, reviews: 22 },
  { lat: 40.4100, lng: -3.7200, neighbourhood: 'Lavapiés', price: 80, reviews: 60 },
  { lat: 40.4250, lng: -3.6800, neighbourhood: 'Malasaña', price: 110, reviews: 50 },
  { lat: 40.4050, lng: -3.6950, neighbourhood: 'Atocha', price: 95, reviews: 35 },
  { lat: 40.4400, lng: -3.7000, neighbourhood: 'Almagro', price: 180, reviews: 18 },
  { lat: 40.3950, lng: -3.7100, neighbourhood: 'Usera', price: 65, reviews: 70 },
  { lat: 40.4500, lng: -3.6900, neighbourhood: 'Chamartín', price: 160, reviews: 25 },
  { lat: 40.4150, lng: -3.7300, neighbourhood: 'Carabanchel', price: 55, reviews: 80 },
]

function SectionHeader({ emoji, title, subtitle }: any) {
  return (
    <div className="flex items-center gap-4 mb-6">
      <div className="bg-white/5 border border-white/10 backdrop-blur-xl rounded-lg p-2 shadow-md">
        <span className="text-xl">{emoji}</span>
      </div>
      <div>
        <h2 className="text-xl font-semibold text-white tracking-tight">
          {title}
        </h2>
        <p className="text-gray-400 text-xs uppercase tracking-widest">
          {subtitle}
        </p>
      </div>
      <div className="flex-1 h-px bg-gradient-to-r from-white/10 to-transparent ml-4" />
    </div>
  )
}

export default function Dashboard() {
  return (
    <div className="min-h-screen bg-[radial-gradient(circle_at_top,_#0f172a,_#02061f)] text-white px-6 py-10 space-y-14">
      <div>
        <h1 className="text-4xl font-bold tracking-tight bg-gradient-to-r from-teal-400 via-cyan-400 to-blue-500 bg-clip-text text-transparent drop-shadow-[0_0_20px_rgba(45,212,191,0.3)]">
          Dashboard
        </h1>
        <p className="text-gray-400 text-sm tracking-widest uppercase mt-2">
          Análisis del mercado de alojamientos turísticos · 2026
        </p>
      </div>
    </div>
  )
}

```

Descripción:

Fecha de creación:
20260501043700

Versión:
20260505092400

Página 42 de 72

```

    { /* Resumen empresa */ }
    <div className="bg-white/5 backdrop-blur-2xl border border-white/10 rounded-2xl p-8 shadow-xl shadow-
black/30">

        <div className="flex items-start gap-6">

            <div className="bg-teal-500/10 border border-teal-400/30 rounded-xl p-4 shadow-md shadow-teal-
500/20">
                <span className="text-4xl drop-shadow-[0_0_10px_rgba(45,212,191,0.7)]"> 📊 </span>
            </div>

            <div className="flex-1">

                <span className="text-xs font-semibold tracking-widest uppercase text-teal-400 bg-teal-400/10 px-3
py-1 rounded-full border border-teal-400/20">
                    Informe Ejecutivo · 2026
                </span>

                <h2 className="text-white font-bold text-2xl mt-3 mb-3">
                    DataInsight Analytics
                </h2>

                <p className="text-gray-300 text-sm leading-relaxed max-w-3xl">
                    Somos una empresa especializada en el análisis del mercado de alojamientos turísticos a nivel
global.

                    Este dashboard presenta un análisis comparativo de{ ' ' }
                    <span className="text-white font-semibold">6 ciudades clave</span> –
                    Sydney, New York, Madrid, London, Milan y Tokyo – evaluando precios medios,
                    disponibilidad y rentabilidad por barrio. Nuestro objetivo es proporcionar{ ' ' }
                    <span className="text-teal-400 font-semibold">insights accionables</span>.
                </p>

                { /* Stats */ }
                <div className="border-t border-white/10 mt-6 pt-6 grid grid-cols-4 gap-4">

                    {[
                        { value: '6', label: 'Ciudades', color: 'text-white' },
                        { value: '220K', label: 'Alojamientos', color: 'text-teal-400' },
                        { value: '€136', label: 'Precio medio', color: 'text-blue-400' },
                        { value: '120', label: 'Días disp.', color: 'text-white' },
                    ].map((item, i) => (
                        <div
                            key={i}
                            className="bg-white/5 backdrop-blur-xl border border-white/10 rounded-xl p-4 text-center
hover:bg-white/10 transition-all shadow-md shadow-black/20"
                        >
                            <p className={`text-3xl font-bold ${item.color}`}>
                                {item.value}
                            </p>
                            <p className="text-gray-400 text-xs mt-1 uppercase tracking-wider">
                                {item.label}
                            </p>
                        </div>
                    ))}

                </div>

            </div>
        </div>

    </div>

    { /* Secciones */ }
    <section>
        <SectionHeader
            emoji="🌐"
            title="Comparativa de Precios por Ciudad"
            subtitle="Precio medio · 6 ciudades globales"
        />
        <CityBarChart
            data={cityData}
            title="Precio promedio por ciudad"
            analysis="Sydney lidera con el precio más alto (€208)..."
        />
    </section>

    <section>
        <SectionHeader
            emoji="🏠"
            title="Rentabilidad por Barrio"
            subtitle="Precio vs. reseñas · Madrid & London"
        />
        <NeighbourhoodScatter
            data={scatterData}
            title="Promedio de price vs reseñas"
            analysis="Los barrios con precios entre €50-€150 concentran mayor demanda..."
        />
    </section>

```

Descripción:

Fecha de creación:
20260501043700

Versión:
20260505092400

Página 43 de 72

```

    />
  </section>

  <section>
    <SectionHeader
      emoji="🏠"
      title="Ranking de Barrios por Precio"
      subtitle="Top barrios · precio medio"
    />
    <NeighbourhoodHorizontalBar
      data={horizontalData}
      title="Promedio de price por neighbourhood"
      threshold={300}
    />
  </section>

  <section>
    <SectionHeader
      emoji="📍"
      title="Distribución Geográfica – Madrid"
      subtitle="Mapa de precios"
    />
    <PriceMap
      data={madridData}
      title="Mapa de precios por ubicación – Madrid"
    />
  </section>

  <section>
    <SectionHeader
      emoji="👤"
      title="Comparación Host vs Superhost"
      subtitle="Distribución de anfitriones"
    />
    <DonutChart
      data={hostData}
      title="Comparación entre host y superhost"
      subtitle="Superhost = 4+ propiedades"
      analysis="El 73,83% de los alojamientos pertenecen a hosts regulares..."
    />
  </section>
</div>
)
}
====> vite.config.ts at 20260504135607 <=====
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    open: true,
  },
  build: {
    outDir: 'dist',
    sourcemap: false,
  },
})
====> tsconfig.json at 20260504135607 <=====
{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "esModuleInterop": true,
    "allowSyntheticDefaultImports": true,

    /* Bundler mode */
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "strict": true,
    "noEmit": true,
    "forceConsistentCasingInFileNames": true
  },
  "include": ["src"],
  "references": [{ "path": "./tsconfig.node.json" }]
}

```

Descripción:

Fecha de creación:
20260501043700

Versión:
20260505092400

Página 44 de 72

====> Scripts de package.json at 20260504135607 <====

```
"scripts": {  
  "dev": "vite",  
  "build": "tsc && vite build",  
  "lint": "eslint . --ext ts,tsx --report-unused-disable-directives --max-warnings 0",  
  "preview": "vite preview",  
  "type-check": "tsc --noEmit"  
},  
"dependencies": {  
  "@types/leaflet": "^1.9.21",  
  "axios": "^1.5.0",  
  "leaflet": "^1.9.4",
```

```
C:\Users\Coder\Proyectos\P8-Data-Analyst\logs\Consulta.sh
```

```
# 1. Confirmar de dónde salen los datos del dashboard (mock vs API)
echo -e "\n====> src/services/apiClient.ts at $(date +%Y%m%d%H%M%S) <====="
cat src/services/apiClient.ts
echo -e "\n====> src/hooks/useApi.ts at $(date +%Y%m%d%H%M%S) <====="
cat src/hooks/useApi.ts
echo -e "\n====> Componentes que cargan datos at $(date +%Y%m%d%H%M%S) <====="
grep -rn "axios\|fetch\|mock\|sample\|sydney\|madrid" src/ --include="*.ts" --include="*.tsx" | head -40

# 2. Listar componentes principales y páginas
echo -e "\n====> Estructura src/ at $(date +%Y%m%d%H%M%S) <====="
find src -type f \( -name "*.ts" -o -name "*.tsx" \) | sort

# 3. Ver el componente del mapa (Leaflet) y los gráficos
echo -e "\n====> MapChart.tsx at $(date +%Y%m%d%H%M%S) <====="
cat src/components/MapChart.tsx
echo -e "\n====> Charts.tsx at $(date +%Y%m%d%H%M%S) <====="
cat src/components/Charts.tsx
echo -e "\n====> Dashboard.tsx at $(date +%Y%m%d%H%M%S) <====="
cat src/pages/Dashboard.tsx

# 4. Comprobar configuración Vite y TypeScript
echo -e "\n====> vite.config.ts at $(date +%Y%m%d%H%M%S) <====="
cat vite.config.ts
echo -e "\n====> tsconfig.json at $(date +%Y%m%d%H%M%S) <====="
cat tsconfig.json

# 5. Comprobar qué scripts existen y qué versiones efectivas se han instalado
echo -e "\n====> Scripts de package.json at $(date +%Y%m%d%H%M%S) <====="
cat package.json | grep -A 10 '"scripts'"
```

Buenas, JJ. Toda la información recibida y registrada. Antes de redactar nada, hay un par de cosas que te debo comentar como hallazgos del análisis y una tensión en el alcance que conviene resolver. Sin prisa.

Hallazgos técnicos relevantes del log de investigación

He revisado lo que extrajiste y hay dos datos que vale la pena anotar (aunque, como verás más abajo, quizá no entren todos en el README):

1. **El frontend tiene dos capas de datos, y solo una está activa.** El archivo `src/services/apiClient.ts` define un cliente HTTP completo con axios apuntando a `http://localhost:8000/api/dashboard`, `/api/analytics`, `/api/charts/{id}`, `/api/analytics/filter` y `/api/powerbi/token`. Es decir, la API *está prevista* pero no implementada en el repo. Sin embargo, mirando `src/pages/Dashboard.tsx` los datos que viste en pantalla (Madrid · Centro · 220 €, Salamanca · 250 €, etc.) están **hardcodeados directamente en el componente**, no vienen de `apiClient`. Lo que esto significa: el dashboard funciona standalone porque los datos están dentro del propio código fuente; `apiClient.ts` es una pieza preparada para el día que exista un backend real.
2. **La estructura de componentes es la mínima viable.** Solo hay 4 componentes (`Charts.tsx`, `MapChart.tsx`, `Navbar.tsx`, `footer.tsx`), una página (`Dashboards.tsx`), un servicio (`apiClient.ts`), un hook (`useApi.ts`) y utilidades de formato. Es un proyecto sencillo y bien delimitado.
3. **Confirmación cruzada del comportamiento de PowerShell.** Tu segunda comprobación muestra de nuevo `script-shell = null` y `$SHELL = /usr/bin/bash`. Confirma que el cambio a PowerShell que viste no es configuración tuya: es comportamiento del propio Node.js en Windows cuando levanta una consola interactiva.

Una tensión en el alcance que conviene resolver antes de redactar

Tus respuestas son muy claras en cada pregunta individual, pero hay una pequeña tensión cuando las leo en conjunto:

- **Ubicación:** raíz de `P8_Data_Analysis` (el repo del equipo).
- **Alcance declarado antes:** "describir el frontend de Naiza de la forma más completa posible".
- **El frontend es un repo aparte (R6):** por tanto su código *no* va a estar dentro de `P8_Data_Analysis`.
- **Secciones marcadas con N:** requisitos previos, instalación, ejecución, build, estructura, `.env`, endpoints, estado actual, capturas.

Si juntas eso, lo que queda es: un README en el repo del equipo que apunta a un repo externo (el frontend) sin instrucciones técnicas para reproducirlo. Eso es perfectamente coherente como documento de presentación general del proyecto P8, pero no encaja con "describir el frontend lo más completo posible" – para eso harían falta justamente las secciones que has descartado.

Mi lectura más probable de tus instrucciones reales: lo que necesitas es un README de proyecto en `P8_Data_Analysis` que presente el conjunto del proyecto P8 (equipo, stack, contexto Airbnb 6 ciudades, enlaces a entregables externos como el frontend de Naiza y la presentación Power BI), **no un manual técnico del frontend**. La descripción técnica del frontend, si hace falta en algún momento, vivirá en el propio repo de Delo-sangeles.

Si esa lectura es correcta, los datos técnicos que encontré arriba se quedan fuera de este README (y los guardamos por si más adelante hace falta otro documento más detallado).

Esqueleto propuesto del README (a validar antes de redactar)

1. Título y subtítulo
2. Badges (license MIT, status, etc.) – opcional, ligero
3. Descripción del proyecto
 - Contexto: dataset Airbnb, 6 ciudades (Sydney, NY, Madrid, London, Milan, Tokyo)
 - Objetivo del análisis
4. Equipo y roles (Scrum)
 - Juanma – Product Owner
 - Naiza – Scrum Master
 - Andy, JJ, Pal – Data Analyst
5. Stack tecnológico
 - Análisis: Python, Jupyter, Pandas, Seaborn, Plotly
 - Visualización: Power BI, Looker Studio
 - Frontend: React + TypeScript + Vite + Tailwind (repo externo)
 - Gestión: GitHub, GitHub Projects (Kanban), Git Flow (feature → develop → main)
6. Entregables y enlaces externos
 - Frontend (repo Delo-sangeles/Data_Analysis): enlace
 - Presentación Power BI (Drive): enlace + nota de requisitos (Windows / Power BI Desktop o web)
 - Tablero Kanban del proyecto: enlace
7. Roadmap / Estado del Kanban
 - Resumen de fases o issues principales
8. Cómo contribuir
 - Flujo de ramas (feature → develop → main, PR review)
 - Convenciones de commits
9. Licencia
 - MIT

Si validas este esqueleto, paso a redactar contenido real. Si prefieres mover, añadir o quitar secciones, lo ajustamos antes.

Preguntas finales antes de redactar

markdown

Preguntas de Claude para el README – 2026-05-04 (segunda ronda)

P11: ¿Confirmas la lectura de "README de proyecto P8 a nivel global, con el frontend como entregable referenciado", o realmente quieres que el documento se centre solo en el frontend (en cuyo caso reabriríamos las secciones técnicas marcadas como N)?

R11:

P12: ¿La rama `v_01` ya existe en `P8_Data_Analysis` o hay que crearla? Y para que el flujo sea limpio: ¿partirá de main o de develop?

R12:

P13: Sobre los enlaces externos:

- URL exacta del repo del equipo `P8_Data_Analysis`:
- URL del tablero Kanban del equipo (no tu sandbox personal):
- URL del repo del frontend (Delo-sangeles/Data_Analysis): https://github.com/Delo-sangeles/Data_Analysis (¿confirmas?)

Descripción:

- URL de la presentación Power BI: <https://drive.google.com/file/d/1i-yYdaCaJXoN9tgTbRrr-hGY1Zb4Lhlo/view?usp=sharing> (¿confirmas?)
R13:

P14: Sobre los nombres del equipo: ¿uso solo nombres de pila como me los has pasado (Juanma, Naiza, Andy, JJ, Pal) o quieres apellidos / handles de GitHub asociados a cada uno?
R14:

P15: Para "Stack tecnológico" – ¿quieres que liste herramientas usadas por todo el equipo (incluyendo Power BI de Andy, etc.) o solo las que tú dominas/usas? El primer enfoque es más fiel al proyecto, el segundo más prudente para no atribuir cosas que no controlas.
R15:

P16: Para "Cómo contribuir" – ¿el equipo tiene ya convenciones acordadas sobre commits y PRs (formato de mensaje, plantilla de PR), o lo dejo en propuesta abierta sujeta a aprobación?
R16:

P17: ¿Quieres que genere el README como archivo descargable (formato `README.md`) listo para que lo coloques en la raíz del repo del equipo, o prefieres que te lo entregue inline en el chat para revisarlo antes?
R17:

P18: Sobre la versión bilingüe ES/EN que mencionaste valorar – ¿la dejamos para una iteración posterior o la abordamos de inicio? Si de inicio, ¿en archivos separados (`README.md` español + `README.en.md` inglés) o en un mismo archivo con secciones por idioma?
R18:
Tómate el tiempo que necesites. Cuando vuelvas con respuestas (parciales o completas) seguimos.

Hola Claude desde vscode01.jj@gmail.com

- Ubicación: raíz de P8_Data_Analysis (el repo del equipo).
Ahi irá este README.md, pero en mi rama. Yo haré una propuesta sencilla.
 - Alcance declarado antes: "describir el frontend de Naiza de la forma más completa posible".
Mi README.md no debe pisar el de Naiza, debe citar y delegar en lo ya hecho.
 - El frontend es un repo aparte (R6): por tanto su código no va a estar dentro de P8_Data_Analysis.
Juanma y Naiza plantearán si transferir el código y cuanto código. Mañana lo hablamos.
 - Secciones marcadas con N: requisitos previos, instalación, ejecución, build, estructura, .env, endpoints, estado actual, capturas.
- Si juntas eso, lo que queda es: un README en el repo del equipo que apunta a un repo externo (el frontend) sin instrucciones técnicas para reproducirlo. Eso es perfectamente coherente como documento de presentación general del proyecto P8, pero no encaja con "describir el frontend lo más completo posible" – para eso harían falta justamente las secciones que has descartado.
- He hablado con Juanma, la parte técnica de cada módulo es para sus autores. Cuando dije, de la forma más completa posible, quería decir eso, no puedo solapar con lo que ya ha hecho Naiza o Juanma. Del análisis que hemos hecho, hemos podido ir poniendo cotas y sacar dudas coherentes que comentar con el equipo. No debo solapar con el trabajo de mis compañeros, pero sí aprender y plantear dudas constructivas.
- README de proyecto en P8_Data_Analysis que presente el conjunto del proyecto P8 (equipo, stack, contexto Airbnb 6 ciudades, enlaces a entregables externos como el frontend de Naiza y la presentación Power BI), no un manual técnico del frontend. Las descripciones técnicas corresponden a sus autores. Yo hago lo que se me pide.

Esqueleto propuesto del README (a validar antes de redactar)

1. Título y subtítulo
2. Badges (license MIT, status, etc.) – opcional, ligero
3. Descripción del proyecto
 - Contexto: dataset Airbnb, 6 ciudades (Sydney, NY, Madrid, London, Milan, Tokyo)
 - Objetivo del análisis
4. Equipo y roles (Scrum)
 - Juanma – Product Owner
 - Naiza – Scrum Master
 - Andy, JJ, Pal – Data Analyst
5. Stack tecnológico
 - Análisis: Python, Jupyter, Pandas, Seaborn, Plotly
 - Visualización: Power BI. (Como plan alternativo a desarrollar en otra ocasión) => Looker Studio
 - Frontend: React + TypeScript + Vite + Tailwind (repo externo)
 - Gestión: GitHub, GitHub Projects (Kanban), Git Flow (feature → develop → main)
6. Entregables y enlaces externos
 - Frontend (repo Delo-sangeles/Data_Analysis): enlace
 - Presentación Power BI (Drive): enlace + nota de requisitos (Windows / Power BI Desktop o web)
 - Tablero Kanban del proyecto: enlace
7. Roadmap / Estado del Kanban
 - Resumen de fases o issues principales
8. Cómo contribuir
 - Flujo de ramas (feature → develop → main, PR review)
 - Convenciones de commits
9. Licencia
 - MIT

Si validas este esqueleto, paso a redactar contenido real. Si prefieres mover, añadir o quitar secciones, lo ajustamos antes.

Preguntas finales antes de redactar

markdown

Preguntas de Claude para el README – 2026-05-04 (segunda ronda)

P11: ¿Confirmas la lectura de "README de proyecto P8 a nivel global, con el frontend como entregable referenciado"?

R11:

Según conversación mantenida debo referenciar con los enlaces dados. Ver enlaces debajo.

P12: ¿La rama v_01 ya existe en P8_Data_Analysis o hay que crearla? Y para que el flujo sea limpio: ¿partirá de main o de develop?

R12:

https://github.com/Bootcamp-IA-P6/P8_Data_Analysis.git

https://github.com/Bootcamp-IA-P6/P8_Data_Analysis/tree/v_01

P13: Sobre los enlaces externos:

- URL exacta del repo del equipo P8_Data_Analysis:
https://github.com/Bootcamp-IA-P6/P8_Data_Analysis.git
 - URL del tablero Kanban del equipo (no tu sandbox personal):
<https://github.com/orgs/Bootcamp-IA-P6/projects/46/views/1>
 - URL del repo del frontend (Delo-sangeles/Data_Analysis): https://github.com/Delo-sangeles/Data_Analysis (¿confirmas?)
 - URL de la presentación Power BI: <https://drive.google.com/file/d/1i-yYdaCaJXoN9tgTbRrr-hGYlZb4Lhlo/view?usp=sharing> (¿confirmas?)
- Presentación Power BI : <https://drive.google.com/file/d/1i-yYdaCaJXoN9tgTbRrr-hGYlZb4Lhlo/view?usp=sharing>

R13:

P14: Sobre los nombres del equipo: ¿uso solo nombres de pila como me los has pasado (Juanma, Naiza, Andy, JJ, Pal) o quieres apellidos / handles de GitHub asociados a cada uno?

R14:

Esos nombres, si procede, se editarán manualmente.

P15: Para "Stack tecnológico" – ¿quieres que liste herramientas usadas por todo el equipo (incluyendo Power BI de Andy, etc.) o solo las que tú dominas/usas? El primer enfoque es más fiel al proyecto, el segundo más prudente para no atribuir cosas que no controlas.

Descripción:

R15:

Las herramientas usadas por todo el equipo.

P16: Para "Cómo contribuir" – ¿el equipo tiene ya convenciones acordadas sobre commits y PRs (formato de mensaje, plantilla de PR), o lo dejo en propuesta abierta sujeta a aprobación?

R16:

Propuesta abierta.

P17: ¿Quieres que genere el README como archivo descargable (formato `README.md`) listo para que lo coloques en la raíz del repo del equipo, o prefieres que te lo entregue inline en el chat para revisarlo antes?

R17:

Descargable.

P18: Sobre la versión bilingüe ES/EN que mencionaste valorar – ¿la dejamos para una iteración posterior o la abordamos de inicio? Si de inicio, ¿en archivos separados (`README.md` español + `README.en.md` inglés) o en un mismo archivo con secciones por idioma?

R18:

Castellano.

Tómate el tiempo que necesites. Cuando vuelvas con respuestas (parciales o completas) seguimos.

Gracias Claude desde vscode01.jj@gmail.com

20260504151418*

Perfecto JJ, todo cuadra. Resumen de lo que has cerrado:

- **Documento de proyecto general**, no manual técnico del frontend.
- **Citar y delegar** la parte técnica de cada módulo a sus autores (no solapar a Naiza ni a Juanma).
- En tu rama v_01 que ya existe, como **propuesta sencilla** sujeta a revisión.
- Castellano. MIT. Sin capturas. Sin desglose de warnings de npm.
- Stack del equipo completo. Power BI como herramienta actual; Looker Studio como plan alternativo futuro.
- Convenciones de contribución como propuesta abierta.
- Entregable como archivo descargable.

Voy a redactarlo y dejártelo listo para colocar en la raíz del repo en tu rama v_01.

Archivo creado, ejecutó un comando, leyó un archivo

Archivo creado, ejecutó un comando, leyó un archivo

Aquí tienes el README.md listo para colocar en la raíz del repo en tu rama v_01. Algunas notas sobre las decisiones que tomé al redactarlo, por si quieres ajustar algo:

Tono y posicionamiento. El documento se presenta explícitamente como una **propuesta tuya** sujeta a revisión del equipo (lo dice en el bloque final "Sobre este documento" y se reitera en las secciones de roadmap y contribuir). Esto cubre tu objetivo de "no solapar a Naiza ni a Juanma": el README inventaría y enlaza los entregables, pero **delega expresamente la documentación técnica de cada módulo a su autor**. Hay una frase de delegación al final de la sección 1, otra al inicio de la sección 4, y nominaciones explícitas ("Autora de referencia: Naiza", "Autor de referencia: Andy") en cada entregable.

Sección de equipo. He puesto los nombres de pila en orden: Juanma (PO), Naiza (SM), y los tres Data Analyst en orden alfabético (Andy, JJ, Pal). Si lo prefieres por otro criterio (por ejemplo, JJ al final del bloque de Data Analyst), editálo manualmente.

Stack. Incluye todo lo del equipo. He agrupado por área (análisis / visualización / frontend / gestión) para que no parezca una lista plana. He respetado tu indicación: Power BI como herramienta principal del entregable, Looker Studio mencionado solo como plan alternativo futuro no implementado.

Roadmap. Como no me diste detalles concretos de issues, he optado por un enfoque conservador: enlace el Kanban como fuente única de verdad y enumero las áreas grandes del proyecto (homogeneización, EDA, Power BI, frontend, documentación). Si tu equipo prefiere algo más detallado, se rellena después.

Contribuir. Toda la sección está marcada como **propuesta abierta**, exactamente como pediste.

Licencia. Texto MIT íntegro, con Copyright (c) 2026 Bootcamp IA P4 – Factoría F5 Madrid. Si el equipo prefiere otra atribución (por ejemplo nombrar a las personas, o a la organización GitHub Bootcamp-IA-P6), se cambia esa línea.

Cosas que NO incluí, por respeto al alcance que cerraste.

- Capturas de pantalla del frontend.
- Instrucciones de instalación, ejecución, build, .env, endpoints (todo eso vive en el repo de Delo-sangeles).
- Desglose técnico de los notebooks o de los dashboards Power BI.
- Versión en inglés.

Convención de nombrado. El archivo se llama README.md porque es el nombre que debe tener al colocarse en la raíz del repo. Si para tu archivo de propuesta/notas locales prefieres tu convención

P8.README_propuesta.YYYYMMDDHHMMSSd.md, puedes hacer una copia con ese nombre en tu directorio local de trabajo y mantener el README.md para el commit.

Próximos pasos sugeridos

bash

```
# 1. Asegurarte de estar en tu rama v_01 del repo del equipo
cd ~/Proyectos/P8_Data_Analysis      # o donde tengas tu clon del repo del equipo
git fetch --all --prune
git switch v_01
git pull
```

```
# 2. Colocar el README descargado en la raíz del repo (sustituyendo el placeholder)
# (cp desde tu carpeta de descargas o desde donde lo guardes)
```

```
# 3. Comprobar el render visual antes de hacer commit
# En VS Code: clic derecho sobre README.md -> "Open Preview" (Ctrl+Shift+V)
```

```
# 4. Cuando estés conforme:
git add README.md
echo "docs: propuesta inicial de README de proyecto en v_01" > /tmp/commit_msg.txt
git commit -F /tmp/commit_msg.txt
git push origin v_01
```

Después de eso, lo natural sería abrir una Pull Request hacia develop (o hacia main si así lo decide el equipo) y pedir revisión a Juanma y Naiza.

markdown

Preguntas de Claude tras la entrega del README – 2026-05-04

P19: ¿Quieres que revisemos juntos el render visual del README en VS Code antes de hacer commit, o lo prefieres revisar tú primero y volver con cambios?

R19:

P20: ¿Hay algún dato del archivo que quieras corregir en esta misma iteración antes de hacer push? (nombre, orden, sección, frase concreta...)

R20:

P21: Para la PR hacia `develop`, ¿quieres que te prepare un texto sugerido de descripción de la Pull Request (qué cambia, por qué, a quién se pide review)?

R21:

P22: Cuando el equipo dé feedback (mañana en la conversación con Juanma y Naiza), ¿quieres registrarlo en este chat para iterar el README en una v_02, o vas a abrir un chat nuevo para esa fase?

R22:

Tómate el tiempo que necesites para revisarlo. Si algo no encaja, lo iteramos.

Readme

Documento · MD

Descargar

Readme.md

P8 – Análisis del Mercado AirBnB

Análisis comparativo del mercado de alojamientos turísticos AirBnB en seis ciudades globales: Sydney, Nueva York, Madrid, Londres, Milán y Tokio.

Proyecto del Bootcamp IA P4 – Factoría F5 Madrid.

Tabla de contenidos

1. [Descripción del proyecto](#)
2. [Equipo y roles \(Scrum\)](#)
3. [Stack tecnológico](#)
4. [Entregables y enlaces externos](#)
5. [Roadmap y estado del proyecto](#)
6. [Cómo contribuir \(propuesta\)](#)
7. [Licencia](#)
8. [Sobre este documento](#)

1. Descripción del proyecto

P8 es un proyecto de análisis de datos cuyo objetivo es estudiar el mercado de alojamientos turísticos de AirBnB a nivel internacional, a partir de un conjunto de datos público que cubre seis ciudades de referencia:

Ciudad	Continente
Sydney	Oceanía
Nueva York	Norteamérica
Madrid	Europa
Londres	Europa
Milán	Europa
Tokio	Asia

El equipo aborda el problema desde tres frentes complementarios:

- **Análisis exploratorio de datos (EDA)** y homogeneización de los datasets de las distintas ciudades.
- **Visualización y reporting** mediante dashboards profesionales.
- **Aplicación web** para presentar los hallazgos de forma interactiva.

Las preguntas de negocio guía incluyen, entre otras: comparación de precios medios entre ciudades, distribución por barrio, rentabilidad relativa, peso del segmento *superhost* frente a hosts regulares y patrones de disponibilidad.

La descripción detallada de cada uno de los frentes (metodología, decisiones técnicas, hallazgos) corresponde a sus autores y vive en los entregables enlazados en la sección 4.

2. Equipo y roles (Scrum)

Persona	Rol
Juanma	Product Owner
Naiza	Scrum Master
Andy	Data Analyst
JJ	Data Analyst
Pal	Data Analyst

Los nombres de pila pueden completarse con apellidos y handles de GitHub si el equipo así lo decide.

3. Stack tecnológico

El proyecto combina varias áreas y herramientas. Cada miembro del equipo es referente de las tecnologías de su frente.

Análisis de datos

- **Python 3.13** como lenguaje base.
- **Jupyter Lab** para los notebooks de homogeneización y EDA.
- **Pandas** para manipulación tabular.
- **Seaborn y Plotly** para visualización exploratoria (gráficos estáticos e interactivos).
- **uv** como gestor de entornos y dependencias.

Visualización y reporting

- **Power BI Desktop y Power BI web** como herramienta principal del entregable de reporting.
- **Looker Studio** queda contemplado como plan alternativo de futuro (no implementado en esta iteración).

Aplicación web (frontend)

- **React** con **TypeScript**.
- **Vite** como servidor de desarrollo y empaquetador.
- **Tailwind CSS** para estilos.
- **Recharts y Leaflet** para gráficos y mapas.
- Repositorio externo, ver sección 4.

Gestión del proyecto

- **Git y GitHub** para control de versiones.
- **GitHub Projects (Kanban)** para seguimiento de tareas.
- Flujo de trabajo basado en ramas: `feature/*` → `develop` → `main`.

Descripción:

Fecha de creación:
20260501043700

Versión:
20260505092400

Página 53 de 72

4. Entregables y enlaces externos

Los entregables principales del proyecto están repartidos en distintos repositorios y soportes. Este README los inventaría y enlaza; la documentación técnica de cada uno reside en su propia ubicación y bajo la responsabilidad de su autor o autora.

Repositorio del equipo (este repositorio)

- URL: https://github.com/Bootcamp-IA-P6/P8_Data_Analysis
- Función: punto de entrada al proyecto y coordinación general.

Frontend – Aplicación web del Dashboard

- Repositorio: https://github.com/Delo-sangeles/Data_Analysis
- Stack: React + TypeScript + Vite + Tailwind + Recharts + Leaflet.
- Autora de referencia: Naiza.
- Documentación técnica de instalación y uso: disponible en el README.md del propio repositorio del frontend.

Presentación Power BI

- Enlace: <https://drive.google.com/file/d/1i-yYdaCaJXoN9tgTbRrr-hGY1Zb4Lhlo/view?usp=sharing>
- Autor de referencia: Andy.
- Requisitos para visualización:
 - Power BI Desktop (solo Windows) o Power BI Web.
 - Es necesario disponer de una cuenta de Microsoft / Power BI con acceso adecuado para abrir el archivo en su formato nativo, según la opción que se utilice.

Tablero Kanban del proyecto

- URL: <https://github.com/orgs/Bootcamp-IA-P6/projects/46/views/1>
- Uso: seguimiento de issues, asignación de tareas y estado actual del trabajo.

5. Roadmap y estado del proyecto

El estado actualizado del proyecto, las tareas en curso y el desglose por issues se mantienen en el **tablero Kanban** del proyecto:

<https://github.com/orgs/Bootcamp-IA-P6/projects/46/views/1>

Las grandes áreas de trabajo del proyecto son:

- Homogeneización de los datasets de las seis ciudades.
- Análisis exploratorio de datos (EDA).
- Construcción de dashboards en Power BI.
- Desarrollo de la aplicación web del frontend.
- Documentación, presentación final y entrega.

Cada área tiene una o varias issues asociadas en el tablero, con personas responsables asignadas.

6. Cómo contribuir (propuesta)

Esta sección recoge una **propuesta abierta** de convenciones de trabajo. Las decisiones definitivas las toma el equipo y pueden modificar lo aquí expuesto.

Flujo de ramas (Git Flow simplificado)

- main → rama estable, solo recibe merges desde develop para releases o entregas.
- develop → rama de integración del trabajo del equipo.
- feature/<descripción-corta> → rama de trabajo individual o de pareja para una issue concreta.

Flujo recomendado para una contribución:

1. Crear una rama feature/<descripción> desde develop.
2. Trabajar y hacer commits en la rama feature.
3. Abrir una **Pull Request** hacia develop.
4. Solicitar revisión a al menos una persona del equipo.
5. Tras aprobación, integrar en develop.
6. Las integraciones a main se acuerdan en equipo y se asocian a entregas o hitos.

Convenciones de commits (propuesta)

- Mensajes claros y en imperativo: "Añade gráfico de precios por barrio", "Corrige cálculo de mediana en Madrid".
- Un commit por unidad lógica de cambio cuando sea razonable.
- Se valora el uso de prefijos tipo *Conventional Commits* (feat:, fix:, docs:, chore:, refactor:) si el equipo así lo decide.

Pull Requests

- Título descriptivo y enlace a la issue del Kanban si aplica.
- Descripción breve de qué cambia y por qué.
- Capturas de pantalla cuando el cambio sea visual.

Tanto el formato de mensajes como la plantilla de PR son **propuestas**. Cualquier ajuste debe acordarse con el equipo.

7. Licencia

Este proyecto se distribuye bajo licencia MIT.
MIT License

Copyright (c) 2026 Bootcamp IA P4 – Factoría F5 Madrid

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell

Descripción:

Fecha de creación:
20260501043700

Versión:
20260505092400

Página 54 de 72

copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Nota: si el equipo decide adoptar otra licencia, esta sección debe actualizarse antes de la entrega final.

8. Sobre este documento

- **Tipo:** Propuesta de README de proyecto.
- **Autor de la propuesta:** JJ.
- **Rama:** [v 01](#).
- **Estado:** sujeto a revisión y aprobación del equipo.
- **Alcance:** documento de presentación general del proyecto P8. La descripción técnica detallada de cada módulo (frontend, dashboards Power BI, notebooks de análisis) corresponde a sus autores y vive en los entregables enlazados en la sección 4.
- **Idioma:** castellano. Una versión en inglés podría plantearse en una iteración futura si el equipo lo considera oportuno.

Hola Claude desde vscode01.jj@gmail.com

He corregido el README.md que me has ofrecido.

Somo la P6 que usa DataSets suministrados por la P4.

El autor del Trabajo de Power Bi para su análisis y producción es Juanma.

El archivo lo he renombrado como:

c:\Users\Coder\Proyectos\P8-Data-Analyst\docs\

README.Temp.20260504203351L.md

9.328 2026-05-04 20.36 -a--

Querría pedirte que tengas en cuenta estas observaciones y que generes un cuestionario en MarkDown que sea fácil de contestar, pero lo más completo posible.

El propósito de este cuestionario es poder personalizarlo en base a una reunión de equipo.

Dar opciones. Preferiblemente haz preguntas tipo Y/N. Completo pero fácil.

Si tienes que poner preguntas con contestaciones abiertas, pon ejemplos para facilitar las respuestas.

Incluye cuestiones sobre como describir la parte técnica de Naiza con lo que ya tenemos.

Incluye tecnologías.

El tiempo se echa encima y debo presentar lo que tengo.

Aunque el cuestionario se basa en el repo de Naiza, Juanma es el encargado de la presentación de Power Bi.

¿Puedes poner cuestiones de Power Bi para Juanma?.

Andy va a hablar sobre el EDA, ¿puedes poner cuestiones para Andy?.

Yo explicaré el README.md.

Añade una sección de comentarios libres para Pal.

Recuerda que con lo que ya tenemos, no puedo contextualizar más en lo que queda de tiempo.

En base a las respuestas que se den mañana por mis compañeros, ¿podrías generar mañana un README.md que se ajuste a esos detalles?.

Gracias Claude desde vscode01.jj@gmail.com

20260504210938*

Hola JJ. Anotadas las tres correcciones importantes:

1. **P6, no P4** (sois la cohorte que usa los datasets de P4).
2. **Juanma** es autor del Power BI (Naiza es Scrum Master, no autora del reporting).
3. **Andy** se encarga del EDA en la presentación.

Y sí: confirmado, mañana cuando me traigas las respuestas del equipo te genero el README.md actualizado en una sola pasada. Ahora vamos a por el cuestionario.

Te lo dejo descargable como archivo Markdown listo para llevar a la reunión, repartido por personas y con formato Y/N o multi-opción siempre que se puede. Las preguntas abiertas llevan ejemplos para no dejar a nadie en blanco.

Archivo creado, ejecuté un comando, leyó un archivo

Archivo creado, ejecuté un comando, leyó un archivo

Aquí lo tienes. Algunas notas para que vayas a la reunión con la cabeza ordenada:

Estructura. Está dividido en 14 bloques numerados (0-13). Cada bloque tiene un foco claro:

- **0** – Aprobación general (la pregunta más importante, va primero por si hay que detenerse a discutirla).
- **1** – Datos generales (P6 vs P4, ciudades, fuente del dataset).
- **2** – Equipo y roles (apellidos, handles GitHub, especializaciones).
- **3** – Stack tecnológico.
- **4** – Bloque dedicado a **Naiza** (frontend).
- **5** – Bloque dedicado a **Juanma** (Power BI).
- **6** – Bloque dedicado a **Andy** (EDA y notebooks).
- **7** – Roadmap y Kanban.
- **8** – Cómo contribuir.
- **9** – Licencia.
- **10** – Otros aspectos (capturas, badges, FAQ, bilingüe).
- **11** – **Comentarios libres de Pal** (sección dedicada como pediste).
- **12** – Comentarios libres del resto.
- **13** – Acuerdos finales de la reunión.

Decisiones de diseño que tomé.

- Convención **Y/N/NS** explicada al principio del documento, así nadie se queda mirando una marca rara.
- Casillas tipo - [] para que se puedan marcar rápido (en VS Code, Obsidian o cualquier visor Markdown se ven como checkboxes).
- Cuando una pregunta es abierta, **incluyo siempre ejemplos** entre paréntesis o tras → para que la persona no parta de cero.
- Los datos que ya tenemos confirmados aparecen como **predeterminados** en las preguntas (URLs, tu apellido y handle, las 6 ciudades, etc.), de forma que la respuesta por defecto es marcar Y. Eso ahorra tiempo.
- En el bloque de Naiza incluí una pregunta clave (P4.5) sobre si conviene mencionar lo que descubrimos juntos (datos hardcodeados + apiClient preparado para backend futuro) – es información valiosa pero **delicada**, así que la pongo como decisión suya y de Juanma, no como algo que tú impongas.
- El bloque de Juanma (P5.5) toca el tema de permisos del Drive, que es algo que suele dar problemas en evaluaciones y conviene comprobar antes.
- El bloque de Andy (P6.2) le pregunta directamente por los nombres de los notebooks, que es la pieza concreta que el README necesita para enlazar el EDA.

Tiempo estimado. Marqué 20-30 minutos. Si vais con prisa, los bloques **mínimos imprescindibles** son: 0 (aprobación), 4 (Naiza), 5 (Juanma), 6 (Andy) y 13 (acuerdos finales). El resto puede quedar en blanco y yo asumo lo que ya hay en el README actual.

Para mañana. Cuando vuelvas con el cuestionario respondido (o las partes que se hayan podido completar), me lo subes a este chat y genero el README.md v_02 ajustado a esas respuestas en una sola pasada. Si hay puntos pendientes los marco explícitamente en el documento como "TODO equipo" para que no se pierdan.

markdown
Preguntas de Claude tras entregar el cuestionario – 2026-05-04

P23: ¿Quieres que el cuestionario quede también versionado en algún sitio (en docs/ del repo de equipo, en tu sandbox personal, en tu carpeta local de prompts)?

R23:

P24: Cuando me traigas las respuestas mañana, ¿prefieres que actualice el README en una sola pasada (v_02 final) o que vayamos sección a sección por si hay debates?

R24:

P25: ¿Quieres que prepare ya el texto de la PR (de v_01 hacia develop) para tenerlo listo cuando el README esté aprobado, o lo dejamos para después de la reunión?

R25:

Que vaya bien la reunión mañana. Descansa, JJ.

Cuestionario equipo readme 20260504

Documento · MD

c:\Users\Coder\Proyectos\P8-Data-Analyst\docs\
 Cuestionario_equipo_README_20260504.md
 14.193 2026-05-04 21.28 -a--

Cuestionario de equipo – README del proyecto P8

Propósito: recoger las respuestas del equipo en la reunión para personalizar el README.md que JJ presentará en su rama v_01.

Cómo usarlo: marca con [X] las opciones elegidas, escribe texto libre donde se pida, y deja en blanco lo que no aplique. Las preguntas que se puedan saltar están marcadas con (*opcional*).

Formato de respuesta: Y = Sí, N = No, NS = No sé / Lo decidimos después.

Tiempo estimado de la reunión para responder: 20-30 minutos.

Bloque 0 – Aprobación general del README propuesto

P0.1: ¿Aprobáis el README propuesto en la rama v_01 como base de trabajo?

- ☐ Y – Aprobado tal cual.
- ☐ Y – Aprobado con cambios menores (ver bloques siguientes).
- ☐ N – Necesita cambios estructurales (especificar al final, sección "Comentarios libres").
- ☐ NS

P0.2: ¿La rama destino final del README es develop, main u otra?

- ☐ develop
- ☐ main
- ☐ Otra: _____

P0.3: ¿Quién o quiénes deben aprobar la PR antes de hacer merge?

- ☐ Solo Product Owner (Juanma).
- ☐ Solo Scrum Master (Naiza).
- ☐ Ambos (Juanma y Naiza).
- ☐ Todo el equipo.
- ☐ Otra configuración: _____

Bloque 1 – Datos generales del proyecto (a confirmar por el equipo)

P1.1: ¿Confirmáis "Bootcamp IA P6 – Factoría F5 Madrid" como identificador del bootcamp en el README?

- ☐ Y
- ☐ N → corrección: _____

P1.2: ¿Confirmáis que el proyecto se llama "P8 – Análisis del Mercado AirBnB"?

- ☐ Y
- ☐ N → nombre alternativo: _____

P1.3: ¿Las seis ciudades del análisis son Sydney, Nueva York, Madrid, Londres, Milán y Tokio?

- ☐ Y
- ☐ N → corrección: _____

P1.4: ¿Hay una **fuentes oficial del dataset** que debamos citar en el README? (*ejemplo: Inside Airbnb, Kaggle, dataset interno P4, etc.*)

- ☐ Y → fuente: _____
- ☐ N
- ☐ NS

P1.5: ¿Los datasets vienen suministrados por la promoción P4 y los usamos en P6? ¿Conviene mencionarlo explícitamente en el README?

- ☐ Y, mencionar.
- ☐ N, omitir.
- ☐ NS

Bloque 2 – Equipo y roles

P2.1: ¿Confirmáis los roles tal y como aparecen en el README?

- Juanma – Product Owner: ☐ Y ☐ N
- Naiza – Scrum Master: ☐ Y ☐ N
- Andy – Data Analyst: ☐ Y ☐ N
- JJ – Data Analyst: ☐ Y ☐ N
- Pal – Data Analyst: ☐ Y ☐ N

P2.2: ¿Queréis que aparezcan **apellidos** además de los nombres de pila?

- ☐ Y → indicar apellido de cada uno:
 - Juanma: _____
 - Naiza: _____
 - Andy: _____
 - JJ: Ramírez y Sánchez-Escobar
 - Pal: _____

- ☐ N, solo nombres de pila.

P2.3: ¿Queréis que aparezca el **handle de GitHub** de cada miembro?

- ☐ Y → indicar handle de cada uno:
 - Juanma: @_____
 - Naiza: @_____

Descripción:

- ☐ Andy: @_____
- ☐ JJ: @Jose-JulioRamirezSanchez-Escobar
- ☐ Pal: @_____

• ☐ N
 P2.4: ¿Queréis añadir una **sub-especialización** debajo del rol? (ejemplo: "Data Analyst – EDA y homogeneización", "Data Analyst – Visualización interactiva")

- ☐ Y → especialización de cada uno:
 - ☐ Andy: _____
 - ☐ JJ: _____
 - ☐ Pal: _____
- ☐ N

Bloque 3 – Stack tecnológico

P3.1: ¿La sección de **Análisis de datos** está completa? Marca lo que falte añadir:

- ☐ NumPy
- ☐ Matplotlib
- ☐ Scikit-learn
- ☐ SciPy
- ☐ Statsmodels
- ☐ Otro: _____
- ☐ Nada que añadir.

P3.2: ¿La sección de **Visualización y reporting** está bien planteada (Power BI principal, Looker Studio como alternativa futura)?

- ☐ Y
- ☐ N → corrección: _____

P3.3: ¿La sección del **frontend** refleja correctamente el stack del repo de Naiza?

- ☐ Y
- ☐ N → corrección: _____

P3.4: ¿Falta alguna herramienta de **gestión / colaboración** que usemos como equipo?

- ☐ Discord / Slack – ¿cuál? _____
- ☐ Notion / Google Drive
- ☐ Figma / Miro
- ☐ Otro: _____
- ☐ Nada que añadir.

P3.5: ¿Queréis que el README incluya **versiones específicas** de las herramientas? (ejemplo: Python 3.13, Node 18+, etc.)

- ☐ Y, indicar versiones siempre que se conozcan.
- ☐ N, solo nombres.

Bloque 4 – Frontend (Naiza)

Estas preguntas son para que Naiza valide cómo se referencia su trabajo en el README, sin entrar en su documentación técnica detallada.

P4.1: ¿Confirmas que el repositorio de referencia es https://github.com/Delo-sangeles/Data_Analysis?

- ☐ Y
- ☐ N → URL correcta: _____

P4.2: ¿La rama por defecto del repo del frontend es develop o main?

- ☐ develop
- ☐ main
- ☐ Otra: _____

P4.3: ¿Quieres que aparezcas en el README como "**Autora de referencia**" del frontend?

- ☐ Y
- ☐ N → forma preferida: _____

P4.4: ¿El stack listado (React + TypeScript + Vite + Tailwind + Recharts + Leaflet) está completo?

- ☐ Y
- ☐ N → falta o sobra: _____

P4.5: ¿Quieres que el README mencione que el frontend usa actualmente **datos hardcodeados de demostración** y que apiClient.ts está preparado para un backend futuro?

- ☐ Y, mencionarlo.
- ☐ N, omitirlo.
- ☐ Solo mencionarlo si Juanma está de acuerdo.

P4.6: ¿Quieres que el README enlace a algún documento concreto **dentro** del repo del frontend? (ejemplo: README.md, SETUP.md, BACKEND_EXAMPLES.md)

- ☐ Y → cuáles:
 - ☐ README.md
 - ☐ SETUP.md
 - ☐ BACKEND_EXAMPLES.md
 - ☐ Otro: _____

- ☐ N, basta con el enlace al repo.
- P4.7: ¿Hay alguna **captura de pantalla oficial** del dashboard que prefieras que se incluya, o ninguna por ahora?
- ☐ Y → ruta o enlace: _____
 - ☐ N, sin capturas.
- P4.8: ¿Algún comentario o aclaración sobre cómo describir tu trabajo en el README?

Bloque 5 – Power BI (Juanma)

Estas preguntas son para que Juanma valide cómo se referencia su trabajo en el README.

P5.1: ¿Confirmas que el enlace a la presentación es <https://drive.google.com/file/d/1i-yYdaCaJXoN9tgTbRrr-hGY1Zb4Lhlo/view?usp=sharing>?

- ☐ Y
 - ☐ N → URL correcta: _____
- P5.2: ¿El enlace de Drive es de **solo visualización** o de **descarga / edición**? ¿Tiene los permisos correctos para evaluadores externos?
- ☐ Solo visualización, permisos correctos.
 - ☐ Descarga, permisos correctos.
 - ☐ No estoy seguro de los permisos → revisar antes de la entrega.
- P5.3: ¿Quieres que aparezcas como **"Autor de referencia"** del entregable Power BI?

- ☐ Y
 - ☐ N → forma preferida: _____
- P5.4: ¿El requisito técnico que figura ("Power BI Desktop solo Windows, o Power BI Web") es correcto?
- ☐ Y
 - ☐ N → corrección: _____
- P5.5: ¿El archivo Power BI requiere alguna **cuenta concreta** o es accesible con cuenta personal de Microsoft?
- ☐ Cuenta personal de Microsoft basta.
 - ☐ Requiere cuenta de organización.
 - ☐ Otro: _____
- P5.6: ¿Quieres que el README incluya un **resumen de 2-3 líneas** de qué muestra la presentación Power BI? (ejemplo: "Comparativa de precios y ocupación en las 6 ciudades, con desglose por barrio y tipología de alojamiento")
- ☐ Y → resumen propuesto: _____

- ☐ N, basta con el enlace.
- P5.7: ¿Hay un **archivo .pbix** alojado en algún repositorio o solo está en Drive?
- ☐ Solo en Drive.
 - ☐ En Drive y en repo: _____
 - ☐ Otra ubicación: _____
- P5.8: ¿Algún comentario o aclaración sobre cómo describir tu trabajo en el README?

Bloque 6 – EDA y notebooks (Andy)

Estas preguntas son para que Andy valide cómo se referencia su trabajo en el README.

P6.1: ¿El EDA está alojado en el repo del equipo P8_Data_Analysis o en un repo aparte?

- ☐ En el repo del equipo, dentro de carpeta: _____
 - ☐ En repo aparte: _____
- P6.2: ¿Cuáles son los **notebooks principales** del EDA? (ejemplos típicos: *00_homogenizacion.ipynb*, *01_eda.ipynb*, *02_visualizaciones.ipynb*)
- Notebook 1: _____
 - Notebook 2: _____
 - Notebook 3: _____
 - Notebook 4: _____
- P6.3: ¿Quieres aparecer como **"Autor de referencia"** del EDA en el README?
- ☐ Y
 - ☐ N → forma preferida: _____
- P6.4: ¿Hay un **informe de conclusiones del EDA** (Markdown, PDF, Word) que debamos enlazar?
- ☐ Y → ruta o enlace: _____
 - ☐ N
- P6.5: ¿Quieres que el README mencione las **principales preguntas de negocio** que responde el EDA? (ejemplo: *precios medios por ciudad, distribución por barrio, peso del segmento superhost*)
- ☐ Y → preguntas: _____

- ☐ N, basta con el enlace.
- P6.6: ¿El EDA requiere algún **dataset descargado aparte** que no esté en el repo? ¿Dónde se documenta?
- ☐ Y → ubicación documentada en: _____
 - ☐ N, todo está en el repo.
- P6.7: ¿Algún comentario o aclaración sobre cómo describir tu trabajo en el README?

Bloque 7 – Roadmap y Kanban

Descripción:

Fecha de creación:
20260501043700

Versión:
20260505092400

Página 60 de 72

P7.1: ¿La URL del Kanban es <https://github.com/orgs/Bootcamp-IA-P6/projects/46/views/1?>

- ☐ Y
- ☐ N → URL correcta: _____

P7.2: ¿Las áreas grandes listadas en la sección 5 del README cubren bien el roadmap?

- ☐ Y
- ☐ N → áreas a añadir/quitar/renombrar: _____

P7.3: ¿Queréis añadir una **fecha de entrega** o una fecha de presentación visible en el README?

- ☐ Y → fecha: _____
- ☐ N

P7.4: ¿Queréis incluir un mini **estado actual** ("Fase 3 de 5: visualización en curso")?

- ☐ Y → estado a fecha de hoy: _____
- ☐ N

Bloque 8 – Cómo contribuir

P8.1: ¿Aprobáis el flujo de ramas propuesto (feature/* → develop → main)?

- ☐ Y
- ☐ N → flujo alternativo: _____

P8.2: ¿Adoptáis **Conventional Commits** (feat:, fix:, docs:, etc.)?

- ☐ Y, oficial.
- ☐ N, formato libre.
- ☐ Recomendado pero no obligatorio.

P8.3: ¿Vais a usar una **plantilla de Pull Request** (.github/PULL_REQUEST_TEMPLATE.md)?

- ☐ Y
- ☐ N
- ☐ NS

P8.4: ¿Vais a usar un **CODEOWNERS** o asignación automática de revisores?

- ☐ Y
- ☐ N
- ☐ NS

P8.5: ¿Cuántas **aprobaciones mínimas** se requieren para mergear a develop?

- ☐ 1
- ☐ 2
- ☐ Todo el equipo
- ☐ Sin requisito formal

Bloque 9 – Licencia y atribución

P9.1: ¿Confirmáis licencia MIT?

- ☐ Y
- ☐ N → otra licencia: _____

P9.2: ¿La línea de copyright debe poner "Bootcamp IA P6 – Factoría F5 Madrid", los nombres del equipo, o ambas?

- ☐ Solo Bootcamp IA P6 – Factoría F5 Madrid.
- ☐ Solo nombres del equipo (Juanma, Naiza, Andy, JJ, Pal).
- ☐ Ambas cosas.
- ☐ Otra: _____

Bloque 10 – Otros aspectos del README

P10.1: ¿Queréis incluir una sección de "**Capturas de pantalla**" del proyecto?

- ☐ Y → de qué (Power BI / frontend / notebooks): _____
- ☐ N

P10.2: ¿Queréis incluir una sección de "**Agradecimientos**" (profesorado, mentores, fuentes de datos)?

- ☐ Y → texto: _____
- ☐ N

P10.3: ¿Queréis añadir **badges adicionales** además de los actuales (License, Status, Bootcamp)?

- ☐ Python version
- ☐ Node version
- ☐ Última actualización
- ☐ Tamaño del repo
- ☐ Otros: _____
- ☐ N, basta con los actuales.

P10.4: ¿Queréis añadir una sección "**FAQ**" o "**Preguntas frecuentes**" para evaluadores?

- ☐ Y → temas: _____
- ☐ N

P10.5: ¿Queréis añadir un **diagrama de arquitectura** del proyecto (ASCII art o imagen)?

- ☐ Y → quién lo prepara: _____
- ☐ N

Descripción:

P10.6: ¿Versión bilingüe ES/EN?

- ☐ Sí, ambos idiomas en archivos separados (README.md y README.en.md).
- ☐ Sí, ambos idiomas en el mismo archivo.
- ☐ No, solo castellano (decisión actual).
- ☐ Más adelante, no en esta iteración.

Bloque 11 – Comentarios libres de Pal

Espacio para que Pal aporte cualquier observación, sugerencia, corrección, idea o duda sobre el README, el proyecto, el flujo de trabajo o cualquier otro aspecto que considere relevante.

Bloque 12 – Comentarios libres del resto del equipo

Espacio común para cualquier observación que no haya encajado en los bloques anteriores.

Juanma:

Naiza:

Andy:

JJ:

Bloque 13 – Acuerdos de la reunión (a rellenar al final)

P13.1: Próxima fecha de revisión del README: _____

P13.2: Persona responsable de aplicar los cambios al README: _____

P13.3: ¿Hay algún punto que requiere segunda reunión para decidir?

- ☐ Y → cuáles: _____
- ☐ N

P13.4: Versión del README tras esta ronda:

- ☐ v_02
 - ☐ Otra: _____
-

Cuestionario generado el 2026-05-04 para personalizar el README.md propuesto en la rama v_01 del repo Bootcamp-IA-P6/P8_Data_Analysis. Las respuestas servirán como insumo para generar la siguiente versión del README.

Descripción:

Fecha de creación:
20260501043700

Versión:
20260505092400

Página 62 de 72

Hola Claude desde vscode01.jj@gmail.com

Habría cosas que pulir sobre el cuestionario, pero el tiempo manda.

Espero poder hablarlo mañana en equipo.

Ahora debería poder hacer un push de:

```
c:\Users\Coder\Proyectos\P8-Data-Analyst\docs\  
Cuestionario_equipo_README_20260504.md
```

```
14.193 2026-05-04 21.28 -a--
```

```
c:\Users\Coder\Proyectos\P8-Data-Analyst\docs\  
README.Temp.20260504203351L.md
```

```
9.328 2026-05-04 20.36 -a--
```

a :

```
https://github.com/Bootcamp-IA-P6/P8_Data_Analysis.git
```

rama:

```
https://github.com/Bootcamp-IA-P6/P8_Data_Analysis/tree/v_01
```

Ojo, los archivos están en P8-Data-Analyst\docs\

```
Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/P8-Data-Analyst (main)
```

\$

Es distinto de:

```
Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/P8_Data_Analysis (v_01)
```

\$

Habría que pasarlos a

```
P8_Data_Analysis (v_01)
```

¿Puedes darme la secuencia de acciones?

Gracias Claude desde vscode01.jj@gmail.com

20260504213615*

Hola JJ. Bien visto el detalle de los dos directorios – son repos distintos y conviene no mezclarlos. Vamos paso a paso.

Lo que tienes

- **Origen** (tu sandbox personal): ~/Proyectos/P8-Data-Analyst/docs/
 - Cuestionario_equipo_README_20260504.md
 - README.Temp.20260504203351L.md
- **Destino** (repo del equipo, rama v_01): ~/Proyectos/P8_Data_Analysis/
 - Allí debe acabar el README.md (en la raíz) y el cuestionario probablemente en docs/.

Decisión previa: dónde y con qué nombre van los archivos en destino

Antes de mover nada, **una decisión:** el archivo README.Temp.20260504203351L.md debe llegar al repo del equipo como README.md en la raíz (porque GitHub solo renderiza como portada el archivo llamado exactamente README.md). Si lo dejas con el nombre Temp..., no se mostrará como portada.

Tu convención de nombres con timestamp es genial para tu flujo personal local, pero **el archivo committeado al repo del equipo debe llamarse README.md**. Mi propuesta:

Origen (sandbox)	Destino en repo del equipo	Notas
~/Proyectos/P8-Data-Analyst/docs/README.Temp.20260504203351L.md	~/Proyectos/P8_Data_Analysis/README.md	Sustituye el placeholder actual de main (que tras tu PR llegará también a v_01).
~/Proyectos/P8-Data-Analyst/docs/Cuestionario_equipo_README_20260504.md	~/Proyectos/P8_Data_Analysis/docs/Cuestionario_equipo_README_20260504.md	Mantiene tu nombre original con timestamp.

Si no te convence colocar el cuestionario en docs/ del repo del equipo (porque es un documento de trabajo interno y quizá no quieres exponerlo), dímelo y te doy una alternativa (por ejemplo, dejarlo solo en tu sandbox).

Secuencia de acciones

Te la divido en 5 fases, todas verificables paso a paso. Pega cada bloque, comprueba la salida, y solo entonces pasa al siguiente.

Fase 1 – Verificar el estado de partida

```
bash
echo "=>FASE 1: estado de partida at $(date +%Y%m%d%H%M%S)<==" ;\
echo "--- Repo destino: P8_Data_Analysis ---" ;\
cd ~/Proyectos/P8_Data_Analysis ;\
pwd ;\
git fetch --all --prune ;\
git status ;\
git branch -a ;\
echo "" ;\
echo "--- Repo origen (sandbox): P8-Data-Analyst/docs/ ---" ;\
ls -la ~/Proyectos/P8-Data-Analyst/docs/README.Temp.20260504203351L.md ~/Proyectos/P8-Data-Analyst/docs/Cuestionario_equipo_README_20260504.md
```

Qué busco confirmar antes de seguir:

- Estás en ~/Proyectos/P8_Data_Analysis.
- La rama v_01 aparece (local o remota).
- No hay cambios sin commitear pendientes (working tree clean).
- Los dos archivos origen existen donde dices.

Si algo no cuadra, paramos aquí y me lo dices antes de continuar.

Fase 2 – Cambiar a la rama v_01 y sincronizarla

```
bash
echo "=>FASE 2: cambiar a v_01 at $(date +%Y%m%d%H%M%S)<==" ;\
cd ~/Proyectos/P8_Data_Analysis ;\
git switch v_01 ;\
git pull origin v_01 ;\
git status ;\
ls -la
```

Qué busco confirmar:

- Estás en v_01 (el prompt mostrará (v_01)).
- git status dice "up to date with 'origin/v_01'" y "working tree clean".
- En ls -la ves el README.md placeholder actual.

Fase 3 – Copiar los archivos al repo del equipo

```
bash
echo "=>FASE 3: copiar archivos at $(date +%Y%m%d%H%M%S)<==" ;\
cd ~/Proyectos/P8_Data_Analysis ;\
mkdir -p docs ;\
cp ~/Proyectos/P8-Data-Analyst/docs/README.Temp.20260504203351L.md README.md ;\
cp ~/Proyectos/P8-Data-Analyst/docs/Cuestionario_equipo_README_20260504.md docs/Cuestionario_equipo_README_20260504.md ;\
echo "" ;\
echo "--- Verificación ---" ;\
ls -la README.md docs/Cuestionario_equipo_README_20260504.md ;\
echo "" ;\
echo "--- Primeras líneas del README.md copiado ---" ;\
head -10 README.md ;\
git status
```

Qué busco confirmar:

- README.md existe en la raíz y empieza por # P8 – Análisis del Mercado AirBnB.
- docs/Cuestionario_equipo_README_20260504.md existe.
- git status muestra ambos archivos como modificados o nuevos.

Nota sobre cp: estoy usando **copia**, no movimiento. Los archivos originales se quedan en tu sandbox como copia de seguridad. Si prefieres mv (mover), dímelo, pero recomiendo cp para esta primera vez.

Fase 4 – Commit en local

Aquí uso tu patrón habitual: mensaje de commit en archivo .txt separado y aplicado con git commit -F.

```
bash
echo "=>FASE 4: commit at $(date +%Y%m%d%H%M%S)<==" ;\
cd ~/Proyectos/P8_Data_Analysis ;\
mkdir -p .git/commit_msgs ;\
cat > .git/commit_msgs/commit_v_01_readme.txt <<'EOF'
docs: propuesta inicial de README de proyecto y cuestionario de equipo
```

- Reemplaza el README placeholder de la raíz por una propuesta de README de proyecto que presenta el conjunto del proyecto P8 (equipo, stack, contexto AirBnB 6 ciudades, enlaces a entregables externos) sin solapar con la documentación técnica de cada módulo, que reside en sus propios repositorios y bajo la responsabilidad de sus autores.

- Añade docs/Cuestionario_equipo_README_20260504.md como insumo para personalizar el README en la siguiente reunión de equipo.

Propuesta sujeta a revisión y aprobación por parte del equipo.

```
EOF
git add README.md docs/Cuestionario_equipo_README_20260504.md ;\
git status ;\
echo "" ;\
echo "--- Mensaje de commit ---" ;\
cat .git/commit_msgs/commit_v_01_readme.txt ;\
echo "" ;\
git commit -F .git/commit_msgs/commit_v_01_readme.txt ;\
git log --oneline -5
Qué busco confirmar:
```

- git status antes del commit muestra los dos archivos en *Changes to be committed*.
- El commit se crea limpiamente.
- En git log --oneline -5 ves tu commit nuevo en la cima.

Sin tildes ni caracteres raros en el mensaje a propósito – Git Bash en Windows a veces tiene problemas de encoding, así que mejor curarse en salud. Si quieres tildes, dímelo y te doy una variante con LC_ALL=C.UTF-8.

Fase 5 – Push a remoto

```
bash
echo "=>FASE 5: push at $(date +%Y%m%d%H%M%S)<==" ;\
cd ~/Proyectos/P8_Data_Analysis ;\
git push origin v_01 ;\
echo "" ;\
git status ;\
echo "" ;\
echo "URL para revisar la rama y abrir PR:" ;\
echo "https://github.com/Bootcamp-IA-P6/P8_Data_Analysis/tree/v_01" ;\
echo "https://github.com/Bootcamp-IA-P6/P8_Data_Analysis/compare/develop...v_01"
```

Qué busco confirmar:

- El push termina sin errores.
- git status dice "Your branch is up to date with 'origin/v_01'".
- Los dos enlaces que imprime te llevan a (1) ver tu rama en GitHub y (2) preparar la PR de v_01 hacia develop cuando quieras.

Resumen del orden

1. Fase 1: comprobar estado.
2. Fase 2: estar en v_01 sincronizada.
3. Fase 3: copiar archivos con los nombres correctos.
4. Fase 4: commit con mensaje en archivo.
5. Fase 5: push.

Pega la salida de cada fase y voy validando contigo. Si algo no cuadra en la Fase 1 (por ejemplo, no tienes ~/Proyectos/P8_Data_Analysis clonado en local todavía, o la rama v_01 no existe en local), paramos y resolvemos eso antes de tocar nada.

markdown

Preguntas de Claude antes de ejecutar la secuencia – 2026-05-04

P26: ¿Tienes ya clonado en local el repo del equipo en ~/Proyectos/P8_Data_Analysis? Si no, ¿quieres que te dé el clonado como Fase 0?

R26:

P27: ¿Confirmas que el cuestionario va en `docs/` del repo del equipo, o prefieres dejarlo solo en tu sandbox personal y no subirlo al repo del equipo?

R27:

P28: ¿Confirmas que en este push solo van README.md (raíz) + Cuestionario (docs/), o quieres incluir también algún otro archivo en el mismo commit (.gitignore, plantilla de PR, etc.)?

R28:

Hola Claude desde vscode01.jj@gmail.com

```
Fase1 01> Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/P8_Data_Analysis (v_01)
Fase1 02> Shell $ # Fase 1 – Verificar el estado de partida
Fase1 03> Shell echo "==>FASE 1: estado de partida at $(date +%Y%m%d%H%M%S)<==" ;\
Fase1 04> Shell echo "--- Repo destino: P8_Data_Analysis ---" ;\
Fase1 05> Shell cd ~/Proyectos/P8_Data_Analysis ;\
Fase1 06> Shell pwd ;\
Fase1 07> Shell git fetch --all --prune ;\
Fase1 08> Shell git status ;\
Fase1 09> Shell git branch -a ;\
Fase1 10> Shell echo "" ;\
Fase1 11> Shell echo "--- Repo origen (sandbox): P8-Data-Analyst/docs/ ---" ;\
Fase1 12> Shell ls -la ~/Proyectos/P8-Data-Analyst/docs/README.Temp.20260504203351L.md
~/Proyectos/P8-Data-Analyst/docs/Cuestionario_equipo_README_20260504.md
Fase1 13> Shell ==>FASE 1: estado de partida at 20260504215158<==
Fase1 14> Shell --- Repo destino: P8_Data_Analysis ---
Fase1 15> Shell /c/Users/Coder/Proyectos/P8_Data_Analysis
Fase1 16> Shell Fetching origin
Fase1 17> Shell From https://github.com/Bootcamp-IA-P6/P8_Data_Analysis
Fase1 18> Shell - [deleted] (none) -> origin/feature/v_01
Fase1 19> Shell remote: Enumerating objects: 7, done.
Fase1 20> Shell remote: Counting objects: 100% (7/7), done.
Fase1 21> Shell remote: Compressing objects: 100% (3/3), done.
Fase1 22> Shell remote: Total 7 (delta 4), reused 7 (delta 4), pack-reused 0 (from 0)
Fase1 23> Shell Unpacking objects: 100% (7/7), 3.16 MiB | 4.35 MiB/s, done.
Fase1 24> Shell e8f2b55..a4b743c feature/v_02 -> origin/feature/v_02
Fase1 25> Shell Fetching sandbox
Fase1 26> Shell remote: Enumerating objects: 137, done.
Fase1 27> Shell remote: Counting objects: 100% (116/116), done.
Fase1 28> Shell remote: Compressing objects: 100% (87/87), done.
Fase1 29> Shell remote: Total 137 (delta 27), reused 108 (delta 21), pack-reused 21 (from 1)
Fase1 30> Shell Receiving objects: 100% (137/137), 85.82 MiB | 3.01 MiB/s, done.
Fase1 31> Shell Resolving deltas: 100% (27/27), done.
Fase1 32> Shell From https://github.com/Jose-JulioRamirezzySanchez-Escobar/P8-Data-Analyst
Fase1 33> Shell * [new branch] main -> sandbox/main
Fase1 34> Shell On branch v_01
Fase1 35> Shell Your branch is up to date with 'origin/v_01'.
Fase1 36> Shell
Fase1 37> Shell Untracked files:
Fase1 38> Shell (use "git add <file>..." to include in what will be committed)
Fase1 39> Shell .obsidian/
Fase1 40> Shell P8_Data_Analysis.kanban_debug.20260416102006J.sh
Fase1 41> Shell P8_Data_Analysis.kanban_download.20260416042954J.sh
Fase1 42> Shell "Sin t\303\255tulo.md"
Fase1 43> Shell issues/
Fase1 44> Shell uv.lock
Fase1 45> Shell
Fase1 46> Shell nothing added to commit but untracked files present (use "git add" to track)
Fase1 47> Shell develop
Fase1 48> Shell main
Fase1 49> Shell * v_01
Fase1 50> Shell remotes/origin/HEAD -> origin/main
Fase1 51> Shell remotes/origin/develop
Fase1 52> Shell remotes/origin/feature/v_02
Fase1 53> Shell remotes/origin/feature/v_05
Fase1 54> Shell remotes/origin/main
Fase1 55> Shell remotes/origin/v_01
Fase1 56> Shell remotes/origin/v_04
Fase1 57> Shell remotes/sandbox/HEAD -> sandbox/main
Fase1 58> Shell remotes/sandbox/main
Fase1 59> Shell
Fase1 60> Shell --- Repo origen (sandbox): P8-Data-Analyst/docs/ ---
Fase1 61> Shell -rw-r--r-- 1 Coder 197121 14193 May 4 21:28 /c/Users/Coder/Proyectos/P8-Data-
Analyst/docs/Cuestionario_equipo_README_20260504.md
Fase1 62> Shell -rw-r--r-- 1 Coder 197121 9328 May 4 20:36 /c/Users/Coder/Proyectos/P8-Data-
Analyst/docs/README.Temp.20260504203351L.md
Fase1 63> Shell
Fase1 64> Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/P8_Data_Analysis (v_01)
Fase1 65> Shell $
Fase1 66> Shell

Fase2 01> Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/P8_Data_Analysis (v_01)
Fase2 02> Shell $ # Fase 2 – Cambiar a la rama v_01 y sincronizarla
Fase2 03> Shell echo "==>FASE 2: cambiar a v_01 at $(date +%Y%m%d%H%M%S)<==" ;\
Fase2 04> Shell cd ~/Proyectos/P8_Data_Analysis ;\
Fase2 05> Shell git switch v_01 ;\
Fase2 06> Shell git pull origin v_01 ;\
Fase2 07> Shell git status ;\
Fase2 08> Shell ls -la
Fase2 09> Shell ==>FASE 2: cambiar a v_01 at 20260504215658<==
Fase2 10> Shell Already on 'v_01'
Fase2 11> Shell Your branch is up to date with 'origin/v_01'.
Fase2 12> Shell From https://github.com/Bootcamp-IA-P6/P8_Data_Analysis
Fase2 13> Shell * branch v_01 -> FETCH_HEAD
```

Descripción:

```

Fase2 14>_ Shell Already up to date.
Fase2 15>_ Shell On branch v_01
Fase2 16>_ Shell Your branch is up to date with 'origin/v_01'.
Fase2 17>_ Shell
Fase2 18>_ Shell Untracked files:
Fase2 19>_ Shell (use "git add <file>..." to include in what will be committed)
Fase2 20>_ Shell .obsidian/
Fase2 21>_ Shell P8_Data_Analysis.kanban_debug.20260416102006J.sh
Fase2 22>_ Shell P8_Data_Analysis.kanban_download.20260416042954J.sh
Fase2 23>_ Shell "Sin t\303\255tulo.md"
Fase2 24>_ Shell issues/
Fase2 25>_ Shell uv.lock
Fase2 26>_ Shell
Fase2 27>_ Shell nothing added to commit but untracked files present (use "git add" to track)
Fase2 28>_ Shell total 121
Fase2 29>_ Shell drwxr-xr-x 1 Coder 197121 0 May 1 05:12 ./
Fase2 30>_ Shell drwxr-xr-x 1 Coder 197121 0 May 1 05:11 ../
Fase2 31>_ Shell drwxr-xr-x 1 Coder 197121 0 May 4 21:56 .git/
Fase2 32>_ Shell -rw-r--r-- 1 Coder 197121 766 Apr 15 12:08 .gitignore
Fase2 33>_ Shell drwxr-xr-x 1 Coder 197121 0 Apr 27 10:31 .obsidian/
Fase2 34>_ Shell -rw-r--r-- 1 Coder 197121 6 Apr 15 12:08 .python-version
Fase2 35>_ Shell drwxr-xr-x 1 Coder 197121 0 Apr 15 12:11 .venv/
Fase2 36>_ Shell -rwxr-xr-x 1 Coder 197121 1312 Apr 16 10:21
P8_Data_Analysis.kanban_debug.20260416102006J.sh*
Fase2 37>_ Shell -rwxr-xr-x 1 Coder 197121 9746 Apr 16 04:31
P8_Data_Analysis.kanban_download.20260416042954J.sh*
Fase2 38>_ Shell -rw-r--r-- 1 Coder 197121 20 Apr 15 11:57 README.md
Fase2 39>_ Shell -rw-r--r-- 1 Coder 197121 33444 Apr 27 10:40 'Sin titulo.md'
Fase2 40>_ Shell drwxr-xr-x 1 Coder 197121 0 Apr 15 12:11 data/
Fase2 41>_ Shell drwxr-xr-x 1 Coder 197121 0 Apr 27 12:26 docs/
Fase2 42>_ Shell drwxr-xr-x 1 Coder 197121 0 Apr 27 16:23 issues/
Fase2 43>_ Shell -rw-r--r-- 1 Coder 197121 100 Apr 15 12:08 main.py
Fase2 44>_ Shell drwxr-xr-x 1 Coder 197121 0 Apr 24 13:45 notebooks/
Fase2 45>_ Shell -rw-r--r-- 1 Coder 197121 169 Apr 15 12:08 pyproject.toml
Fase2 46>_ Shell -rw-r--r-- 1 Coder 197121 136 Apr 15 12:11 uv.lock
Fase2 47>_ Shell
Fase2 48>_ Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/P8_Data_Analysis (v_01)
Fase2 49>_ Shell $
Fase2 50>_ Shell

Fase3 01>_ Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/P8_Data_Analysis (v_01)
Fase3 02>_ Shell $ # Fase 3 – Copiar los archivos al repo del equipo
Fase3 03>_ Shell echo "==>FASE 3: copiar archivos at $(date +%Y%m%d%H%M%S)<==" ;\
Fase3 04>_ Shell cd ~/Proyectos/P8_Data_Analysis ;\
Fase3 05>_ Shell mkdir -p docs ;\
Fase3 06>_ Shell cp ~/Proyectos/P8-Data-Analyst/docs/README.Temp.20260504203351L.md README.md ;\
Fase3 07>_ Shell cp ~/Proyectos/P8-Data-Analyst/docs/Cuestionario_equipo_README_20260504.md
docs/Cuestionario_equipo_README_20260504.md ;\
Fase3 08>_ Shell echo "" ;\
Fase3 09>_ Shell echo "--- Verificación ---" ;\
Fase3 10>_ Shell ls -la README.md docs/Cuestionario_equipo_README_20260504.md ;\
Fase3 11>_ Shell echo "" ;\
Fase3 12>_ Shell echo "--- Primeras líneas del README.md copiado ---" ;\
Fase3 13>_ Shell head -10 README.md ;\
Fase3 14>_ Shell git status
Fase3 15>_ Shell ==>FASE 3: copiar archivos at 20260504220112<==
Fase3 16>_ Shell
Fase3 17>_ Shell --- Verificación ---
Fase3 18>_ Shell -rw-r--r-- 1 Coder 197121 9328 May 4 22:01 README.md
Fase3 19>_ Shell -rw-r--r-- 1 Coder 197121 14193 May 4 22:01
docs/Cuestionario_equipo_README_20260504.md
Fase3 20>_ Shell
Fase3 21>_ Shell --- Primeras líneas del README.md copiado ---
Fase3 22>_ Shell # P8 – Análisis del Mercado AirBnB
Fase3 23>_ Shell
Fase3 24>_ Shell > Análisis comparativo del mercado de alojamientos turísticos AirBnB en seis
ciudades globales: **Sydney, Nueva York, Madrid, Londres, Milán y Tokio**.
Fase3 25>_ Shell >
Fase3 26>_ Shell > Proyecto del **Bootcamp IA P6 – Factoría F5 Madrid**.
Fase3 27>_ Shell
Fase3 28>_ Shell [![License: MIT](https://img.shields.io/badge/License-MIT-yellow.svg)](#7-
licencia)
Fase3 29>_ Shell [![Status](https://img.shields.io/badge/status-en%20desarrollo-blue.svg)](#5-
roadmap-y-estado-del-proyecto)
Fase3 30>_ Shell [![Bootcamp](https://img.shields.io/badge/Factor%C3%ADa%20F5-Bootcamp%20IA%20P6-
orange.svg)](https://factoriaf5.org/)
Fase3 31>_ Shell
Fase3 32>_ Shell On branch v_01
Fase3 33>_ Shell Your branch is up to date with 'origin/v_01'.
Fase3 34>_ Shell
Fase3 35>_ Shell Changes not staged for commit:
Fase3 36>_ Shell (use "git add <file>..." to update what will be committed)
Fase3 37>_ Shell (use "git restore <file>..." to discard changes in working directory)
Fase3 38>_ Shell modified: README.md
Fase3 39>_ Shell

```

```

Fase3 40> Shell Untracked files:
Fase3 41> Shell (use "git add <file>..." to include in what will be committed)
Fase3 42> Shell .obsidian/
Fase3 43> Shell P8_Data_Analysis.kanban_debug.20260416102006J.sh
Fase3 44> Shell P8_Data_Analysis.kanban_download.20260416042954J.sh
Fase3 45> Shell "Sin t\303\255tulo.md"
Fase3 46> Shell docs/Cuestionario_equipo_README_20260504.md
Fase3 47> Shell issues/
Fase3 48> Shell uv.lock
Fase3 49> Shell
Fase3 50> Shell no changes added to commit (use "git add" and/or "git commit -a")
Fase3 51> Shell
Fase3 52> Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/P8_Data_Analysis (v_01)
Fase3 53> Shell $
Fase3 54> Shell

Fase4 01> Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/P8_Data_Analysis (v_01)
Fase4 02> Shell $ head -v -n -1 Fase4.txt ; bash $_
Fase4 03> Shell ==> Fase4.txt <==
Fase4 04> Shell # Fase 4 - Commit en local
Fase4 05> Shell # Aquí uso tu patrón habitual: mensaje de commit en archivo .txt separado y
aplicado con git commit -F.
Fase4 06> Shell echo "==>FASE 4: commit at $(date +%Y%m%d%H%M%S)<==" ;\
Fase4 07> Shell cd ~/Proyectos/P8_Data_Analysis ;\
Fase4 08> Shell mkdir -p .git/commit_msgs ;\
Fase4 09> Shell cat > .git/commit_msgs/commit_v_01_readme.txt <<'EOF'
Fase4 10> Shell docs: propuesta inicial de README de proyecto y cuestionario de equipo
Fase4 11> Shell
Fase4 12> Shell - Reemplaza el README placeholder de la raiz por una propuesta de README de
Fase4 13> Shell proyecto que presenta el conjunto del proyecto P8 (equipo, stack,
Fase4 14> Shell contexto AirBnB 6 ciudades, enlaces a entregables externos) sin solapar
Fase4 15> Shell con la documentacion tecnica de cada modulo, que reside en sus propios
Fase4 16> Shell repositorios y bajo la responsabilidad de sus autores.
Fase4 17> Shell
Fase4 18> Shell - Anade docs/Cuestionario_equipo_README_20260504.md como insumo para
Fase4 19> Shell personalizar el README en la siguiente reunion de equipo.
Fase4 20> Shell
Fase4 21> Shell Propuesta sujeta a revision y aprobacion por parte del equipo.
Fase4 22> Shell EOF
Fase4 23> Shell git add README.md docs/Cuestionario_equipo_README_20260504.md ;\
Fase4 24> Shell git status ;\
Fase4 25> Shell echo "" ;\
Fase4 26> Shell echo "--- Mensaje de commit ---" ;\
Fase4 27> Shell cat .git/commit_msgs/commit_v_01_readme.txt ;\
Fase4 28> Shell echo "" ;\
Fase4 29> Shell git commit -F .git/commit_msgs/commit_v_01_readme.txt ;\
Fase4 30> Shell ==>FASE 4: commit at 20260504221638<==
Fase4 31> Shell warning: in the working copy of 'README.md', LF will be replaced by CRLF the next
time Git touches it
Fase4 32> Shell warning: in the working copy of 'docs/Cuestionario_equipo_README_20260504.md', LF
will be replaced by CRLF the next time Git touches it
Fase4 33> Shell On branch v_01
Fase4 34> Shell Your branch is up to date with 'origin/v_01'.
Fase4 35> Shell
Fase4 36> Shell Changes to be committed:
Fase4 37> Shell (use "git restore --staged <file>..." to unstage)
Fase4 38> Shell modified: README.md
Fase4 39> Shell new file: docs/Cuestionario_equipo_README_20260504.md
Fase4 40> Shell
Fase4 41> Shell Untracked files:
Fase4 42> Shell (use "git add <file>..." to include in what will be committed)
Fase4 43> Shell .obsidian/
Fase4 44> Shell Fase4.txt
Fase4 45> Shell P8_Data_Analysis.kanban_debug.20260416102006J.sh
Fase4 46> Shell P8_Data_Analysis.kanban_download.20260416042954J.sh
Fase4 47> Shell "Sin t\303\255tulo.md"
Fase4 48> Shell issues/
Fase4 49> Shell uv.lock
Fase4 50> Shell
Fase4 51> Shell
Fase4 52> Shell --- Mensaje de commit ---
Fase4 53> Shell docs: propuesta inicial de README de proyecto y cuestionario de equipo
Fase4 54> Shell
Fase4 55> Shell - Reemplaza el README placeholder de la raiz por una propuesta de README de
Fase4 56> Shell proyecto que presenta el conjunto del proyecto P8 (equipo, stack,
Fase4 57> Shell contexto AirBnB 6 ciudades, enlaces a entregables externos) sin solapar
Fase4 58> Shell con la documentacion tecnica de cada modulo, que reside en sus propios
Fase4 59> Shell repositorios y bajo la responsabilidad de sus autores.
Fase4 60> Shell
Fase4 61> Shell - Anade docs/Cuestionario_equipo_README_20260504.md como insumo para
Fase4 62> Shell personalizar el README en la siguiente reunion de equipo.
Fase4 63> Shell
Fase4 64> Shell Propuesta sujeta a revision y aprobacion por parte del equipo.
Fase4 65> Shell

```

```

Fase4 66>_ Shell [v_01 59d11d1] docs: propuesta inicial de README de proyecto y cuestionario de
equipo
Fase4 67>_ Shell 2 files changed, 612 insertions(+), 1 deletion(-)
Fase4 68>_ Shell create mode 100644 docs/Cuestionario_equipo_README_20260504.md
Fase4 69>_ Shell 59d11d1 (HEAD -> v_01) docs: propuesta inicial de README de proyecto y
cuestionario de equipo
Fase4 70>_ Shell 0f3c487 (origin/v_01) docs+notebooks: aportes Data Viz Expert (Issue #2)
Fase4 71>_ Shell d684685 feat: añadir notebooks EDA y documentación (Issue #2 Data Viz Expert)
Fase4 72>_ Shell 5430fcd (origin/feature/v_05, origin/develop, develop) Upload .gitignore
Fase4 73>_ Shell 8a5f58f Librerías instaladas
Fase4 74>_ Shell
Fase4 75>_ Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/P8_Data_Analysis (v_01)
Fase4 76>_ Shell $
Fase4 77>_ Shell

Fase5 01>_ Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/P8_Data_Analysis (v_01)
Fase5 02>_ Shell $ # Fase 5 - Push a remoto
Fase5 03>_ Shell echo "==>FASE 5: push at $(date +%Y%m%d%H%M%S)<==" ;\
Fase5 04>_ Shell cd ~/Proyectos/P8_Data_Analysis ;\
Fase5 05>_ Shell git push origin v_01 ;\
Fase5 06>_ Shell echo "" ;\
Fase5 07>_ Shell git status
Fase5 08>_ Shell ==>FASE 5: push at 20260504222243<==
Fase5 09>_ Shell Enumerating objects: 8, done.
Fase5 10>_ Shell Counting objects: 100% (8/8), done.
Fase5 11>_ Shell Delta compression using up to 8 threads
Fase5 12>_ Shell Compressing objects: 100% (5/5), done.
Fase5 13>_ Shell Writing objects: 100% (5/5), 9.36 KiB | 4.68 MiB/s, done.
Fase5 14>_ Shell Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Fase5 15>_ Shell remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
Fase5 16>_ Shell To https://github.com/Bootcamp-IA-P6/P8_Data_Analysis.git
Fase5 17>_ Shell 0f3c487..59d11d1 v_01 -> v_01
Fase5 18>_ Shell
Fase5 19>_ Shell On branch v_01
Fase5 20>_ Shell Your branch is up to date with 'origin/v_01'.
Fase5 21>_ Shell
Fase5 22>_ Shell Untracked files:
Fase5 23>_ Shell (use "git add <file>..." to include in what will be committed)
Fase5 24>_ Shell .obsidian/
Fase5 25>_ Shell Fase4.txt
Fase5 26>_ Shell P8_Data_Analysis.kanban_debug.20260416102006J.sh
Fase5 27>_ Shell P8_Data_Analysis.kanban_download.20260416042954J.sh
Fase5 28>_ Shell "Sin t\303\255tulo.md"
Fase5 29>_ Shell issues/
Fase5 30>_ Shell uv.lock
Fase5 31>_ Shell
Fase5 32>_ Shell nothing added to commit but untracked files present (use "git add" to track)
Fase5 33>_ Shell
Fase5 34>_ Shell Coder@F5-LAPMAD-124 MINGW64 ~/Proyectos/P8_Data_Analysis (v_01)
Fase5 35>_ Shell $
Fase5 36>_ Shell

```

Resumen del orden

Fase 1: comprobar estado.
 Fase 2: estar en v_01 sincronizada.
 Fase 3: copiar archivos con los nombres correctos.
 Fase 4: commit con mensaje en archivo.
 Fase 5: push.

Hecho.

Algunos comandos eran redundantes, pero no observé riesgo (dentro de lo que sé).

¿Puedes dar una descripción para pasar por discord los enlaces raw
https://raw.githubusercontent.com/Bootcamp-IA-P6/P8_Data_Analysis/refs/heads/v_01/README.md
https://raw.githubusercontent.com/Bootcamp-IA-P6/P8_Data_Analysis/refs/heads/v_01/docs/Cuestionario_equipo_README_20260504.md
 README.md actual queda propuesto para ser reemplazado tras la reunión de mañana.
 ?

Gracias Claude desde vscode01.jj@gmail.com

20260504223320*

Hola JJ. Excelente trabajo: **las cinco fases han salido limpias**. El commit 59d11d1 está en origin/v_01 con +612 / -1 líneas, exactamente lo previsto. Antes de pasar al mensaje de Discord, tres comentarios cortos sobre lo que has observado:

Sobre la "redundancia" que detectaste – tienes razón. El git switch v_01 de la Fase 2 era innecesario porque ya estabas en v_01 desde la Fase 1, y el git pull era redundante porque el git fetch previo ya había sincronizado. No hay riesgo: git switch a la rama actual no hace nada, y git pull sin commits nuevos por traer simplemente dice "Already up to date." Tu instinto de que algo era redundante es correcto y demuestra que estás leyendo cada paso, no solo copiando.

Sobre los warnings LF will be replaced by CRLF – son normales y no preocupan. Git en Windows está configurado para convertir saltos de línea entre estilos Unix (LF) y Windows (CRLF) automáticamente. Los archivos en el repo se guardan con LF (estándar) y al hacer checkout en Windows se ven con CRLF. Solo es un aviso informativo.

Sobre el remote sandbox que apareció – alguien (tú probablemente, en algún momento anterior) configuró tu repo personal P8-Data-Analyst como remote secundario llamado sandbox. No molesta, pero conviene saber que está ahí: si en algún momento ejecutas git push sandbox ... por error, empujarías cosas a tu repo personal. Para verlo: git remote -v.

Sobre raw vs blob en Discord – me has pedido enlaces raw y te los doy como pides. Una nota útil por si quieres alternar: las URL blob/... (sin raw.githubusercontent.com) **renderizan el Markdown** en GitHub al hacer clic, lo cual suele ser más cómodo para revisión humana. La raw es texto plano. He preparado tres variantes del mensaje con la opción raw como pediste; si prefieres blob, basta sustituir el dominio.

Ahora el mensaje. Tres enfoques distintos según lo que quieras conseguir:

A Breve y transaccional

Hola equipo,

He subido a la rama v_01 del repo del equipo una propuesta inicial de README y un cuestionario para personalizarlo en la reunión de mañana.

README propuesto (raw):

https://raw.githubusercontent.com/Bootcamp-IA-P6/P8_Data_Analysis/refs/heads/v_01/README.md

Cuestionario para la reunión (raw):

https://raw.githubusercontent.com/Bootcamp-IA-P6/P8_Data_Analysis/refs/heads/v_01/docs/Cuestionario_equipo_README_20260504.md

El README actual de la raíz queda propuesto para ser reemplazado tras la reunión.

JJ

B Con contexto

Hola equipo,

He subido a la rama v_01 del repo del equipo dos documentos:

1) Una propuesta inicial de ****README**** para reemplazar el placeholder actual de la raíz. Es un documento de presentación general del proyecto P8 (equipo, stack, contexto AirBnB 6 ciudades, enlaces a entregables externos). No entra en la documentación técnica de cada módulo: eso corresponde a sus autores y vive en sus propios repos.

Raw: https://raw.githubusercontent.com/Bootcamp-IA-P6/P8_Data_Analysis/refs/heads/v_01/README.md

2) Un ****cuestionario en Markdown**** para personalizar el README en la reunión de mañana. Tiene bloques específicos para Naiza (frontend), Juanma (Power BI), Andy (EDA) y un espacio de comentarios libres para Pal.

Raw: https://raw.githubusercontent.com/Bootcamp-IA-P6/P8_Data_Analysis/refs/heads/v_01/docs/Cuestionario_equipo_README_20260504.md

El README de la raíz queda propuesto para ser reemplazado tras la reunión, con las respuestas que recojamos.

Cualquier comentario es bienvenido.

JJ

C Pedir review previo

Hola equipo,

Para aprovechar mejor el tiempo de la reunión de mañana, ¿podríais echar un vistazo previo a estos dos documentos que he subido a la rama v_01?

README propuesto para la raíz (sustituiría el placeholder actual):

https://raw.githubusercontent.com/Bootcamp-IA-P6/P8_Data_Analysis/refs/heads/v_01/README.md

Cuestionario que rellenaríamos juntos en la reunión:

https://raw.githubusercontent.com/Bootcamp-IA-P6/P8_Data_Analysis/refs/heads/v_01/docs/Cuestionario_equipo_README_20260504.md

El cuestionario tiene bloques específicos para cada uno: Naiza (frontend), Juanma (Power BI), Andy (EDA) y un espacio de comentarios libres para Pal. Si podéis ojearlo antes y traerlo medio respondido, mejor.

El README actual queda propuesto para ser reemplazado tras la reunión.

Gracias.

JJ

Tres opciones:

- **Breve y transaccional** – solo enlaces y qué son. Útil si Discord del equipo es de poca conversación y prefieres no inundar el canal.
- **Con contexto** – explica el alcance del README ("documento de presentación general, no técnico") para evitar el malentendido de "JJ está documentando el frontend de Naiza". Recomendable si en algún momento pudiera surgir esa duda.
- **Pedir review previo** – invita explícitamente a leer antes de la reunión. Es el más útil

estratégicamente porque mañana la reunión irá más rápida si llegan con el cuestionario medio leído. Mi recomendación, dado el contexto: "**Con contexto**" o "**Pedir review previo**". La transaccional se queda corta para alguien que no haya seguido el hilo contigo.

Si al pegar en Discord notas que las URL raw generan demasiado *clutter* visual, puedes envolverlas entre `< y >` para suprimir el preview, así: `<https://raw.githubusercontent.com/...>`.
markdown

Preguntas de Claude tras el push exitoso – 2026-05-04

P29: ¿Quieres que prepare ya un borrador de la PR de v_01 hacia develop (título + descripción), para tenerlo listo cuando el equipo apruebe el README, o esperamos a tener feedback de mañana?

R29:

P30: ¿Quieres que añadamos `Fase4.txt`, `Sin título.md` y los `.kanban\*.sh` al `.gitignore` del repo del equipo en otro commit aparte, para limpiar la lista de "Untracked files" de futuros `git status`? Es opcional pero higiénico.

R30:

P31: Mañana, cuando me traigas las respuestas del cuestionario, ¿prefieres que abramos un chat nuevo o seguimos en este?

R31:

Que descanses. Mañana lo retomamos con calma.

Este cuestionario constituye una propuesta en un intento de invitar en la elaboración colaborativa del README de forma coordinada y abierta a instrucciones/novedades en base a la contextualización realizada en Claude. JJ.
20260505074116*

JJ – ayer a las 15:28lunes, 4 de mayo de 2026 15:28 Hola, tengo un README.md. :thumbsup: Haz clic para reaccionar :slight_smile: Haz clic para reaccionar :heart: Haz clic para reaccionar Añadir reacción Editar Reenviar Más
[15:29]lunes, 4 de mayo de 2026 15:29 Tiene muchas cosas para corregir. :thumbsup: Haz clic para reaccionar :slight_smile: Haz clic para reaccionar :heart: Haz clic para reaccionar Añadir reacción Editar Reenviar Más
[15:29]lunes, 4 de mayo de 2026 15:29 Voy con ello y mañana lo hablamos. :thumbsup: Haz clic para reaccionar :slight_smile: Haz clic para reaccionar :heart: Haz clic para reaccionar Añadir reacción Editar Reenviar Más
[15:30]lunes, 4 de mayo de 2026 15:30 Puedo pasar lo que tengo ahora si fuera necesario. :thumbsup: Haz clic para reaccionar :slight_smile: Haz clic para reaccionar :heart: Haz clic para reaccionar Añadir reacción Editar Reenviar Más
[15:30]lunes, 4 de mayo de 2026 15:30 Voy a corregir datos.

JJ – ayer a las 22:44lunes, 4 de mayo de 2026 22:44
Hola equipo,

He subido a la rama v_01 del repo del equipo dos documentos:

1) Una propuesta inicial de README para reemplazar el placeholder actual de la raíz. Es un documento de presentación general del proyecto P8 (equipo, stack, contexto AirBnB 6 ciudades, enlaces a entregables externos). No entra en la documentación técnica de cada módulo: eso corresponde a sus autores y vive en sus propios repos.

Raw: https://raw.githubusercontent.com/Bootcamp-IA-P6/P8_Data_Analysis/refs/heads/v_01/README.md

2) Un cuestionario en Markdown para personalizar el README en la reunión de mañana. Tiene bloques específicos para Naiza (frontend), Juanma (Power BI), Andy (EDA) y un espacio de comentarios libres para Pal.

Raw: https://raw.githubusercontent.com/Bootcamp-IA-P6/P8_Data_Analysis/refs/heads/v_01/docs/Cuestionario_equipo_README_20260504.md

El README de la raíz queda propuesto para ser reemplazado tras la reunión, con las respuestas que recojamos.

Cualquier comentario es bienvenido.

JJ

Sinopsis:

He subido a la rama v_01 del repo del equipo una propuesta inicial de README y un cuestionario para personalizarlo en la reunión de mañana.

README propuesto (raw):
https://raw.githubusercontent.com/Bootcamp-IA-P6/P8_Data_Analysis/refs/heads/v_01/README.md

Cuestionario para la reunión (raw):
https://raw.githubusercontent.com/Bootcamp-IA-P6/P8_Data_Analysis/refs/heads/v_01/docs/Cuestionario_equipo_README_20260504.md

El README actual de la raíz queda propuesto para ser reemplazado tras la reunión.

JJ – 7:41martes, 5 de mayo de 2026 7:41
Este cuestionario constituye una propuesta en un intento de invitar en la elaboración colaborativa del README de forma coordinada y abierta a instrucciones/novedades en base a la contextualización realizada en Claude. JJ.
20260505074116*

:thumbsup:
Haz clic para reaccionar
:slight_smile:
Haz clic para reaccionar
:heart:
Haz clic para reaccionar
Añadir reacción
Editar
Reenviar
Más

Enviar mensaje a #ayuda-con-la-tarea

ayuda-con-la-tarea
<https://discord.com/channels/1493575099264012288/1493575100417572900>